Rust로 만드는 미니 GPT

2026-10-14

머리말

이 교재를 끝까지 따라가면, 외부 crate를 하나도 쓰지 않고 순수 Rust로 작성한 작은 GPT가 셰익스피어 문체의 문장을 생성하게 됩니다. 모든 코드는 10~40줄 정도의 조각으로 나뉘어 있고, 각 조각마다 "무엇을 하는가"와 "왜 이렇게 짜는가"를 함께 설명합니다.

누구를 위한 교재인가

- Rust 문법(struct, enum, impl, Vec, 소유권)은 알고 있는 분을 대상으로 합니다.
- 반복자 메서드(`map`, `zip`, `collect`)와 클로저를 읽을 수 있으면 더 좋습니다. lifetime처럼 낯선 문법은 등장할 때 설명합니다.
- 딥러닝과 transformer는 처음이어도 괜찮습니다. 수학은 사칙연산과 함수의 개념만 알면 충분합니다. 미분처럼 필요한 개념은 0부에서 숫자 예시로 설명하고, 본문의 유도는 0부의 도구만으로 진행합니다(0.3절 표에 정리한 기본 함수들의 미분 공식만은 유도하지 않고 숫자로 확인합니다).
- 행렬 곱셈은 "행과 열을 곱해서 더한다" 정도만 알면 됩니다. 필요한 수식은 등장할 때마다 다시 설명합니다.

진행 방식

교재는 수학 준비(0부)와 본문 다섯 부분으로 나뉘며, 앞부분에서 만든 코드를 뒷부분에서 그대로 재사용합니다.

1. **1부 스칼라 autograd**: 숫자 하나 단위로 자동 미분을 만들고, 작은 신경망을 학습시킵니다. 역전파의 원리를 이 단계에서 완전히 이해하는 것이 목표입니다.
2. **2부 Tensor autograd**: 다루는 단위를 숫자 하나에서 행렬로 바꿉니다. 1부와 구조는 같고, 연산 하나하나의 backward 공식만 새로 유도합니다.
3. **3부 데이터**: 텍스트를 숫자로 바꾸는 tokenizer와 학습용 배치를 만듭니다.
4. **4부 Transformer 부품**: embedding, layernorm, attention, MLP를 하나씩 만듭니다.
5. **5부 조립과 학습**: 부품을 GPT로 조립하고, 학습시킨 뒤 문장을 생성합니다.

각 코드 조각은 아래 형식을 따릅니다.

- **목표**: 이 조각이 끝나면 무엇이 가능해지는지 한 문장으로 적습니다.
- **코드**: 그대로 복사해서 붙여 넣으면 컴파일되는 코드입니다. 주석은 영어로 작성했습니다.
- **설명**: 줄 단위로 무엇을 하는지, 왜 그렇게 했는지 설명합니다.
- **확인**: 실행해서 눈으로 확인할 수 있는 결과가 있으면 함께 적습니다.

설계 원칙

- **외부 crate를 쓰지 않습니다.** 난수 생성기까지 직접 만듭니다. `Cargo.toml`에 dependency가 하나도 없으므로 버전 문제로 막힐 일이 없습니다.
- **속도보다 읽기 쉬움을 우선합니다.** 행렬 곱셈도 가장 단순한 삼중 반복문으로 작성합니다. 대신 모델을 작게 잡아서 CPU에서 몇 분 안에 학습이 끝나도록 했습니다.
- **계산 그래프는 tape 방식으로 만듭니다.** 모든 값을 하나의 `Vec`에 순서대로 쌓고, 노드끼리는 포인터 대신 인덱스(`usize`)로 서로를 가리킵니다. 이 방식을 쓰면 `Rc<RefCell<>>` 없이도 borrow checker 오류를 피하면서 그래프를 만들 수 있습니다.

준비물

1. Rust toolchain을 설치합니다. 이미 설치되어 있다면 `cargo --version`으로 확인만 하면 됩니다. 이 교재의 코드는 Rust 1.58 이상(edition 2021 이상)이 필요하며, Rust 1.94에서 검증했습니다.
2. 프로젝트를 만듭니다.

```
cargo new minigpt
cd minigpt
```

3. 5부에서 사용할 학습 데이터(셰익스피어 작품 약 1MB)를 미리 받아 둡니다.

```
curl -o input.txt https://raw.githubusercontent.com/karpathy/char-
rnn/master/data/tinyshakespeare/input.txt
```

이 파일은 Andrej Karpathy의 char-rnn 저장소(MIT License)에 포함된 tinyshakespeare 데이터셋으로, 셰익스피어 작품 일부를 이어 붙인 텍스트입니다. 셰익스피어의 원작은 저작권 보호 기간이 끝난 퍼블릭 도메인 저작물입니다. 이 교재는 파일을 직접 싣지 않으며, 독자가 원 저장소에서 받아 사용합니다. 이 교재가 안내하는 외부 파일과 저장소는 각 저작권자의 것이며 각자의 라이선스를 따릅니다. 링크는 집필 시점(2026년 10월) 기준이며, 이후 주소나 내용이 바뀔 수 있습니다.

4. 학습은 반드시 release 모드로 실행합니다. debug 모드는 release 모드보다 수십 배 느립니다.

```
cargo run --release
```

최종 파일 구성

교재를 끝내면 프로젝트는 아래 파일로 구성됩니다. 각 파일은 해당하는 부에서 처음부터 작성합니다.

파일	작성하는 곳	내용
`src/rng.rs`	1부, 2부, 3부	난수 생성기(필요할 때마다 함수를 하나씩 추가합니다)
`src/scalar.rs`	1부	숫자 하나 단위의 autograd와 작은 MLP
`src/tensor.rs`	2부	행렬 단위의 autograd, 연산 16개, gradient 검사 테스트
`src/data.rs`	3부	tokenizer와 배치 생성
`src/model.rs`	4부, 5부	Linear, LayerNorm, Attention, Mlp, Block, GPT 조립
`src/optim.rs`	5부	AdamW optimizer
`src/main.rs`	1부, 3부, 4부, 5부	실습용 진입점(`scalar`, `data`, `train` 명령을 하나씩 추가합니다)

0부 수학 준비

이 교재에 필요한 수학은 "입력을 아주 조금 바꾸면 출력이 얼마나 바뀌는가"를 계산하는 방법 하나뿐입니다. 0부는 그 방법과 몇 가지 기호를 숫자 예시로만 설명합니다. 본문의 유도는 여기서 배운 도구만 사용하고, 0.3절 표에 정리한 기본 함수들의 미분 공식은 유도하지 않고 숫자로 확인합니다.

0.1 미분 = 조금 바꿨을 때의 변화율

함수 f(x) = 3x + 1을 생각해 봅시다. x = 2일 때 f(2) = 7입니다. x를 0.01만큼 늘려 2.01로 만들면 f(2.01) = 7.03이 됩니다.

- 입력은 0.01 변했습니다.
- 출력은 0.03 변했습니다.
- 따라서 변화율은 0.03 ÷ 0.01 = 3입니다.

이 변화율을 **미분값**이라고 부르고, df/dx라고 씁니다. d는 "아주 작은 변화량"을 뜻하므로, df/dx는 "f의 변화량 ÷ x의 변화량"이라고 읽으면 됩니다.

직선이 아닌 함수도 같은 방식으로 계산합니다. f(x) = x²에서 x = 2를 2.01로 바꾸면 f는 4에서 4.0401이 됩니다. 변화율은 0.0401 ÷ 0.01 = 4.01입니다. 바꾸는 양을 0.0001처럼 더 작게 하면 이 값은 4에 점점 가까워집니다. 그래서 x = 2에서 x²의 미분값은 4입니다. x가 어떤 값이든 x²의 미분값은 2x입니다.

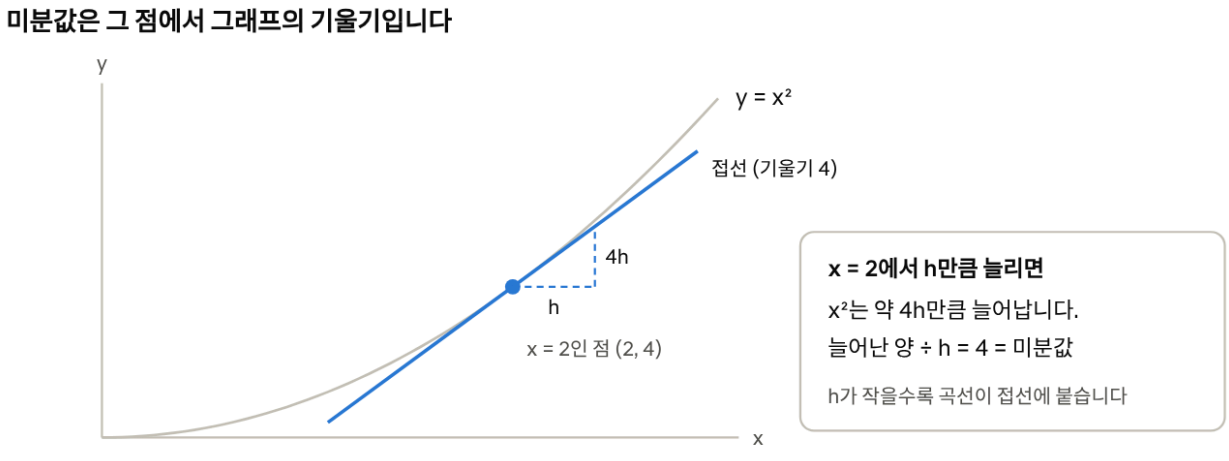


그림 0-1 · y = x²의 x = 2에서의 기울기

그림의 점선 삼각형이 위 계산과 같습니다. 밑변 h만큼 오른쪽으로 가면 높이가 약 4h 올라가므로, 기울기는 4입니다.

이 교재에서는 이렇게 **입력을 h만큼 늘리면 출력이 몇 × h만큼 늘어나는지**를 구하는 방법으로 모든 미분을 유도합니다. h는 0.01처럼 아주 작은 수라고 생각하면 됩니다. "몇"에 해당하는 값이 바로 미분값입니다.

0.2 e와 ln

다음 절의 표에 나오는 두 함수를 먼저 소개합니다.

- **e**는 약 2.718인 상수입니다. e^x는 항상 양수이고, x가 커지면 매우 빨리 커집니다($e^{10} \approx 22026$). x가 음수 쪽으로 작아지면(예: x = −10) e^x는 0에 가까워집니다.
- **ln**은 e^x를 되돌리는 함수입니다. $\ln(e^x) = x$이고, $\ln 1 = 0$이고, $\ln(1/a) = -\ln a$입니다(예: $\ln(1/65) = -\ln 65$). 0과 1 사이의 수에 ln을 취하면 음수가 되고, 0에 가까울수록 매우 큰 음수가 됩니다.
- 이 두 함수는 2.10절의 softmax(점수를 확률로 바꾸기)와 2.13절의 cross entropy(확률을 벌점으로 바꾸기)에서 사용합니다.

0.3 자주 쓰는 미분

아래 표의 공식은 외울 필요가 없습니다. 각 공식이 나오는 절에서 이 표로 돌아와 확인하면 됩니다. 마지막 열은 직접 계산해 본 값입니다.

함수	미분값	숫자로 확인 (입력을 0.01 늘렸을 때)
x + c (c는 상수)	1	입력이 0.01 늘면 출력도 0.01 늘어납니다.
c × x	c	3x는 0.03 늘어납니다.
x^2	2x	x = 2에서 변화율 4.01 ≈ 4
$\sqrt{x}$	$1 \div (2\sqrt{x})$	x = 4에서 변화율 0.2498 ≈ 1 ÷ 4
1/x	$-1 \div x^2$	x = 2에서 변화율 −0.2488 ≈ −1 ÷ 4
e^x	e^x (자기 자신)	x = 1에서 변화율 2.732, $e^1 = 2.718$
ln x	1/x	x = 2에서 변화율 0.499 ≈ 1/2
tanh(x)	$1 - \tanh(x)^2$	x = 0.5에서 변화율 0.783, $1 - \tanh(0.5)^2 = 0.786$
max(0, x) (ReLU)	x > 0이면 1, 아니면 0	양수 구간은 기울기 1인 직선, 음수 구간은 평평합니다.

tanh는 1부에서 쓰는 함수로, 어떤 실수든 −1과 1 사이로 눌러 줍니다. 미분 공식의 유도는 이 교재의 범위를 넘으므로 숫자 확인으로 대신합니다. 바꾸는 양을 0.01보다 작게 하면 표의 두 값(0.783과 0.786)은 더 가까워집니다.

0.4 입력이 여러 개일 때

신경망의 출력은 숫자 수만 개에 동시에 의존합니다. 이때는 **입력을 하나씩만 바꾸고, 나머지는 고정한 채로** 변화율을 구합니다.

예를 들어 f(a, b) = a × b에서 a = 2, b = −3이면 f = −6입니다.

- a만 2.01로 바꾸면 f = −6.03입니다. a에 대한 변화율은 $-0.03 \div 0.01 = -3$이고, 이는 b의 값과 같습니다.
- b만 −2.99로 바꾸면 f = −5.98입니다. b에 대한 변화율은 $0.02 \div 0.01 = 2$이고, 이는 a의 값과 같습니다.

이렇게 하나만 바꾼 변화율을 **편미분**이라고 부르고, d 대신 ∂("라운드"라고 읽습니다)를 써서 $\partial f/\partial a$처럼 표기합니다. 기호만 다를 뿐 계산 방법은 0.1절과 똑같습니다.

이 교재에서 **gradient**는 "최종 loss(모델이 틀린 정도를 나타내는 숫자, 1.1절)를 어떤 값으로 편미분한 값"을 뜻합니다. 즉 "그 값을 h만큼 늘리면 loss가 몇 × h만큼 변하는가"에서 "몇"에 해당하는 값입니다.

0.5 chain rule: 변화가 전달되는 방식

연산이 여러 단계로 이어져 있을 때는 변화가 단계마다 정해진 배율로 곱해지며 전달됩니다. $y = 3x$이고 $z = y^2$인 경우를 봅시다. $x = 1$이면 $y = 3$, $z = 9$입니다.

1. x를 h만큼 늘리면 y는 3h만큼 늘어납니다. ($y = 3x$의 미분값은 3)
2. y가 3h만큼 늘면 z는 $6 \times 3h = 18h$만큼 늘어납니다. (y^2의 $y = 3$에서의 미분값은 $2y = 6$)
3. 따라서 x에 대한 z의 변화율은 $3 \times 6 = 18$입니다.

숫자로 확인해 보면, $x = 1.01$일 때 $y = 3.03$이고 $z = 9.1809$입니다. 변화율은 $0.1809 \div 0.01 \approx 18.09$로, 18과 거의 같습니다.

이것이 **chain rule**입니다. 단계마다의 변화율을 모두 곱하면 전체 변화율이 됩니다. 역전파는 이 곱셈을 마지막 단계부터 거꾸로 수행합니다. z에서 시작해 y의 gradient 6을 먼저 구하고, 그 값에 3을 곱해 x의 gradient 18을 구합니다.

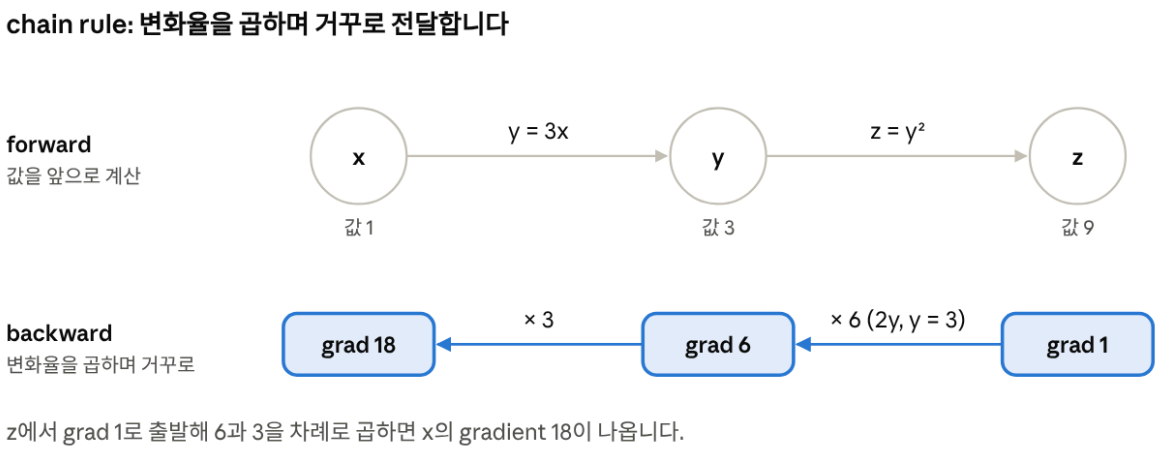


그림 0-2 · $y = 3x$, $z = y^2$의 forward와 backward

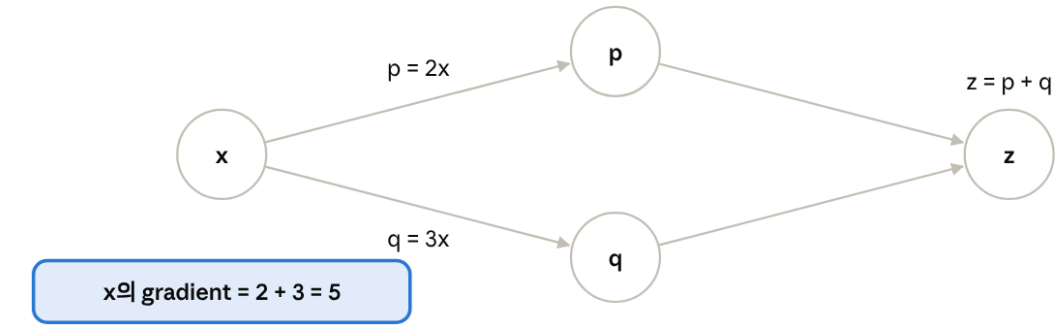
위쪽 줄은 값을 계산하는 forward이고, 아래쪽 줄은 gradient를 구하는 backward입니다. backward는 오른쪽 끝에서 grad 1로 출발해, 화살표를 하나 지날 때마다 그 단계의 변화율을 곱합니다.

0.6 경로가 여러 개이면 더한다

값 하나가 여러 곳에 쓰이면, 그 값을 바꿨을 때 여러 경로로 동시에 변화가 전달됩니다. 예를 들어 $p = 2x$, $q = 3x$, $z = p + q$라고 하겠습니다.

- x를 h 늘리면 p를 거쳐 z가 2h 늘어납니다.
- 동시에 q를 거쳐 z가 3h 늘어납니다.
- 따라서 z는 모두 합쳐 5h 늘어나고, 변화율은 2 + 3 = 5입니다.

경로별 변화율은 더합니다. 이 규칙 때문에 코드에서 gradient를 `=`가 아니라 `+=`로 쌓습니다.



x를 h 늘리면 p 경로로 2h, q 경로로 3h, 합쳐서 z가 5h 늘어납니다.

그림 0-3 · p = 2x, q = 3x, z = p + q

x에서 z로 가는 길이 두 개이므로, 두 길의 변화율 2와 3을 더합니다.

0.7 Σ 기호와 아래첨자

Σ(시그마)는 "모두 더하라"는 기호입니다. $\Sigma_k a_k$는 k를 1, 2, 3, …으로 바꿔 가며 $a_1 + a_2 + a_3 + \dots$을 더한 값입니다. Rust 코드로는 `for` 반복문 안에서 `sum +=` 하는 것과 같습니다.

아래첨자는 위치를 나타냅니다. A_{ik}는 행렬 A의 i행 k열 값이고, 코드로는 `a[i * cols + k]` 입니다. 예를 들어 A = [[1, 2], [3, 4]]이면 A_{12} = 2입니다(수식에서는 1부터, 코드에서는 0부터 셉니다).

0.8 작은 수 근사

h가 아주 작으면 h × h는 훨씬 더 작아서 무시할 수 있습니다. h = 0.01이면 h × h = 0.0001입니다. 이 사실에서 유도에 필요한 근사식 세 개가 나옵니다.

근사식	숫자로 확인 (a = b = h = 0.01)
(1 + a)(1 + b) ≈ 1 + a + b	1.01 × 1.01 = 1.0201 ≈ 1.02
1 ÷ (1 + a) ≈ 1 − a	1 ÷ 1.01 = 0.990099 ≈ 0.99
$e^{x+h} \approx e^x \times (1 + h)$	$e^{1.01}$ = 2.7456, e × 1.01 = 2.7455 (e^x의 미분이 e^x이므로, h만큼 늘면 e^x × h만큼 늘어납니다)

2부의 softmax와 layernorm 유도는 이 근사식과 0.3절의 표, 그리고 0.9절의 성질만으로 끝납니다.

0.9 평균, 분산, 표준편차

숫자 여러 개가 있을 때 그 숫자들이 "어디에 모여 있고 얼마나 퍼져 있는가"를 나타내는 값 세 가지입니다. x = [1, 2, 3, 6]을 예로 들겠습니다.

이름	계산 방법	예시
평균 μ	모두 더해 개수로 나눕니다	(1 + 2 + 3 + 6) ÷ 4 = 3
편차	각 값에서 평균을 뺍니다	[−2, −1, 0, 3]
분산 σ^2	편차를 제곱해 평균을 냅니다	(4 + 1 + 0 + 9) ÷ 4 = 3.5
표준편차 σ	분산의 제곱근입니다	√3.5 = 1.8708

편차를 모두 더하면 항상 0이 됩니다(−2 − 1 + 0 + 3 = 0). 평균보다 큰 쪽과 작은 쪽이 정확히 균형을 이루는 지점이 평균이기 때문입니다. 이 성질은 2.11절의 layernorm 유도에서 사용합니다.

정규분포는 평균 근처의 값이 가장 자주 나오고, 평균에서 멀어질수록 드물게 나오는 무작위 분포입니다. 표준편차가 σ라면 뽑은 값의 약 95%가 평균 ± 2σ 안에 들어갑니다. 예를 들어 평균 0, 표준편차 0.02인 정규분포에서 뽑은 값은 대부분 −0.04와 0.04 사이입니다. 4부에서는 가중치를 이 분포로 초기화합니다.

0.10 벡터와 내적

이 교재에서 **벡터**는 길이가 정해진 숫자 목록입니다. 행렬의 한 행도 벡터입니다.

길이가 같은 두 벡터를 칸끼리 곱해서 모두 더한 값을 **내적**이라고 합니다. [1, 2]와 [3, 4]의 내적은 1 × 3 + 2 × 4 = 11입니다. 행렬 곱셈의 각 칸도 "A의 한 행과 B의 한 열의 내적"입니다.

내적은 두 벡터가 얼마나 같은 방향을 가리키는지와 두 벡터의 크기를 함께 반영합니다. 아래 표처럼 크기가 1인 벡터끼리 비교하면 방향만 드러납니다.

두 벡터	관계	내적
[1, 0]과 [1, 0]	같은 방향	1
[1, 0]과 [0, 1]	직각	0
[1, 0]과 [−1, 0]	반대 방향	−1

4부의 attention은 이 성질을 이용해 "지금 찾는 정보와 가장 비슷한 위치"를 고릅니다.

1부 스칼라 autograd

1부가 끝나면 숫자 41개짜리 작은 신경망이 네 개의 예제를 학습하고, loss가 4.5에서 0.007로 떨어지는 것을 직접 보게 됩니다. 1부에서 만드는 "계산을 기록하고 거꾸로 미분한다"는 구조는 GPT까지 그대로 이어집니다. 그래서 1부는 교재 전체에서 가장 중요한 부분입니다.

1.1 미분을 왜 구하는가

신경망 학습은 "틀린 정도(loss)를 줄이는 방향으로 숫자들을 조금씩 움직이는 일"입니다. 이때 각 숫자를 어느 방향으로 얼마나 움직일지 알려 주는 것이 미분값, 즉 gradient입니다.

- gradient가 양수라면, 그 숫자를 키울 때 loss도 커집니다. 따라서 숫자를 줄여야 합니다.
- gradient가 음수라면 반대로 숫자를 키워야 합니다.
- gradient의 절댓값이 크다면, 그 숫자가 loss에 큰 영향을 주고 있다는 뜻입니다.

그래서 학습의 한 단계는 아래 식 하나로 요약됩니다. 여기서 lr(learning rate)은 한 번에 움직이는 폭을 정하는 작은 양수입니다.

$w \leftarrow w - lr \times \partial loss/\partial w$

문제는 숫자가 수백만 개일 때 미분을 어떻게 효율적으로 구하느냐입니다. 답은 **계산 과정을 전부 기록해 두었다가, 마지막 결과에서 처음 입력 쪽으로 거꾸로 한 번 훑으며 계산하는 것**입니다. 이 방법을 역전파(backpropagation)라고 부르고, 이것을 자동으로 해 주는 도구를 autograd라고 부릅니다.

1.2 난수 생성기 (`src/rng.rs`)

목표: 신경망의 초기값을 무작위로 정할 수 있게 됩니다. 같은 seed를 주면 항상 같은 결과가 나오므로, 이 교재에 적힌 출력과 여러분의 출력이 일치합니다.

```
/// A tiny pseudo-random number generator (xorshift64*).
/// Same seed -> same sequence, so every run is reproducible.
pub struct Rng {
    state: u64,
}

impl Rng {
    pub fn new(seed: u64) -> Self {
        // A zero state would stay zero forever, so avoid it.
        Rng { state: seed.max(1) }
    }

    /// Next raw 64-bit random number.
    pub fn next_u64(&mut self) -> u64 {
        let mut x = self.state;
        x ^= x >> 12;
        x ^= x << 25;
        x ^= x >> 27;
        self.state = x;
```

```
        x.wrapping_mul(0x2545_F491_4F6C_DD1D)
    }
```

- xorshift는 비트를 밀고(`>>`, `<<`) 섞는(`^=`) 연산만으로 난수처럼 보이는 수열을 만드는 가장 단순한 방법 중 하나입니다. 암호용으로는 쓸 수 없지만, 초기값을 무작위로 정하는 용도로는 충분합니다.
- 상태가 0이 되면 어떤 시프트와 XOR을 해도 0에 머물기 때문에 `seed.max(1)`로 0을 막습니다.
- 마지막에 큰 홀수를 곱하는 단계(`wrapping_mul`)는 상위 비트까지 골고루 섞이게 만드는 마무리 작업입니다. `wrapping_`은 오버플로가 일어나도 panic하지 않고 넘친 상위 비트를 버리라는 뜻입니다.

```
    /// Uniform float in [0, 1).
    pub fn uniform(&mut self) -> f64 {
        // Keep the top 53 bits: exactly the precision of an f64 mantissa.
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
}
```

- `uniform`은 64비트 정수를 0 이상 1 미만의 실수로 바꿉니다. f64가 정확하게 표현할 수 있는 자릿수가 53비트이므로 상위 53비트만 남기고 2^{53}으로 나눕니다.
- 마지막 줄의 `}`는 `impl Rng` 블록을 닫습니다. 2.6절과 3.3절에서 이 블록 안에 함수를 하나씩 더 추가합니다.

1.3 값을 가리키는 핸들과 연산의 종류 (`src/scalar.rs`)

목표: "값 하나"와 "그 값이 어떻게 만들어졌는가"를 표현하는 타입을 정의합니다.

```
use crate::rng::Rng;

/// Handle to one value on the tape. It is just an index, so it is Copy.
#[derive(Clone, Copy, Debug)]
pub struct V(pub usize);

/// How a value was produced. The inputs are indices of earlier values.
#[derive(Clone, Copy, Debug)]
enum Op {
    Leaf,
    Add(usize, usize),
    Mul(usize, usize),
    Tanh(usize),
}
```

- `V`는 값 자체가 아니라 값이 저장된 위치, 즉 인덱스입니다. 실제 숫자는 잠시 뒤에 만들 `Tape` 안에 들어 있습니다.
- `V`가 단순한 `usize`이므로 `Copy`를 붙일 수 있습니다. 그래서 `a`를 여러 연산에 넘겨도 소유권이 이동하지 않고, `&`나 `.clone()`을 붙일 필요도 없습니다. Rust로 계산 그래프를 만들 때 가장 까다로운 문제는 소유권입니다. 노드를 인덱스로 가리키면 이 문제가 생기지 않습니다.
- `Op`는 값이 만들어진 방법을 기록합니다. `Leaf`는 외부에서 직접 넣은 값(입력, 가중치, 상수)입니다. `Add(a, b)`는 "a번 값과 b번 값을 더해서 만들었다"는 뜻입니다.
- 연산은 덧셈, 곱셈, tanh 세 가지뿐입니다. 이 세 가지만 있어도 신경망 하나를 학습시키기에 충분합니다.

1.4 Tape: 모든 값을 순서대로 쌓는 곳

목표: 계산한 값을 하나씩 쌓아 두는 저장소를 만듭니다.

```
/// The tape: every value ever computed, in the order it was computed.
pub struct Tape {
    vals: Vec<f64>,
    grads: Vec<f64>,
    ops: Vec<Op>,
}

impl Tape {
    pub fn new() -> Self {
        Tape { vals: Vec::new(), grads: Vec::new(), ops: Vec::new() }
    }

    /// Append a value and return its handle.
    fn push(&mut self, val: f64, op: Op) -> V {
        self.vals.push(val);
        self.grads.push(0.0);
        self.ops.push(op);
        V(self.vals.len() - 1)
    }

    pub fn leaf(&mut self, x: f64) -> V {
        self.push(x, Op::Leaf)
    }

    pub fn val(&self, v: V) -> f64 {
        self.vals[v.0]
    }

    pub fn grad(&self, v: V) -> f64 {
        self.grads[v.0]
    }
```

- 세 개의 `Vec`은 같은 인덱스끼리 짝을 이룹니다. i번 값의 숫자는 `vals[i]`, i번 값에 대한 gradient는 `grads[i]`, i번 값을 만든 방법은 `ops[i]`에 있습니다.
- `push`는 세 `Vec`의 끝에 동시에 하나씩 추가하고, 방금 추가한 위치를 `V`로 돌려줍니다.
- 이 구조에는 중요한 성질이 하나 있습니다. **어떤 값의 입력은 항상 그 값보다 앞쪽 인덱스에 있습니다.** 연산을 하려면 입력이 이미 존재해야 하기 때문입니다. 1.6절의 역전파는 이 성질 덕분에 단순한 역순 반복문 하나로 끝납니다.

이 코드 조각은 `impl Tape {`로 시작하지만 닫는 `}`가 없습니다. 1.5절, 1.6절, 1.8절의 함수는 모두 이 블록 안에 들어가고, 1.8절 코드의 마지막 `}`가 블록을 닫습니다.

1.5 연산 만들기 (forward)

목표: 값을 계산하는 동시에 "어떻게 계산했는지"를 tape에 기록합니다.

```
    pub fn add(&mut self, a: V, b: V) -> V {
        let y = self.vals[a.0] + self.vals[b.0];
        self.push(y, Op::Add(a.0, b.0))
    }
```

```
    pub fn mul(&mut self, a: V, b: V) -> V {
        let y = self.vals[a.0] * self.vals[b.0];
        self.push(y, Op::Mul(a.0, b.0))
    }

    pub fn tanh(&mut self, a: V) -> V {
        let y = self.vals[a.0].tanh();
        self.push(y, Op::Tanh(a.0))
    }

    /// a - b, built from the ops we already have: a + (-1 * b).
    pub fn sub(&mut self, a: V, b: V) -> V {
        let minus_one = self.leaf(-1.0);
        let neg_b = self.mul(b, minus_one);
        self.add(a, neg_b)
    }
```

- 각 연산은 두 줄로 이루어져 있습니다. 첫 줄에서 값을 계산하고, 둘째 줄에서 그 값과 입력 정보를 함께 기록합니다.
- 이렇게 값을 앞에서부터 계산하는 과정을 **forward**라고 부릅니다.
- `tanh`는 어떤 실수든 –1과 1 사이로 부드럽게 눌러 주는 함수입니다. 신경망에 이런 비선형 함수가 없으면, 층을 아무리 쌓아도 결국 "곱하고 더하는" 일차 함수 하나와 같아집니다.
- `sub`는 새로운 `Op`를 만들지 않고, 이미 있는 덧셈과 곱셈을 조합해서 만듭니다. 새 연산을 조합으로 만들면 backward를 따로 작성할 필요가 없습니다.

1.6 역전파 (backward)

목표: 마지막 값 `out`에 대한 모든 값의 gradient를 한 번에 구합니다.

코드를 보기 전에 연산별 규칙부터 정리하겠습니다. y를 만든 연산이 무엇인지에 따라, y에 도착한 gradient g를 입력 쪽으로 나눠 보내는 방법이 다릅니다.

연산	a에 더해지는 값	b에 더해지는 값	이유
y = a + b	g	g	a가 1 늘면 y도 정확히 1 늘어납니다.
y = a × b	b × g	a × g	a가 1 늘면 y는 b만큼 늘어납니다.
y = tanh(a)	$(1 - y^2) \times g$	없음	0.3절의 표에 있는 tanh의 미분입니다.

표의 규칙은 0부에서 배운 내용을 그대로 옮긴 것입니다. "× g"는 0.5절의 chain rule이고, gradient를 `=`가 아니라 `+=`로 더하는 것은 0.6절의 "경로가 여러 개이면 더한다"입니다. 값 하나가 여러 연산에 쓰였다면 여러 경로의 gradient가 모두 더해집니다.

```
    /// Fill grads[i] = d(out)/d(value i) for every value on the tape.
    pub fn backward(&mut self, out: V) {
        self.grads.fill(0.0);
        self.grads[out.0] = 1.0;
        // Inputs always sit before outputs, so walking backwards
        // visits every node after all of its consumers.
        for i in (0..=out.0).rev() {
            let g = self.grads[i];
            match self.ops[i] {
                Op::Leaf => {}
```

```
                Op::Add(a, b) => {
                    self.grads[a] += g;
                    self.grads[b] += g;
                }
                Op::Mul(a, b) => {
                    self.grads[a] += self.vals[b] * g;
                    self.grads[b] += self.vals[a] * g;
                }
                Op::Tanh(a) => {
                    let t = self.vals[i];
                    self.grads[a] += (1.0 - t * t) * g;
                }
            }
        }
    }
```

- 먼저 모든 gradient를 0으로 지우고, 출력 자신의 gradient를 1로 둡니다. "out을 1 늘리면 out은 1 늘어난다"는 당연한 사실이 역전파의 출발점입니다.
- 그다음 `out`에서 0번까지 거꾸로 돌면서, 각 값에 모인 gradient `g`를 위 표의 규칙대로 입력 쪽에 나눠 줍니다.
- 거꾸로 도는 것만으로 순서가 맞는 이유는 1.4절에서 본 성질 때문입니다. i번 값을 입력으로 사용한 연산은 전부 i보다 뒤에 있습니다. 따라서 i에 도착했을 때는 i에 gradient를 보낼 연산들이 이미 모두 처리되었고, `grads[i]`는 완성된 값입니다. 일반적인 autograd 구현에서 필요한 위상 정렬(topological sort)을 tape 구조가 별도 작업 없이 해결합니다.
- `Tanh`에서는 도함수를 계산하기 위해 tanh를 다시 계산하지 않고, forward에서 저장해 둔 출력값 `vals[i]`를 재사용합니다.
- `self.grads[a]`를 수정하면서 `self.vals[b]`를 읽는 코드가 borrow checker를 통과하는 이유는, `grads`와 `vals`가 서로 다른 필드이기 때문입니다. 또 `a`와 `b`가 같은 값이어도(예: x × x) `+=`를 두 번 수행하므로 결과는 정확히 2x가 됩니다.

1.7 손으로 검산하기

목표: 작은 식 하나에서 backward가 손으로 구한 답과 같은지 확인합니다.

```
/// Check backward by hand on y = a * b + c.
fn hand_check() {
    let mut tape = Tape::new();
    let a = tape.leaf(2.0);
    let b = tape.leaf(-3.0);
    let c = tape.leaf(10.0);
    let ab = tape.mul(a, b);
    let y = tape.add(ab, c);
    tape.backward(y);
    // Expected: dy/da = b = -3, dy/db = a = 2, dy/dc = 1
    println!(
        "y = {}  dy/da = {}  dy/db = {}  dy/dc = {}",
        tape.val(y),
        tape.grad(a),
        tape.grad(b),
        tape.grad(c)
    );
}
```

`hand_check`는 `Tape`의 메서드가 아니라 독립된 함수입니다. 그래서 `impl Tape` 블록 바깥, 즉 1.8절 코드의 마지막 `}` 뒤에 둡니다.

이 코드가 실행되면 tape에는 값 다섯 개가 쌓입니다.

인덱스	이름	만든 방법	값	gradient
0	a	Leaf	2	−3
1	b	Leaf	−3	2
2	c	Leaf	10	1
3	ab	Mul(0, 1)	−6	1
4	y	Add(3, 2)	4	1

a = 2, b = −3은 0.4절에서 손으로 편미분했던 예와 같은 숫자입니다. backward는 이 표를 아래에서 위로 읽습니다. 4번(y)의 gradient 1이 덧셈을 거쳐 3번과 2번에 그대로 전달됩니다. 그다음 3번의 gradient 1이 곱셈을 거치면서, 0번(a)에는 상대편인 b의 값 −3이, 1번(b)에는 a의 값 2가 전달됩니다.

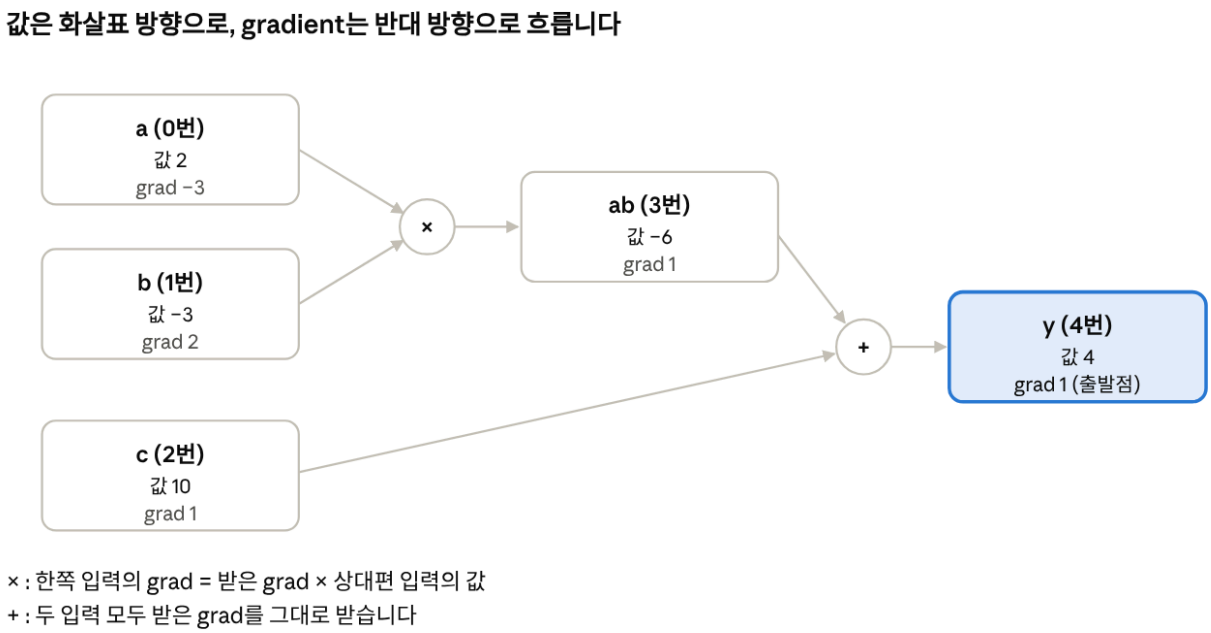


그림 1-1 · y = a × b + c를 기록한 tape

상자의 숫자는 위 표와 같습니다. 각 상자의 grad는 y에서 출발해 화살표를 거꾸로 따라온 값입니다.

확인: 1.10절까지 작성한 뒤 실행하면 첫 줄에 `y = 4  dy/da = −3  dy/db = 2  dy/dc = 1`이 출력됩니다.

1.8 파라미터만 남기고 지우기, 그리고 한 걸음 움직이기

목표: 학습을 반복할 수 있게 합니다.

```
    /// Drop everything after the first `len` values (keeps the parameters).
    pub fn truncate(&mut self, len: usize) {
        self.vals.truncate(len);
        self.grads.truncate(len);
```

```
        self.ops.truncate(len);
    }

    /// Gradient descent step on parameter `p`.
    pub fn step(&mut self, p: V, lr: f64) {
        self.vals[p.0] -= lr * self.grads[p.0];
    }
}
```

학습은 "forward → backward → 파라미터 수정"을 수백 번 반복합니다. 그런데 forward를 할 때마다 tape가 계속 길어지면 메모리가 끊임없이 늘어납니다. 그래서 이 교재는 다음 규칙을 사용합니다.

1. 학습 대상인 파라미터(가중치와 bias)를 **tape의 맨 앞에 먼저** 만듭니다.
2. 학습 단계마다 `truncate(n_params)`로 파라미터 뒤쪽만 지웁니다.
3. 파라미터의 인덱스는 바뀌지 않으므로, 앞에서 받아 둔 `V` 핸들을 계속 쓸 수 있습니다.

`step`은 1.1절의 식을 그대로 코드로 옮긴 것입니다. 이 방식을 경사하강법(gradient descent)이라고 부릅니다.

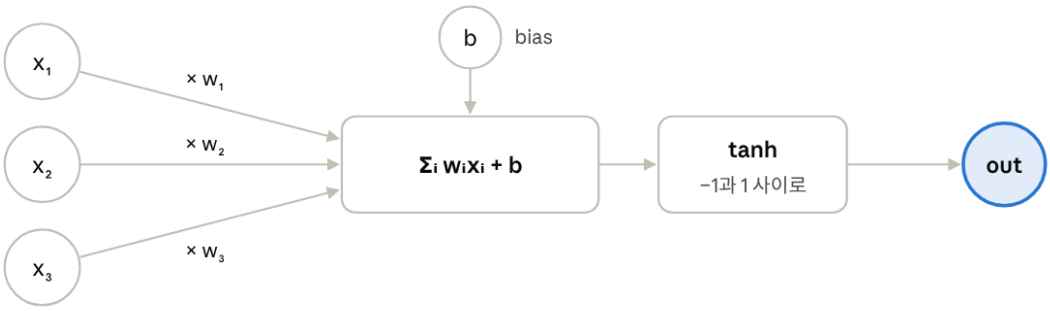
1.9 Neuron, Layer, Mlp

목표: tape 위에 작은 신경망을 쌓습니다.

뉴런 하나는 입력마다 가중치를 곱해 모두 더하고, bias를 더한 뒤 tanh를 적용합니다.

out = tanh(Σ_i $w_i x_i$ + b)

뉴런 하나: 곱하고, 모두 더하고, tanh로 누릅니다



입력이 3개이면 이 뉴런 하나가 tape에 값 7개(곱셈 3, 덧셈 3, tanh 1)를 기록합니다.

그림 1-2 · 입력이 3개인 뉴런

w_1, w_2, w_3와 b는 학습으로 바뀌는 파라미터이고, x_1, x_2, x_3는 바깥에서 들어오는 입력입니다.

```
/// One neuron: tanh(w . x + b).
pub struct Neuron {
    w: Vec<V>,
    b: V,
}

impl Neuron {
    pub fn new(tape: &mut Tape, n_in: usize, rng: &mut Rng) -> Self {
        let w = (0..n_in)
            .map(|_| tape.leaf(rng.uniform() * 2.0 - 1.0))
```

```
            .collect();
        let b = tape.leaf(0.0);
        Neuron { w, b }
    }

    pub fn forward(&self, tape: &mut Tape, x: &[V]) -> V {
        let mut sum = self.b;
        for (wi, xi) in self.w.iter().zip(x) {
            let p = tape.mul(*wi, *xi);
            sum = tape.add(sum, p);
        }
        tape.tanh(sum)
    }

    pub fn params(&self) -> Vec<V> {
        let mut p = self.w.clone();
        p.push(self.b);
        p
    }
}
```

- `Neuron`은 숫자를 직접 들고 있지 않고, tape 안의 위치(`V`)만 들고 있습니다.
- 가중치는 −1에서 1 사이의 무작위 값으로, bias는 0으로 시작합니다. 가중치를 전부 0으로 두면 모든 뉴런이 똑같은 gradient를 받아 똑같이 변하므로, 뉴런을 여러 개 둔 의미가 없어집니다.
- `forward`는 곱셈과 덧셈을 하나씩 tape에 기록합니다. 입력이 3개라면 이 함수 하나가 tape에 값 7개(곱셈 3, 덧셈 3, tanh 1)를 추가합니다.

```
/// A fully connected layer: `n_out` neurons that all see the same input.
pub struct Layer {
    neurons: Vec<Neuron>,
}

impl Layer {
    pub fn new(
        tape: &mut Tape,
        n_in: usize,
        n_out: usize,
        rng: &mut Rng,
    ) -> Self {
        Layer {
            neurons: (0..n_out)
                .map(|_| Neuron::new(tape, n_in, rng))
                .collect(),
        }
    }

    pub fn forward(&self, tape: &mut Tape, x: &[V]) -> Vec<V> {
        self.neurons.iter().map(|n| n.forward(tape, x)).collect()
    }

    pub fn params(&self) -> Vec<V> {
        self.neurons.iter().flat_map(|n| n.params()).collect()
    }
}
```

- `Layer`는 같은 입력을 보는 뉴런 여러 개입니다. 뉴런 수만큼 출력이 나옵니다.

- `params`는 모든 뉴런의 파라미터를 하나의 목록으로 이어 붙입니다. `flat_map`은 "각 원소를 목록으로 바꾼 뒤 모두 이어 붙이기"를 한 번에 하는 반복자 메서드입니다.

```
/// Layers stacked one after another.
pub struct Mlp {
    layers: Vec<Layer>,
}

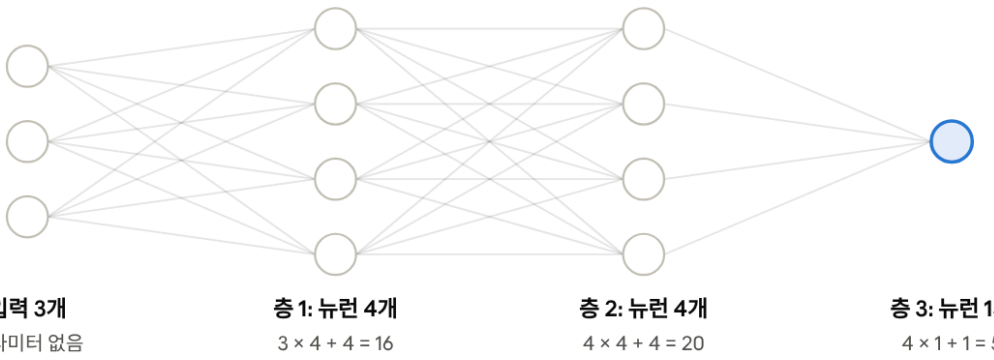
impl Mlp {
    /// sizes = [inputs, hidden1, ..., outputs]
    pub fn new(tape: &mut Tape, sizes: &[usize], rng: &mut Rng) -> Self {
        let layers = sizes
            .windows(2)
            .map(|w| Layer::new(tape, w[0], w[1], rng))
            .collect();
        Mlp { layers }
    }

    pub fn forward(&self, tape: &mut Tape, x: &[V]) -> Vec<V> {
        let mut h = x.to_vec();
        for layer in &self.layers {
            h = layer.forward(tape, &h);
        }
        h
    }

    pub fn params(&self) -> Vec<V> {
        self.layers.iter().flat_map(|l| l.params()).collect()
    }
}
```

- `Mlp`(Multi-Layer Perceptron, 다층 퍼셉트론)는 층을 차례로 쌓습니다. 앞 층의 출력이 뒤 층의 입력이 됩니다.
- `sizes.windows(2)`는 `[3, 4, 4, 1]`을 `[3, 4]`, `[4, 4]`, `[4, 1]`처럼 이웃한 두 개씩 묶어 줍니다. 각 묶음이 층 하나의 입력 수와 출력 수가 됩니다.
- 파라미터 수는 (3 × 4 + 4) + (4 × 4 + 4) + (4 × 1 + 1) = 41개입니다.

Mlp [3, 4, 4, 1]: 파라미터는 16 + 20 + 5 = 41개입니다



층마다 (입력 수 × 뉴런 수)개의 가중치와, 뉴런 수만큼의 bias가 있습니다.

그림 1-3 · sizes = [3, 4, 4, 1]인 Mlp

선 하나가 가중치 하나입니다. 예를 들어 층 1로 들어가는 선은 3 × 4 = 12개이고, 뉴런마다 bias가 하나씩 있으므로 층 1의 파라미터는 16개입니다.

1.10 학습 루프

목표: 네 개의 입력에 대해 정해진 답(1, −1, −1, 1)을 내도록 MLP를 학습시킵니다.

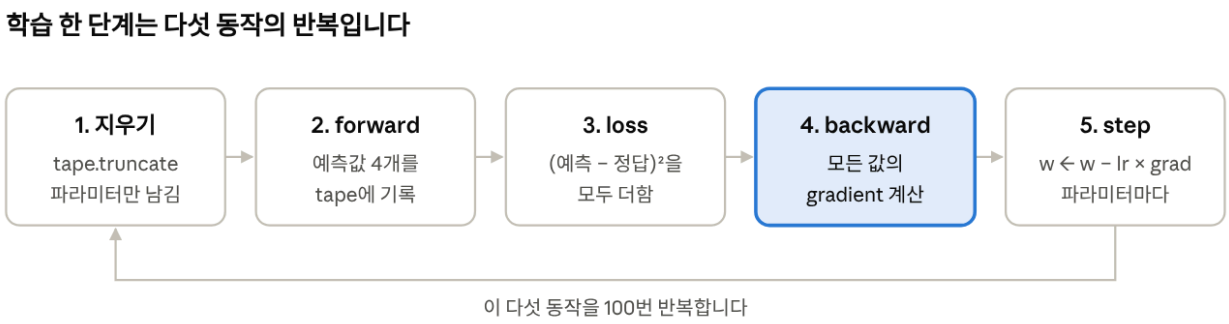


그림 1-4 · 1부 학습 루프 한 바퀴

1.8절의 truncate와 step, 1.6절의 backward가 이 고리 안에서 단계마다 한 번씩 쓰입니다.

```
/// Train a 3-4-4-1 MLP on four hand-made examples.
pub fn demo() {
    hand_check();

    let xs = [
        [2.0, 3.0, -1.0],
        [3.0, -1.0, 0.5],
        [0.5, 1.0, 1.0],
        [1.0, 1.0, -1.0],
    ];
    let ys = [1.0, -1.0, -1.0, 1.0];

    let mut rng = Rng::new(42);
    let mut tape = Tape::new();
    let mlp = Mlp::new(&mut tape, &[3, 4, 4, 1], &mut rng);
    let params = mlp.params();
    let n_params = params.len(); // parameters occupy tape[0..n_params]
    println!("parameters: {n_params}");

    for step in 0..100 {
        // Forget last step's graph, keep the parameters.
        tape.truncate(n_params);

        // Forward: sum of squared errors over the four examples.
        let mut loss = tape.leaf(0.0);
        let mut preds = Vec::new();
        for (x, y) in xs.iter().zip(ys) {
            let x: Vec<V> = x.iter().map(|&v| tape.leaf(v)).collect();
            let pred = mlp.forward(&mut tape, &x)[0];
            let target = tape.leaf(y);
            let diff = tape.sub(pred, target);
            let sq = tape.mul(diff, diff);
            loss = tape.add(loss, sq);
            preds.push(pred);
```

```
        }

        // Backward, then nudge every parameter against its gradient.
        tape.backward(loss);
        for &p in &params {
            tape.step(p, 0.05);
        }

        if step % 10 == 0 || step == 99 {
            let p: Vec<String> = preds
                .iter()
                .map(|&v| format!("{:+.3}", tape.val(v)))
                .collect();
            println!(
                "step {step:3}  loss {:.4}  preds [{}]",
                tape.val(loss),
                p.join(", ")
            );
        }
    }
}
```

- **loss**는 예측값과 정답의 차이를 제곱해 모두 더한 값입니다. 제곱을 하면 차이가 음수든 양수든 항상 양수가 되고, 크게 틀릴수록 더 큰 벌점을 받습니다.
- 제곱은 별도 연산 없이 `mul(diff, diff)`로 구합니다. 1.6절에서 설명한 대로, 같은 값을 두 번 사용해도 gradient는 정확히 계산됩니다.
- `preds`에 저장한 예측값은 `step` 이후에 출력하지만, `step`은 파라미터만 바꾸고 이미 계산된 예측값은 바꾸지 않습니다. 따라서 출력되는 값은 이번 단계의 forward 결과입니다.

`src/main.rs`를 아래처럼 작성합니다. 뒷부분에서도 이 `match`에 명령을 하나씩 추가합니다.

```
mod rng;
mod scalar;

fn main() {
    let cmd = std::env::args().nth(1).unwrap_or_default();
    match cmd.as_str() {
        "scalar" => scalar::demo(),
        _ => println!("usage: cargo run --release -- scalar"),
    }
}
```

확인: `cargo run --release -- scalar`를 실행하면 아래와 같이 출력됩니다. loss가 줄어들면서 예측값이 정답 [1, −1, −1, 1]에 가까워집니다.

```
y = 4  dy/da = -3  dy/db = 2  dy/dc = 1
parameters: 41
step   0  loss 4.5269  preds [-0.363, -0.648, -0.264, -0.416]
step  10  loss 0.1708  preds [+0.812, -0.842, -0.722, +0.819]
step  20  loss 0.0545  preds [+0.892, -0.906, -0.849, +0.895]
...
step  99  loss 0.0074  preds [+0.960, -0.962, -0.947, +0.960]
```

여기까지 만든 것이 autograd의 전부입니다. PyTorch도 구조는 같고, 단지 값 하나 대신 행렬 하나를 다룹니다. 2부에서는 그 차이를 메웁니다.

2부 Tensor autograd

2부가 끝나면 행렬 연산 16종류의 forward와 backward를 모두 갖춘 autograd가 완성되고, `cargo test`로 모든 backward가 정확한지 자동으로 검사할 수 있게 됩니다. 구조는 1부와 완전히 같고, "값 하나"가 "행렬 하나"로 바뀝니다.

2.0 앞으로 만들 GPT와 필요한 연산

2부에서는 연산 16개를 만듭니다. 각 연산이 어디에 쓰이는지 모르고 만들면 지루하고 헷갈리므로, 먼저 완성할 GPT의 구조를 간단히 살펴보겠습니다.

GPT가 하는 일은 하나입니다. **지금까지의 글자를 보고 다음 글자를 맞히는 것**입니다(자세한 내용은 3.1절). 이를 위해 GPT는 글자를 숫자 벡터로 바꾸고, 그 벡터들을 여러 단계에 걸쳐 고친 뒤, 마지막에 "다음 글자가 각 문자일 점수"를 냅니다(구조 그림은 4.1절). 아래 표는 2부의 연산이 그 과정의 어디에 쓰이는지 정리한 것입니다.

연산	만드는 절	GPT에서 쓰이는 곳
`matmul`	2.5	모든 Linear 층, attention의 점수 계산
`add`, `add_row`, `mul_row`, `scale`	2.7	residual 연결, bias, layernorm의 배율, attention 점수 조절
`relu`	2.8	MLP 안의 비선형 함수
`transpose`, `slice_rows`, `slice_cols`, `concat_rows`, `concat_cols`	2.9	attention에서 문장과 head를 나누고 다시 합치기
`causal_mask`, `softmax`	2.10	attention에서 미래 글자 가리기, 점수를 비중으로 바꾸기
`layer_norm`	2.11	각 block의 입력 정규화
`embedding`	2.12	글자 번호와 위치 번호를 벡터로 바꾸기
`cross_entropy`	2.13	예측이 얼마나 틀렸는지 재는 loss

표의 용어는 지금 몰라도 됩니다. 4부에서 하나씩 다시 설명합니다. 2.6절에서는 연산이 아닌 검사 도구를 만듭니다. 이 도구가 있으면 각 연산을 만든 직후 바로 backward가 맞는지 확인할 수 있습니다.

2.1 무엇이 달라지는가

1부의 신경망은 곱셈 하나마다 값을 하나씩 tape에 기록했습니다. GPT는 파라미터가 수십만 개이므로, 같은 방식을 쓰면 한 번의 forward에 수억 개의 값이 쌓입니다. 그래서 이제부터는 행렬 전체를 하나의 노드로 다룹니다.

항목	1부 (스칼라)	2부 (Tensor)
노드 하나가 담는 것	f64 하나	rows × cols 크기의 f32 행렬
핸들	`V(usize)`	`T(usize)`
저장소	`Tape`	`Graph`
backward가 입력에 보내는 것	숫자 하나	입력과 같은 크기의 gradient 행렬
연산 종류	3개	16개

숫자는 메모리와 속도를 위해 f64 대신 f32를 씁니다. 모든 tensor는 2차원(행과 열)으로만 다룹니다. PyTorch는 batch(여러 문장을 한 번에 처리하는 묶음, 3.3절)와 head(4.8절)를 위해 3차원과 4차원 tensor를 쓰지만, 이 교재는 그 차원을 Rust의 `for` 반복문으로 풀어서 처리합니다. 속도는 느려지지만, 모든 연산이 "행렬 하나를 받아 행렬 하나를 돌려주는" 형태로 통일되어 훨씬 읽기 쉽습니다.

행렬은 `Vec<f32>` 하나에 **행 우선(row-major)** 순서로 저장합니다. 즉 0행의 값을 전부 놓고, 그 뒤에 1행의 값을 이어서 놓습니다. 그래서 (r, c) 위치의 값은 `data[r * cols + c]`에 있습니다. 예를 들어 2 × 3 행렬 `[[1, 2, 3], [4, 5, 6]]`은 `[1, 2, 3, 4, 5, 6]`으로 저장되고, (1, 0) 위치의 4는 `data[1 * 3 + 0]`에 있습니다. 2부의 코드는 이 인덱스 계산을 계속 반복하므로 여기서 익숙해지면 좋습니다.



그림 2-1 · 2 × 3 행렬의 행 우선 저장

2부의 모든 연산은 이 규칙으로 (r, c) 위치를 data의 인덱스로 바꿔서 읽고 씁니다.

2.2 T, Op, Node, Graph (`src/tensor.rs`)

목표: tensor를 담는 노드와 저장소를 정의합니다.

2부의 코드는 모두 `src/tensor.rs` 한 파일에 들어갑니다. 파일은 아래 순서로 채워 나갑니다. Rust는 함수가 선언된 순서를 따지지 않으므로, 같은 블록 안에서는 순서를 바꿔도 됩니다.

1. `T`, `Op`, `Node`, `Graph` 정의 (2.2절)
2. `impl Graph { ... }` 블록: 기본 기능(2.3절), `backward`와 `input_grads`(2.4절), 그리고 2.5절부터 각 연산의 forward 함수와 `back_` 함수
3. `impl Graph` 블록 바깥의 도우미 함수 `softmax_in_place`(2.10절)와 `mean_rstd`(2.11절)
4. 파일 맨 끝의 테스트 모듈 `#[cfg(test)] mod tests { ... }` (2.6절부터 각 절에서 테스트 추가)

`src/main.rs`의 맨 위에는 `mod tensor;`를 추가합니다. 2부부터 4부까지는 `main.rs`가 `Graph`의 함수 대부분을 아직 호출하지 않으므로, 빌드할 때 "never used" 경고가 많이 출력됩니다. 이 경고는 5부에서 모델을 학습시키면 사라지므로 무시해도 됩니다.

```
/// Handle to one tensor on the graph (an index, like `V` in part 1).
#[derive(Clone, Copy, Debug)]
pub struct T(pub usize);

/// How a tensor was produced. Inputs are indices of earlier tensors.
#[derive(Clone, Debug)]
enum Op {
    Leaf,
    MatMul(usize, usize),
    Add(usize, usize),
    AddRow(usize, usize),
    MulRow(usize, usize),
    Scale(usize, f32),
    Relu(usize),
    Transpose(usize),
    SliceRows(usize, usize),
    SliceCols(usize, usize),
    ConcatRows(Vec<usize>),
    ConcatCols(Vec<usize>),
    CausalMask(usize),
    Softmax(usize),
    LayerNorm(usize),
    Embedding(usize, Vec<usize>),
    CrossEntropy(usize, Vec<usize>),
}

/// One 2-D tensor, stored row-major: element (r, c) is data[r * cols + c].
struct Node {
    data: Vec<f32>,
    grad: Vec<f32>,
    rows: usize,
    cols: usize,
    op: Op,
}

/// The tape of tensors. Same idea as `Tape` in part 1.
pub struct Graph {
    nodes: Vec<Node>,
}
```

- `Op`의 각 항목은 이 절 뒤에서 하나씩 만듭니다. 입력 인덱스 외에 추가 정보가 필요한 연산도 있습니다. `Scale`은 곱할 상수를, `SliceRows`는 잘라낸 시작 위치를, `Embedding`과 `CrossEntropy`는 토큰 번호 목록을 함께 기억합니다.
- `Vec`을 담는 항목이 생겼기 때문에 1부와 달리 `Op`는 `Copy`가 아닙니다. 그래서 backward에서는 `match &node.op`처럼 참조로 꺼냅니다.
- 1부에서는 값, gradient, 연산을 세 개의 `Vec`에 나눠 담았지만, 여기서는 행렬의 크기까지 함께 다루어야 하므로 `Node` 하나에 묶었습니다.

2.3 Graph의 기본 기능

목표: 노드를 추가하고, 크기와 값을 읽고, 파라미터만 남기고 지우는 기능을 만듭니다.

```
impl Graph {
    pub fn new() -> Self {
        Graph { nodes: Vec::new() }
    }

    fn push(
        &mut self,
        data: Vec<f32>,
        rows: usize,
        cols: usize,
        op: Op,
    ) -> T {
        assert_eq!(
            data.len(),
            rows * cols,
            "data length must equal rows * cols"
        );
        let grad = vec![0.0; data.len()];
        self.nodes.push(Node { data, grad, rows, cols, op });
        T(self.nodes.len() - 1)
    }

    pub fn leaf(&mut self, data: Vec<f32>, rows: usize, cols: usize) -> T {
        self.push(data, rows, cols, Op::Leaf)
    }

    pub fn shape(&self, t: T) -> (usize, usize) {
        (self.nodes[t.0].rows, self.nodes[t.0].cols)
    }

    pub fn data(&self, t: T) -> &[f32] {
        &self.nodes[t.0].data
    }

    #[allow(dead_code)] // only the tests read gradients directly
    pub fn grad(&self, t: T) -> &[f32] {
        &self.nodes[t.0].grad
    }

    /// Mutable values and read-only gradient of one tensor,
    /// for the optimizer.
    pub fn data_and_grad(&mut self, t: T) -> (&mut [f32], &[f32]) {
        let n = &mut self.nodes[t.0];
        (&mut n.data, &n.grad)
    }

    pub fn len(&self) -> usize {
        self.nodes.len()
    }

    pub fn truncate(&mut self, len: usize) {
        self.nodes.truncate(len);
    }
```

- `push`의 `assert_eq!`는 데이터 길이와 크기가 맞지 않는 실수를 즉시 잡아냅니다. 행렬 코드에서는 크기가 어긋난 채로 계산이 진행되면 원인을 찾기가 매우 어렵기 때문에, 이런 검사를 가능한 한 일찍 둡니다.
- `data_and_grad`는 값은 수정하고 gradient는 읽기만 하는 참조를 동시에 돌려줍니다. 같은 `Node`의 서로 다른 필드이므로 borrow checker가 허용합니다. 5부의 optimizer가 이 함수를 사용합니다.
- `grad`는 2.6절부터 만드는 테스트에서만 사용하므로, 일반 빌드에서 나오는 "never used" 경고를 `#[allow(dead_code)]`로 끕니다.
- `truncate`는 1부와 같은 역할을 합니다. 파라미터를 그래프의 맨 앞에 만들어 두고, 학습 단계마다 그 뒤를 지웁니다.
- 이 코드 조각은 `impl Graph {`로 시작하지만 닫는 `}`가 없습니다. 조각 끝에 `}`를 붙여 블록을 닫아 두고, 이후 절에서 만드는 함수는 모두 이 블록 안에 추가합니다.

2.4 backward의 뼈대

목표: 연산별 backward 함수를 호출하는 공통 구조를 만듭니다.

```
    /// Fill every tensor's grad with d(loss)/d(tensor).
    /// `loss` must be 1 x 1.
    pub fn backward(&mut self, loss: T) {
        assert_eq!(
            self.shape(loss),
            (1, 1),
            "backward: loss must be a 1 x 1 tensor"
        );
        for node in self.nodes.iter_mut() {
            node.grad.fill(0.0);
        }
        self.nodes[loss.0].grad[0] = 1.0;
        for i in (0..=loss.0).rev() {
            for (input, g) in self.input_grads(i) {
                for (dst, src) in self.nodes[input].grad.iter_mut().zip(&g)
                {
                    *dst += src;
                }
            }
        }
    }

    /// For node i, return pairs of
    /// (input index, gradient to add to that input).
    fn input_grads(&self, i: usize) -> Vec<(usize, Vec<f32>)> {
        let node = &self.nodes[i];
        let dy = &node.grad;
        match &node.op {
            Op::Leaf => vec![],
            // Each later section adds one line here.
            _ => todo!("backward is not written yet for {:?}", node.op),
        }
    }
```

`backward`는 1부와 흐름이 같습니다. gradient를 0으로 지우고, loss의 gradient를 1로 두고, 뒤에서부터 앞으로 갑니다. 다른 점은 gradient를 계산하는 일과 더하는 일을 두 단계로 나눈 것입니다.

1. **계산**: `input_grads(i)`는 `&self`만 빌려서 각 입력에 더해야 할 gradient를 새 `Vec`에 담아 돌려줍니다.
2. **누적**: 그다음 `backward`가 `&mut self`로 각 입력의 `grad`에 더합니다.

두 단계로 나눠야 하는 이유는 borrow checker 때문입니다. 행렬 곱셈의 backward는 "B의 값을 읽으면서 A의 gradient를 수정"해야 합니다. A와 B는 모두 `self.nodes` 안에 있으므로, 같은 `Vec`에서 하나는 읽고 하나는 쓰는 참조를 동시에 만들 수 없습니다. 게다가 `x @ x`처럼 A와 B가 같은 노드일 수도 있습니다. 먼저 새 `Vec`에 계산해 두고 나중에 더하면 이 문제가 모두 사라집니다. 메모리 할당이 조금 늘어나는 대신 코드가 안전하고 단순해집니다.

`input_grads`는 `node.op`의 종류에 따라 연산별 backward 함수(`back_`으로 시작하는 함수)를 호출합니다. 지금은 아직 만든 연산이 없으므로 `Leaf` 한 줄만 있습니다. 2.5절부터는 연산을 하나 만들 때마다 이 `match`에 한 줄씩 추가합니다. 마지막 줄의 `_ => todo!(...)`는 "아직 backward를 작성하지 않은 연산"을 만나면 그 연산의 이름을 출력하고 멈추게 합니다. 덕분에 연산을 모두 만들기 전에도 코드가 항상 컴파일됩니다. 완성된 `match`는 2.14절에서 한 번에 보여 드립니다.

이어지는 절에서는 다음 이름을 계속 사용합니다.

- `dy` : 연산 결과(y)에 도착한 gradient입니다. y와 크기가 같습니다.
- `da`, `db` : 입력 a와 b에 더해질 gradient입니다. 각각 a, b와 크기가 같습니다.

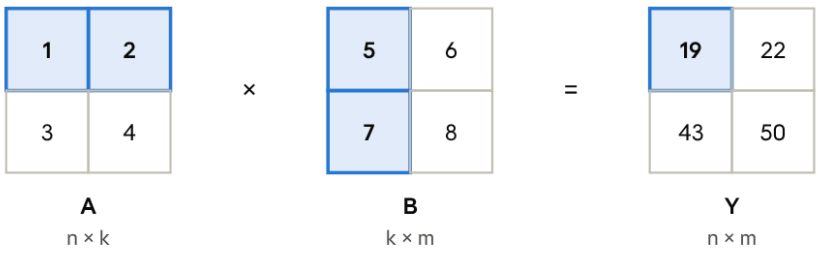
2.5 행렬 곱셈 (matmul)

목표: 신경망 계산의 대부분을 차지하는 행렬 곱셈을 만듭니다.

(n × k) 행렬 A와 (k × m) 행렬 B를 곱하면 (n × m) 행렬 Y가 나옵니다. Y의 각 원소는 A의 한 행과 B의 한 열을 원소끼리 곱해서 더한 값입니다.

$Y_{ij} = \Sigma_k A_{ik} \times B_{kj}$

Y의 한 칸은 A의 한 행과 B의 한 열을 곱해서 더한 값입니다



$Y_{11} = A_{11} \times B_{11} + A_{12} \times B_{21} = 1 \times 5 + 2 \times 7 = 19$

그림 2-2 · A = [[1, 2], [3, 4]], B = [[5, 6], [7, 8]]의 곱

Y의 다른 칸도 같은 방법으로 구합니다. 예를 들어 Y_{22}는 A의 2행 [3, 4]와 B의 2열 [6, 8]로 계산하므로 3 × 6 + 4 × 8 = 50입니다.

```
    /// (n x k) @ (k x m) -> (n x m)
    pub fn matmul(&mut self, a: T, b: T) -> T {
        let (n, k) = self.shape(a);
```

```
        let (k2, m) = self.shape(b);
        assert_eq!(k, k2, "matmul: inner dimensions differ");
        let (ad, bd) = (self.data(a), self.data(b));
        let mut out = vec![0.0; n * m];
        for i in 0..n {
            for kk in 0..k {
                let a_ik = ad[i * k + kk];
                for j in 0..m {
                    out[i * m + j] += a_ik * bd[kk * m + j];
                }
            }
        }
        self.push(out, n, m, Op::MatMul(a.0, b.0))
    }
```

- 반복문 순서가 교과서에 나오는 i → j → k가 아니라 **i → k → j**입니다. 결과는 같지만 속도가 크게 다릅니다. 가장 안쪽 반복문이 j를 돌 때 `bd[kk * m + j]`와 `out[i * m + j]`는 메모리에서 연속된 자리를 차례로 읽습니다. CPU는 연속된 메모리를 읽을 때 가장 빠릅니다. 또 가장 안쪽 반복문이 서로 독립적인 칸을 하나씩 갱신하는 형태가 되므로, 컴파일러가 여러 칸을 한 번에 계산하는 SIMD 명령으로 바꿀 수 있습니다. i → j → k 순서의 안쪽 반복문은 합 하나에 계속 더하는 형태라서 이렇게 바꿀 수 없습니다. 이 두 가지 이유로 반복문의 순서만 바꿔도 release 빌드에서 계산이 몇 배 빨라집니다.
- `let (ad, bd) = (self.data(a), self.data(b));`는 `&self`를 빌린 참조입니다. 마지막 줄의 `self.push`는 `&mut self`가 필요하지만, 그 시점에는 `ad`와 `bd`를 더 이상 쓰지 않으므로 컴파일에 문제가 없습니다.

backward 유도: 2 × 2 행렬로 작게 시작하겠습니다. A = [[1, 2], [3, 4]], B = [[5, 6], [7, 8]]이면, Y의 네 칸을 풀어 쓸 수 있습니다.

Y의 칸	계산식	값
Y_{11}	$A_{11}\cdot B_{11} + A_{12}\cdot B_{21} = 1\cdot 5 + 2\cdot 7$	19
Y_{12}	$A_{11}\cdot B_{12} + A_{12}\cdot B_{22} = 1\cdot 6 + 2\cdot 8$	22
Y_{21}	$A_{21}\cdot B_{11} + A_{22}\cdot B_{21} = 3\cdot 5 + 4\cdot 7$	43
Y_{22}	$A_{21}\cdot B_{12} + A_{22}\cdot B_{22} = 3\cdot 6 + 4\cdot 8$	50

이제 A_{11} 하나만 h만큼 늘려 봅니다. 표에서 A_{11}이 등장하는 곳은 두 줄뿐입니다.

- Y_{11}은 $B_{11} \times h = 5h$만큼 늘어납니다.
- Y_{12}는 $B_{12} \times h = 6h$만큼 늘어납니다.
- Y_{21}과 Y_{22}는 변하지 않습니다.

Y의 각 칸에는 gradient dY가 도착해 있습니다. 예를 들어 dY_{11}은 "Y_{11}이 1 늘면 loss가 dY_{11}만큼 변한다"는 뜻입니다. 0.5절의 chain rule과 0.6절의 "경로가 여러 개이면 더한다"를 적용하면 다음과 같습니다.

$$dA_{11} = dY_{11} \times B_{11} + dY_{12} \times B_{12}$$

숫자로 확인해 보겠습니다. loss = $Y_{11} + Y_{12}$라면 $dY_{11} = dY_{12} = 1$이므로 $dA_{11} = 5 + 6 = 11$입니다. 실제로 A_{11}을 1.01로 바꾸면 Y_{11}은 19.05, Y_{12}는 22.06이 되어 loss가 0.11 늘어납니다. 0.11 ÷ 0.01 = 11로 일치합니다. 같은 방법으로 A_{12}를 보면, A_{12}는 Y_{11}과 Y_{12}에서 각각 B_{21}, B_{22}와 곱해져 있으므로 $dA_{12} = dY_{11} \cdot B_{21} + dY_{12} \cdot B_{22}$입니다.

이 관찰을 일반화하면 다음 규칙이 됩니다.

- $A_{i\,k}$는 Y의 **i행에 있는 칸들**에만 등장합니다. $Y_{i\,j}$에서는 $B_{k\,j}$와 곱해져 있습니다.
- 따라서 $dA_{i\,k}$는 i행의 모든 j에 대해 $dY_{i\,j} \times B_{k\,j}$를 더한 값입니다.
- $B_{k\,j}$는 반대로 Y의 **j열에 있는 칸들**에만 등장하고, 거기서 $A_{i\,k}$와 곱해져 있습니다. 따라서 $dB_{k\,j}$는 모든 i에 대해 $A_{i\,k} \times dY_{i\,j}$를 더한 값입니다.

$dA_{i\,k} = \Sigma_{j}\ dY_{i\,j} \times B_{k\,j}$

$dB_{k\,j} = \Sigma_{i}\ A_{i\,k} \times dY_{i\,j}$

행렬 기호로 쓰면 $dA = dY \cdot B^{T}$, $dB = A^{T} \cdot dY$입니다(B^{T}는 B의 행과 열을 맞바꾼 행렬입니다). 이 표기는 크기를 맞춰 보면 외우기 쉽습니다. dA는 A와 같은 n × k여야 하므로, (n × m)인 dY에 (m × k)인 B^{T}를 곱하는 수밖에 없습니다.

```
    fn back_matmul(
        &self,
        dy: &[f32],
        a: usize,
        b: usize,
    ) -> Vec<(usize, Vec<f32>)> {
        let (n, k) = (self.nodes[a].rows, self.nodes[a].cols);
        let m = self.nodes[b].cols;
        let (ad, bd) = (&self.nodes[a].data, &self.nodes[b].data);
        let mut da = vec![0.0; n * k];
        let mut db = vec![0.0; k * m];
        for i in 0..n {
            for kk in 0..k {
                let a_ik = ad[i * k + kk];
                let mut sum = 0.0;
                for j in 0..m {
                    let g = dy[i * m + j];
                    sum += g * bd[kk * m + j]; // dA = dY @ B^T
                    db[kk * m + j] += a_ik * g; // dB = A^T @ dY
                }
                da[i * k + kk] = sum;
            }
        }
        vec![(a, da), (b, db)]
    }
```

forward와 같은 i → k → j 삼중 반복문 하나에서 dA와 dB를 함께 계산합니다. 한 번의 (i, k, j) 조합에서 $A_{i\,k}$, $B_{k\,j}$, $dY_{i\,j}$ 세 값이 모두 만나기 때문입니다.

match에 추가: 2.4절의 `input_grads`에 있는 `match`에서, `_ => todo!(...)` 줄 바로 위에 아래 줄을 추가합니다. 이후 절에서도 연산마다 같은 방식으로 한 줄씩 추가합니다.

```
            Op::MatMul(a, b) => self.back_matmul(dy, *a, *b),
```

이 backward가 맞는지는 다음 절에서 만드는 검사 도구로 바로 확인합니다.

2.6 gradient 검사 도구

목표: 연산의 backward가 맞는지 자동으로 확인하는 도구를 만듭니다. backward가 틀려도 학습이 "조금 덜 되는" 정도로만 나타나는 경우가 많습니다. 이런 버그는 결과만 보고는 찾기가 매우 어렵습니다. 이후 절에서는 연산을 하나 만들 때마다 이 도구로 바로 검사합니다.

아이디어: 0.1절의 방법 그대로입니다. 입력을 조금 바꿔서 loss가 얼마나 변하는지 직접 재고(수치 미분), 그 값을 backward가 계산한 gradient와 비교합니다. 다만 입력을 한쪽으로만 바꾸지 않고 양쪽으로 바꿉니다.

$\partial L/\partial x \approx (L(x + \varepsilon) - L(x - \varepsilon)) \div (2\varepsilon)$

양쪽으로 바꾸면 훨씬 정확해집니다. 0.1절의 $f(x) = x^2$, $x = 2$ 예에서 한쪽으로만 바꾼 변화율(2 → 2.01)은 4.01이었습니다. 양쪽으로 바꾸면(1.99 → 2.01) (4.0401 − 3.9601) ÷ 0.02 = 4가 나옵니다. x^2에서는 정확히 일치하고, 일반적인 함수에서도 오차가 한쪽 방식보다 훨씬 작습니다(ε을 10배 줄이면 한쪽 방식의 오차는 10배, 양쪽 방식의 오차는 100배 줄어듭니다).

정규분포 난수 추가: 검사에 쓸 입력은 무작위로 만듭니다. `src/rng.rs`의 `impl Rng` 블록 안에 아래 함수를 추가합니다.

```
    /// Standard normal sample (mean 0, std 1) via the Box–Muller transform.
    pub fn normal(&mut self) -> f64 {
        let u1 = self.uniform().max(1e-12); // avoid ln(0)
        let u2 = self.uniform();
        (-2.0 * u1.ln()).sqrt() * (2.0 * std::f64::consts::PI * u2).cos()
    }
```

- 평균 0, 표준편차 1인 정규분포(0.9절)에서 숫자를 하나 뽑습니다.
- Box-Muller 변환은 균등분포 난수 두 개(u1, u2)를 정규분포 난수 하나로 바꾸는 공식입니다. 공식은 외울 필요가 없습니다. 균등 난수를 넣으면 정규 난수가 나온다는 점만 알면 됩니다.
- `max(1e-12)`는 u1이 0일 때 ln 0 = −∞가 되는 것을 막습니다.
- 이 함수는 4부에서 가중치를 초기화할 때도 사용합니다.

숫자 하나로 줄이기: backward는 1 × 1 loss에서만 시작할 수 있습니다(2.4절의 `assert_eq!`). 그런데 검사하려는 연산의 출력은 대부분 행렬입니다. 그래서 출력의 모든 칸에 무작위 가중치를 곱해 더한 값 하나를 loss로 씁니다. 칸마다 0이 아닌 서로 다른 무작위 가중치가 곱해지므로, backward의 어느 부분이 틀려도 우연히 맞아떨어질 가능성은 사실상 없습니다. `src/tensor.rs`의 맨 끝에 테스트 모듈을 만들고 이 함수부터 넣습니다.

```
#[cfg(test)]
mod tests {
    use super::*;
    use crate::rng::Rng;

    /// Turn any tensor y into a 1 x 1 loss: u @ y @ r,
    /// with fixed random u and r. Every element of y gets
    /// its own random weight, so every element is checked.
    fn reduce(g: &mut Graph, y: T) -> T {
        let (n, m) = g.shape(y);
        let mut rng = Rng::new(99); // same seed every call -> same u and r
        let u = g.leaf((0..n).map(|_| rng.normal() as f32).collect(), 1, n);
        let r = g.leaf((0..m).map(|_| rng.normal() as f32).collect(), m, 1);
        let uy = g.matmul(u, y);
```

```
        g.matmul(uy, r)
    }
```

- `#[cfg(test)]`는 이 모듈을 `cargo test`를 실행할 때만 컴파일하라는 표시입니다. 일반 빌드에는 포함되지 않습니다.
- `use super::*;`는 바깥 파일의 `Graph`, `T` 같은 이름을 이 모듈 안에서 그대로 쓰겠다는 뜻입니다.
- u는 1 × n, r은 m × 1 행렬입니다. (1 × n)·(n × m)·(m × 1) = 1 × 1이므로, 행렬 곱셈 두 번으로 숫자 하나가 됩니다. 풀어 쓰면 y의 (i, j) 칸에 $u_i \times r_j$라는 가중치를 곱해서 모두 더한 값입니다.
- seed를 고정했으므로 몇 번을 호출해도 같은 u와 r이 나옵니다. 입력을 조금 바꾼 계산과 원래 계산을 같은 기준으로 비교하려면 이 조건이 필요합니다.
- 이 함수는 matmul로 만들어져 있으므로, matmul의 backward가 틀리면 모든 테스트가 함께 실패합니다. 2부에서 matmul을 가장 먼저 만든 이유입니다.

검사 함수 (앞부분): 무작위 입력을 만들고, "입력값을 받아 loss를 돌려주는" 함수 `eval`을 준비합니다.

```
    /// Compare backward() with finite differences for every input element.
    /// `build` gets the input tensors and returns the tensor to check.
    fn grad_check(
        shapes: &[(usize, usize)],
        build: impl Fn(&mut Graph, &[T]) -> T,
    ) {
        let mut rng = Rng::new(7);
        let inputs: Vec<Vec<f32>> = shapes
            .iter()
            .map(|&(n, m)| {
                (0..n * m).map(|_| rng.normal() as f32).collect()
            })
            .collect();

        // Loss for a given set of input values.
        let eval = |vals: &[Vec<f32>]| -> f32 {
            let mut g = Graph::new();
            let ts: Vec<T> = vals
                .iter()
                .zip(shapes)
                .map(|(v, &(n, m))| g.leaf(v.clone(), n, m))
                .collect();
            let y = build(&mut g, &ts);
            let loss = reduce(&mut g, y);
            g.data(loss)[0]
        };
```

- `shapes`는 입력 행렬들의 크기 목록입니다. 예를 들어 `&[(3, 4), (4, 5)]`는 3 × 4 행렬과 4 × 5 행렬 두 개를 입력으로 쓴다는 뜻입니다.
- `build`는 입력 tensor들을 받아 검사할 연산을 쌓고, 그 결과 tensor를 돌려주는 함수입니다. 테스트마다 이 부분만 바꿉니다.
- `eval`은 입력값이 주어질 때마다 그래프를 새로 만들어 loss만 계산합니다. 입력 원소 하나마다 두 번씩 호출되므로 느리지만, 테스트용 행렬이 작아서 순식간에 끝납니다.

> **Rust 메모: `impl Fn` 과 클로저**
>
> `build: impl Fn(&mut Graph, &[T]) -> T`는 "`&mut Graph`와 `&[T]`를 받아 `T`를 돌려주는 함수라면 무엇이든 받는다"는 뜻입니다. 테스트에서는 이 자리에 `|g, x| g.matmul(x[0], x[1])` 같은 **클로저**를 넘깁니다. 클로저는 `|인자| 본문` 형태로 그 자리에서 만드는 이름 없는 함수입니다. `eval`도 클로저인데, 바깥의 `shapes`와 `build`를 함수 안에서 그대로 사용합니다. 이렇게 바깥 변수를 가져다 쓰는 것을 "캡처한다"고 부릅니다.

검사 함수 (뒷부분): backward로 구한 gradient와 수치 미분으로 구한 gradient를 모든 입력 원소에 대해 비교합니다.

```
        // Analytic gradients from backward().
        let mut g = Graph::new();
        let ts: Vec<T> = inputs
            .iter()
            .zip(shapes)
            .map(|(v, &(n, m))| g.leaf(v.clone(), n, m))
            .collect();
        let y = build(&mut g, &ts);
        let loss = reduce(&mut g, y);
        g.backward(loss);

        // Numeric gradients: (f(x + eps) − f(x − eps)) / (2 eps).
        let eps = 1e-2;
        for (ti, t) in ts.iter().enumerate() {
            for e in 0..inputs[ti].len() {
                let mut plus = inputs.clone();
                plus[ti][e] += eps;
                let mut minus = inputs.clone();
                minus[ti][e] -= eps;
                let numeric = (eval(&plus) − eval(&minus)) / (2.0 * eps);
                let analytic = g.grad(*t)[e];
                let tol = 1e-3 + 1e-2 * numeric.abs();
                assert!(
                    (numeric − analytic).abs() < tol,
                    "input {ti}[{e}]: numeric {numeric}, analytic {analytic}"
                );
            }
        }
    }
```

- 앞의 세 줄 묶음은 원래 입력으로 그래프를 한 번 만들고 backward를 실행합니다. 이 gradient가 검사 대상입니다.
- 이중 반복문은 입력 행렬 하나하나(`ti`)의 원소 하나하나(`e`)를 ±ε만큼 바꿔 수치 미분을 구합니다.
- ε을 너무 작게 잡으면 f32의 반올림 오차가 커지고, 너무 크게 잡으면 근사 오차가 커집니다. f32에서는 0.01 정도가 적당합니다.
- 허용 오차는 "절대 오차 0.001 + 상대 오차 1%"입니다. backward에 실제 버그가 있으면 차이가 보통 수십 퍼센트 이상 나므로, 이 정도 여유를 두어도 버그는 충분히 잡힙니다.

첫 테스트: 2.5절의 matmul을 검사하는 테스트를 추가하고, 모듈을 닫습니다.

```
    #[test]
    fn matmul() {
        grad_check(&[(3, 4), (4, 5)], |g, x| g.matmul(x[0], x[1]));
```

```
    }
}
```

- `#[test]`가 붙은 함수가 하나의 테스트입니다. `cargo test`가 이런 함수를 모두 찾아 실행합니다.
- 클로저 `|g, x| g.matmul(x[0], x[1])`가 "검사할 연산"입니다. `x[0]`과 `x[1]`은 `shapes`에 적은 3 × 4, 4 × 5 크기의 무작위 입력입니다.
- 마지막 `}`는 `mod tests`를 닫습니다. 이후 절의 테스트는 모두 이 `}` 바로 앞에 추가합니다.

확인: `cargo test --release -q matmul`을 실행하면 아래와 같이 출력됩니다. 실패하면 부록 D의 gradient 검사 항목을 참고합니다.

```
running 1 test
.
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

2.7 덧셈, 행 단위 덧셈과 곱셈, 상수배

목표: 원소마다 독립적으로 적용되는 단순한 연산 네 가지를 만듭니다.

```
    /// Elementwise a + b, same shape.
    pub fn add(&mut self, a: T, b: T) -> T {
        assert_eq!(self.shape(a), self.shape(b), "add: shapes differ");
        let out = self
            .data(a)
            .iter()
            .zip(self.data(b))
            .map(|(x, y)| x + y)
            .collect();
        let (n, m) = self.shape(a);
        self.push(out, n, m, Op::Add(a.0, b.0))
    }

    /// (n x m) + (1 x m): the row b is added to every row of a.
    pub fn add_row(&mut self, a: T, b: T) -> T {
        let (n, m) = self.shape(a);
        assert_eq!(self.shape(b), (1, m), "add_row: b must be 1 x cols");
        let bd = self.data(b);
        let out = self
            .data(a)
            .iter()
            .enumerate()
            .map(|(i, x)| x + bd[i % m])
            .collect();
        self.push(out, n, m, Op::AddRow(a.0, b.0))
    }

    /// (n x m) * (1 x m): every row of a is multiplied elementwise by b.
    pub fn mul_row(&mut self, a: T, b: T) -> T {
        let (n, m) = self.shape(a);
        assert_eq!(self.shape(b), (1, m), "mul_row: b must be 1 x cols");
        let bd = self.data(b);
        let out = self
            .data(a)
            .iter()
            .enumerate()
```

```
            .map(|(i, x)| x * bd[i % m])
            .collect();
        self.push(out, n, m, Op::MulRow(a.0, b.0))
    }

    /// Every element times the constant s.
    pub fn scale(&mut self, a: T, s: f32) -> T {
        let out = self.data(a).iter().map(|x| x * s).collect();
        let (n, m) = self.shape(a);
        self.push(out, n, m, Op::Scale(a.0, s))
    }
```

- `add`는 크기가 같은 두 행렬을 원소끼리 더합니다. 4부의 residual 연결과 5부의 위치 embedding에 사용합니다.
- `add_row`는 한 줄짜리 행렬 b를 a의 모든 행에 더합니다. 평탄화된 인덱스 i에서 열 번호는 `i % m`이므로, 행이 바뀌어도 같은 열에는 항상 같은 b 값이 더해집니다. 이처럼 작은 행렬을 반복해서 큰 행렬에 맞추는 것을 broadcast라고 부릅니다. 신경망의 bias가 바로 이 연산입니다.
- `mul_row`는 같은 방식으로 곱합니다. 4부의 layernorm에서 사용합니다.
- `scale`은 모든 원소에 같은 상수를 곱합니다. attention에서 점수를 √d로 나눌 때 사용합니다.

backward: 각 연산은 1부의 규칙을 원소마다 적용한 것입니다.

연산	da	db
`add`	dy	dy
`add_row`	dy	dy를 열별로 모두 더한 값
`mul_row`	dy × b	dy × a를 열별로 모두 더한 값
`scale`	dy × s	없음

`add`와 `scale`은 한 줄이면 충분해서 별도 함수 없이 `match` 안에 직접 적습니다(이 절 끝의 "match에 추가"). 나머지 두 개는 아래와 같습니다.

```
    fn back_add_row(
        &self,
        dy: &[f32],
        a: usize,
        b: usize,
    ) -> Vec<(usize, Vec<f32>)> {
        let m = self.nodes[b].cols;
        let mut db = vec![0.0; m];
        for (i, g) in dy.iter().enumerate() {
            db[i % m] += g; // b was used once per row, so its grads add up
        }
        vec![(a, dy.to_vec()), (b, db)]
    }

    fn back_mul_row(
        &self,
        dy: &[f32],
        a: usize,
        b: usize,
    ) -> Vec<(usize, Vec<f32>)> {
        let m = self.nodes[b].cols;
        let (ad, bd) = (&self.nodes[a].data, &self.nodes[b].data);
```

```
        let mut da = vec![0.0; dy.len()];
        let mut db = vec![0.0; m];
        for (i, g) in dy.iter().enumerate() {
            da[i] = g * bd[i % m];
            db[i % m] += g * ad[i];
        }
        vec![(a, da), (b, db)]
    }
```

broadcast 연산의 backward에는 중요한 규칙이 하나 있습니다. **forward에서 복사되어 여러 번 쓰인 값은, backward에서 gradient가 합산됩니다.** b의 j번째 값은 n개의 행에 한 번씩 쓰였으므로, n개 경로의 gradient를 모두 더합니다. 1부에서 `+=`로 누적한 것과 같은 원리입니다.

숫자로 확인해 보겠습니다. 3 × 2 행렬 a에 b = [10, 20]을 `add_row`로 더하고, loss는 결과의 여섯 칸을 모두 더한 값이라고 하겠습니다. 그러면 dy는 여섯 칸 모두 1입니다. b의 첫 값 10을 10.01로 바꾸면 결과의 첫 열 세 칸이 모두 0.01씩 늘어나므로, loss는 0.03 늘어납니다. 따라서 db의 첫 값은 0.03 ÷ 0.01 = 3이고, 이는 첫 열의 dy 세 개(1 + 1 + 1)를 더한 값과 같습니다.

match에 추가: `input_grads`의 `match`에서 `_ => todo!(...)` 줄 바로 위에 아래 줄을 추가합니다.

```
            Op::Add(a, b) => vec![(*a, dy.clone()), (*b, dy.clone())],
            Op::AddRow(a, b) => self.back_add_row(dy, *a, *b),
            Op::MulRow(a, b) => self.back_mul_row(dy, *a, *b),
            Op::Scale(a, s) => {
                vec![(*a, dy.iter().map(|g| g * s).collect())]
            }
```

테스트: 파일 끝의 `mod tests` 안에서 닫는 `}` 바로 앞에 아래 테스트를 추가합니다.

```
    #[test]
    fn add_and_rows() {
        grad_check(&[(3, 4), (3, 4), (1, 4), (1, 4)], |g, x| {
            let y = g.add(x[0], x[1]);
            let y = g.add_row(y, x[2]);
            let y = g.mul_row(y, x[3]);
            g.scale(y, 0.5)
        });
    }
```

이 테스트는 네 연산을 차례로 이어 붙여 한 번에 검사합니다.

확인: `cargo test --release -q add_and_rows`를 실행하면 `test result: ok. 1 passed`가 출력됩니다.

2.8 ReLU

목표: 음수를 0으로 바꾸는 비선형 함수를 만듭니다. 1부의 tanh와 같은 역할이며, 더 단순하고 계산이 빠릅니다.

```
    pub fn relu(&mut self, a: T) -> T {
        let out = self.data(a).iter().map(|x| x.max(0.0)).collect();
        let (n, m) = self.shape(a);
        self.push(out, n, m, Op::Relu(a.0))
    }
```

```
    fn back_relu(&self, dy: &[f32], a: usize) -> Vec<(usize, Vec<f32>)> {
        let ad = &self.nodes[a].data;
        let da = dy
            .iter()
            .zip(ad)
            .map(|(g, x)| if *x > 0.0 { *g } else { 0.0 })
            .collect();
        vec![(a, da)]
    }
```

ReLU는 입력이 양수일 때 기울기가 1이고, 음수일 때 기울기가 0입니다. 그래서 gradient는 입력이 양수였던 자리로만 그대로 통과하고, 나머지 자리에서는 막힙니다.

match에 추가: `input_grads`의 `match`에서 `_ => todo!(...)` 줄 바로 위에 아래 줄을 추가합니다.

```
            Op::Relu(a) => self.back_relu(dy, *a),
```

테스트: 파일 끝의 `mod tests` 안에서 닫는 `}` 바로 앞에 아래 테스트를 추가합니다.

```
    #[test]
    fn relu() {
        grad_check(&[(4, 5)], |g, x| g.relu(x[0]));
    }
```

확인: `cargo test --release -q relu`를 실행하면 `test result: ok. 1 passed`가 출력됩니다.

2.9 전치, 자르기, 이어 붙이기

목표: 값을 계산하지 않고 위치만 옮기는 연산 다섯 가지를 만듭니다. attention에서 batch와 head를 나누고 다시 합치는 데 사용합니다.

이 연산들의 backward는 모두 같은 원리를 따릅니다. **forward에서 값을 옮긴 경로를 거꾸로 따라가며 gradient를 제자리로 돌려보냅니다.** 값이 변하지 않고 자리만 바뀌었으므로, 미분 계수는 항상 1입니다.

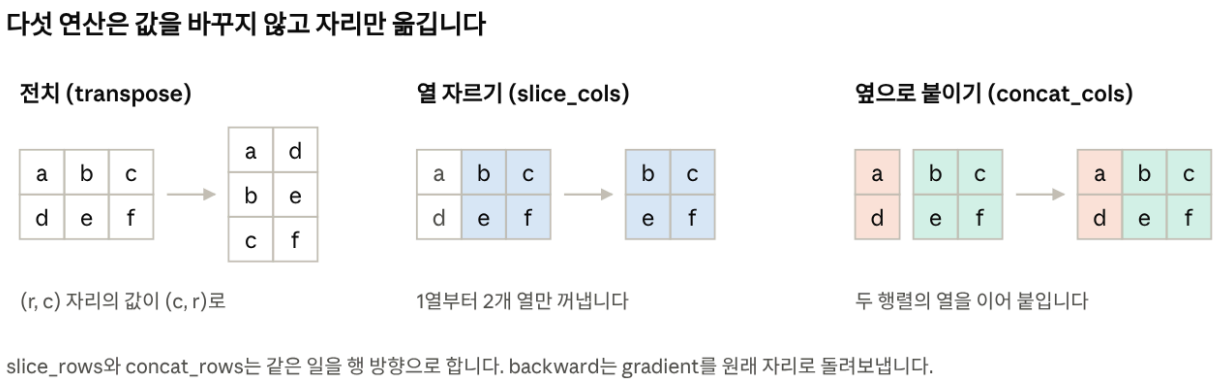


그림 2-3 · 전치, 열 자르기, 옆으로 붙이기

글자 하나가 숫자 하나입니다. 글자가 어디로 옮겨 갔는지 따라가 보면, backward가 gradient를 어느 자리로 돌려보내야 하는지도 알 수 있습니다.

전치(transpose)는 행과 열을 맞바꿉니다. (r, c) 자리의 값이 (c, r) 자리로 갑니다.

```
    /// (n x m) -> (m x n)
    pub fn transpose(&mut self, a: T) -> T {
        let (n, m) = self.shape(a);
        let ad = self.data(a);
        let mut out = vec![0.0; n * m];
        for r in 0..n {
            for c in 0..m {
                out[c * n + r] = ad[r * m + c];
            }
        }
        self.push(out, m, n, Op::Transpose(a.0))
    }
```

```
    fn back_transpose(
        &self,
        dy: &[f32],
        a: usize,
    ) -> Vec<(usize, Vec<f32>)> {
        let (n, m) = (self.nodes[a].rows, self.nodes[a].cols);
        let mut da = vec![0.0; n * m];
        for r in 0..n {
            for c in 0..m {
                da[r * m + c] = dy[c * n + r];
            }
        }
        vec![(a, da)]
    }
```

forward의 대입문 `out[c * n + r] = ad[r * m + c]`에서 양쪽을 맞바꾼 것이 backward의 `da[r * m + c] = dy[c * n + r]` 입니다. 즉 전치의 backward는 gradient를 다시 전치하는 것입니다.

자르기(slice)는 행렬의 일부 행이나 일부 열만 꺼냅니다.

```
    /// Rows start..start+len of a.
    pub fn slice_rows(&mut self, a: T, start: usize, len: usize) -> T {
        let (_, m) = self.shape(a);
        let out = self.data(a)[start * m..(start + len) * m].to_vec();
        self.push(out, len, m, Op::SliceRows(a.0, start))
    }

    /// Columns start..start+len of a.
    pub fn slice_cols(&mut self, a: T, start: usize, len: usize) -> T {
        let (n, m) = self.shape(a);
        let ad = self.data(a);
        let mut out = Vec::with_capacity(n * len);
        for r in 0..n {
            out.extend_from_slice(&ad[r * m + start..r * m + start + len]);
        }
        self.push(out, n, len, Op::SliceCols(a.0, start))
    }
```

- 행 우선 저장 방식에서는 연속된 행들이 메모리에서도 연속되어 있습니다. 그래서 `slice_rows`는 배열을 한 번만 잘라내면 됩니다.

- 반면 열은 행마다 떨어져 있으므로, `slice_cols`는 행을 하나씩 돌면서 필요한 구간만 복사합니다.
- `Op`에는 시작 위치만 저장합니다. 잘라낸 길이는 결과 노드의 크기(`rows` 또는 `cols`)에 이미 들어 있기 때문입니다.

```
    fn back_slice_rows(
        &self,
        dy: &[f32],
        a: usize,
        start: usize,
    ) -> Vec<(usize, Vec<f32>)> {
        let m = self.nodes[a].cols;
        let mut da = vec![0.0; self.nodes[a].data.len()];
        da[start * m..start * m + dy.len()].copy_from_slice(dy);
        vec![(a, da)]
    }

    fn back_slice_cols(
        &self,
        dy: &[f32],
        len: usize,
        a: usize,
        start: usize,
    ) -> Vec<(usize, Vec<f32>)> {
        let (n, m) = (self.nodes[a].rows, self.nodes[a].cols);
        let mut da = vec![0.0; n * m];
        for r in 0..n {
            da[r * m + start..r * m + start + len]
                .copy_from_slice(&dy[r * len..(r + 1) * len]);
        }
        vec![(a, da)]
    }
```

원래 크기의 0 행렬을 만들고, 잘라냈던 자리에만 gradient를 채워 넣습니다. 잘라내지 않은 부분은 결과에 영향을 주지 않았으므로 gradient가 0입니다. 같은 행렬에서 여러 조각을 잘라냈다면, `backward`가 각 조각의 gradient를 모두 더해 주므로 원래 행렬의 gradient가 완성됩니다.

예시로 따라가기: 2 × 3 행렬 a = [[1, 2, 3], [4, 5, 6]]은 메모리에 [1, 2, 3, 4, 5, 6]으로 저장됩니다.

- `slice_cols(a, 1, 2)`는 각 행에서 1번 열부터 2개를 꺼냅니다. 결과는 [[2, 3], [5, 6]]이고, 메모리로는 [2, 3, 5, 6]입니다. 원래 배열의 1, 2, 4, 5번 자리에서 온 값입니다.
- backward에서 dy = [10, 20, 30, 40]이 도착하면, `back_slice_cols`는 각 값을 원래 자리로 돌려보내 da = [0, 10, 20, 0, 30, 40]을 만듭니다.
- 0번과 3번 자리(0번 열)는 잘라낸 결과에 들어가지 않았으므로 gradient가 0입니다.

이어 붙이기(concat)는 자르기의 반대입니다.

```
    /// Stack tensors with the same column count on top of each other.
    pub fn concat_rows(&mut self, parts: &[T]) -> T {
        let m = self.shape(parts[0]).1;
        let mut out = Vec::new();
        let mut n = 0;
        for &p in parts {
            assert_eq!(
                self.shape(p).1,
```

```
            m,
            "concat_rows: column counts differ"
        );
        out.extend_from_slice(self.data(p));
        n += self.shape(p).0;
    }
    let ids = parts.iter().map(|p| p.0).collect();
    self.push(out, n, m, Op::ConcatRows(ids))
}

/// Place tensors with the same row count side by side.
pub fn concat_cols(&mut self, parts: &[T]) -> T {
    let n = self.shape(parts[0]).0;
    let m: usize = parts.iter().map(|&p| self.shape(p).1).sum();
    let mut out = Vec::with_capacity(n * m);
    for r in 0..n {
        for &p in parts {
            let (pn, pm) = self.shape(p);
            assert_eq!(pn, n, "concat_cols: row counts differ");
            out.extend_from_slice(&self.data(p)[r * pm..(r + 1) * pm]);
        }
    }
    let ids = parts.iter().map(|p| p.0).collect();
    self.push(out, n, m, Op::ConcatCols(ids))
}
```

```
fn back_concat_rows(
    &self,
    dy: &[f32],
    parts: &[usize],
) -> Vec<(usize, Vec<f32>)> {
    let mut offset = 0;
    let mut result = Vec::new();
    for &p in parts {
        let size = self.nodes[p].data.len();
        result.push((p, dy[offset..offset + size].to_vec()));
        offset += size;
    }
    result
}

fn back_concat_cols(
    &self,
    dy: &[f32],
    n: usize,
    parts: &[usize],
) -> Vec<(usize, Vec<f32>)> {
    let m: usize = parts.iter().map(|&p| self.nodes[p].cols).sum();
    let mut col = 0;
    let mut result = Vec::new();
    for &p in parts {
        let pm = self.nodes[p].cols;
        let mut dp = Vec::with_capacity(n * pm);
        for r in 0..n {
            dp.extend_from_slice(&dy[r * m + col..r * m + col + pm]);
        }
        result.push((p, dp));
        col += pm;
    }
```

```
        result
    }
```

- 합치기 연산은 입력이 여러 개입니다. 그래서 `Op`에 입력 인덱스를 `Vec`으로 저장하고, backward는 입력 수만큼의 (인덱스, gradient) 쌍을 돌려줍니다.
- backward는 합친 결과의 gradient를 다시 원래 조각 크기로 잘라서 돌려줍니다.

예시로 따라가기: p = [[1], [4]](2 × 1)와 q = [[2, 3], [5, 6]](2 × 2)을 `concat_cols`로 이어 붙이면 [[1, 2, 3], [4, 5, 6]]이 됩니다. backward에서는 dy의 각 행을 앞의 1칸(p의 몫)과 뒤의 2칸(q의 몫)으로 나눕니다. dy = [10, 20, 30, 40, 50, 60]이면 p의 gradient는 [10, 40], q의 gradient는 [20, 30, 50, 60]입니다. `back_concat_cols`의 `col`은 각 조각이 몇 번째 열부터 시작하는지를 기억하는 변수입니다. p를 처리할 때는 0, q를 처리할 때는 1입니다.

match에 추가: `input_grads`의 `match`에서 `_ => todo!(...)` 줄 바로 위에 아래 줄을 추가합니다.

```
            Op::Transpose(a) => self.back_transpose(dy, *a),
            Op::SliceRows(a, start) => self.back_slice_rows(dy, *a, *start),
            Op::SliceCols(a, start) => {
                self.back_slice_cols(dy, node.cols, *a, *start)
            }
            Op::ConcatRows(parts) => self.back_concat_rows(dy, parts),
            Op::ConcatCols(parts) => {
                self.back_concat_cols(dy, node.rows, parts)
            }
```

테스트: 파일 끝의 `mod tests` 안에서 닫는 `}` 바로 앞에 아래 테스트를 추가합니다.

```
    #[test]
    fn transpose_and_slices() {
        grad_check(&[(4, 6)], |g, x| {
            let t = g.transpose(x[0]); // 6 x 4
            let a = g.slice_rows(t, 1, 3); // 3 x 4
            let b = g.slice_cols(a, 1, 2); // 3 x 2
            let c = g.slice_cols(a, 0, 3); // 3 x 3
            let y = g.concat_cols(&[b, c]); // 3 x 5
            g.concat_rows(&[y, y]) // 6 x 5
        });
    }
```

이 테스트는 같은 행렬 `a`에서 겹치는 두 조각(`b`와 `c`)을 잘라내고, `y`를 두 번 이어 붙입니다. 한 값이 여러 경로로 쓰일 때 gradient가 제대로 더해지는지(0.6절)도 함께 검사하려는 의도입니다.

확인: `cargo test --release -q transpose_and_slices`를 실행하면 `test result: ok. 1 passed`가 출력됩니다.

2.10 causal mask와 softmax

목표: attention의 핵심인 "미래 가리기"와 "점수를 확률로 바꾸기"를 만듭니다. 두 연산이 왜 필요한지는 4부에서 자세히 설명하고, 여기서는 계산만 다룹니다.

causal mask는 정사각행렬에서 대각선보다 오른쪽 위에 있는 값(열 번호가 행 번호보다 큰 값)을 모두 음의 무한대로 바꿉니다.

```
    /// Square matrix: entries above the diagonal (col > row) become -inf.
    pub fn causal_mask(&mut self, a: T) -> T {
        let (n, m) = self.shape(a);
        assert_eq!(n, m, "causal_mask: matrix must be square");
        let mut out = self.data(a).to_vec();
        for r in 0..n {
            for c in r + 1..m {
                out[r * m + c] = f32::NEG_INFINITY;
            }
        }
        self.push(out, n, m, Op::CausalMask(a.0))
    }
```

```
    fn back_causal_mask(
        &self,
        dy: &[f32],
        m: usize,
        a: usize,
    ) -> Vec<(usize, Vec<f32>)> {
        let mut da = dy.to_vec();
        for r in 0..m {
            for c in r + 1..m {
                // Masked entries did not affect the output.
                da[r * m + c] = 0.0;
            }
        }
        vec![(a, da)]
    }
```

가려진 자리는 입력값이 무엇이든 출력이 항상 음의 무한대입니다. 따라서 그 자리의 gradient는 0이고, 나머지 자리는 gradient가 그대로 통과합니다.

softmax는 숫자 목록을 확률 분포로 바꿉니다. 모든 값이 0과 1 사이가 되고, 합은 1이 되며, 큰 값일수록 큰 확률을 받습니다.

$y_j = e^{x_j} \div \Sigma_k e^{x_k}$

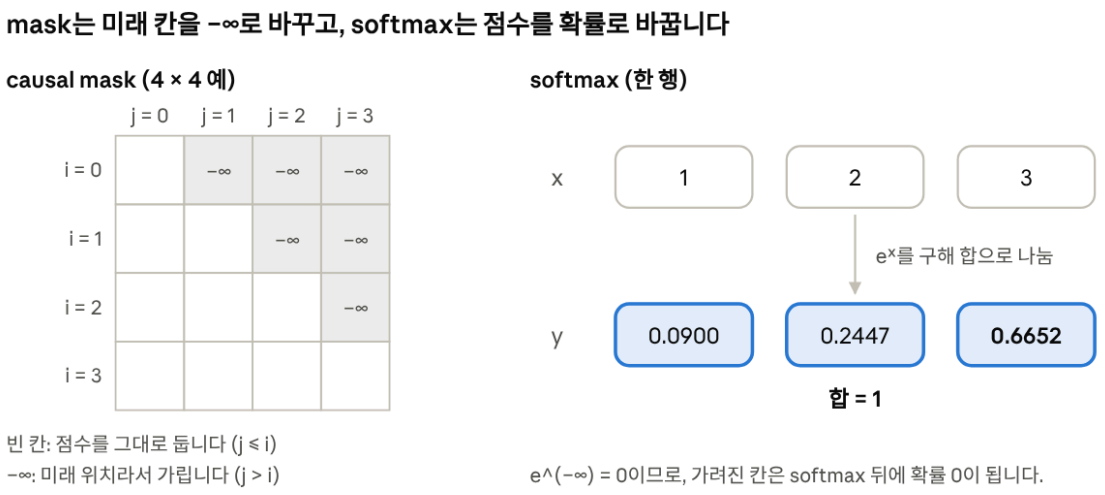


그림 2-4 · causal mask와 softmax

왼쪽 그림에서 i행의 빈 칸은 위치 i가 볼 수 있는 범위입니다. 오른쪽 숫자는 이 절 뒤의 '숫자로 확인'과 같은 예입니다.

e를 음의 무한대만큼 거듭제곱하면 0이 되므로, mask로 가린 자리는 softmax를 거치면 확률이 정확히 0이 됩니다. 두 연산을 이어 붙이는 이유가 바로 이것입니다. 대각선은 가리지 않으므로 어떤 행도 전부 음의 무한대가 되지 않습니다. 한 행이 전부 가려지면 0 ÷ 0이 되어 NaN이 나옵니다.

이 계산은 두 곳에서 필요하므로(여기와 2.13절의 cross entropy) 행 하나를 처리하는 함수로 따로 분리했습니다. 이 함수는 `impl Graph` 블록 바깥에 둡니다. 위치는 impl 블록이 끝난 뒤, 테스트 모듈보다 앞입니다.

```
/// Numerically stable softmax: subtract the max before exp.
pub fn softmax_in_place(row: &mut [f32]) {
    let max = row.iter().cloned().fold(f32::NEG_INFINITY, f32::max);
    let mut sum = 0.0;
    for x in row.iter_mut() {
        *x = (*x - max).exp();
        sum += *x;
    }
    for x in row.iter_mut() {
        *x /= sum;
    }
}
```

가장 큰 값을 먼저 빼는 이유는 overflow를 막기 위해서입니다. f32에서 e^{90}은 이미 무한대가 되므로, 입력에 90이라는 값 하나만 있어도 결과가 NaN이 됩니다. 모든 값에서 같은 수를 빼도 분자와 분모가 같은 비율로 변하므로 softmax의 결과는 바뀌지 않습니다. 최댓값을 빼면 가장 큰 값이 0이 되어 $e^0 = 1$을 넘지 않습니다.

```
    /// Softmax of each row: exp(x) / sum(exp(x)).
    pub fn softmax(&mut self, a: T) -> T {
        let (n, m) = self.shape(a);
        let mut out = self.data(a).to_vec();
        for row in out.chunks_mut(m) {
            softmax_in_place(row);
        }
        self.push(out, n, m, Op::Softmax(a.0))
    }
```

`chunks_mut(m)`은 평탄한 배열을 길이 m의 조각, 즉 행 단위로 나눠 줍니다. 행 우선 저장 방식에서 "행마다 어떤 일을 한다"는 코드는 대부분 이 함수로 작성합니다. 1.5절의 sub처럼 softmax를 기존 연산의 조합으로 만들 수도 있지만, 그러려면 exp, 행별 합, 나눗셈 같은 연산이 더 필요하고 노드도 훨씬 많아집니다. 그래서 softmax를 연산 하나로 만들고 backward를 직접 유도합니다.

softmax backward 유도: 행 하나만 보겠습니다. 분모의 합을 s라고 부르면 $y_j = e^{x_j} \div s$입니다. 이제 입력 하나 x_i를 h만큼 늘렸을 때 무엇이 변하는지 차례로 따라갑니다.

1. **분자 e^{x_i}**: 0.8절의 근사에 따라 $e^{x_i} \times (1 + h)$가 됩니다. 즉 $e^{x_i} \times h$만큼 늘어납니다.
2. **분모 s**: s는 e^{x_i}를 포함한 합이므로 같은 양 $e^{x_i} \times h$만큼 늘어납니다. 그런데 $e^{x_i} = y_i \times s$이므로, s는 $s \times (1 + y_i h)$가 됩니다.
3. **다른 칸 y_j $(j \neq i)$**: 분자는 그대로이고 분모만 $(1 + y_i h)$배가 됩니다. 0.8절의 근사 $1 \div (1 + a) \approx 1 - a$를 쓰면 y_j는 $y_j \times (1 - y_i h)$가 됩니다. 변화량은 $\mathbf{-y_j \times y_i \times h}$입니다.

4. **자기 칸 y_i**: 분자는 (1 + h)배, 분모는 $(1 + y_i h)$배가 되므로 $y_i \times (1 + h) \times (1 - y_i h) \approx y_i \times (1 + h - y_i h)$입니다. 변화량은 $\mathbf{y_i \times h - y_i \times y_i \times h}$입니다.

3번과 4번을 비교하면, 모든 칸에 공통으로 $-y_j \times y_i \times h$가 있고, 자기 칸에만 $y_i \times h$가 더 붙습니다. 이제 각 칸의 변화량에 그 칸의 gradient g_j (2.4절의 dy와 같은 값)를 곱해 모두 더하면 loss의 변화량이 됩니다(0.5절과 0.6절).

$$\Delta\text{loss} = g_i \times y_i \times h - y_i \times h \times \Sigma_j \; g_j y_j$$

h로 나누면 x_i의 gradient가 됩니다.

$$dx_i = y_i \times (g_i - \Sigma_j \; g_j y_j)$$

괄호 안의 합 $\Sigma \, g_j \; y_j$는 i와 상관없이 행마다 한 번만 계산하면 됩니다. 코드에서는 이 값을 `dot`이라고 부릅니다.

숫자로 확인: x = [1, 2, 3]이면 y = [0.0900, 0.2447, 0.6652]입니다. loss를 y_3 하나로 정하면 g = [0, 0, 1]이고, dot = 0 + 0 + 1 × 0.6652 = 0.6652입니다.

입력	공식 $y_i \times (g_i - \text{dot})$	결과	x_i를 조금 늘려 직접 잰 변화율
x_1	0.0900 × (0 − 0.6652)	−0.0599	−0.0599
x_2	0.2447 × (0 − 0.6652)	−0.1628	−0.1628
x_3	0.6652 × (1 − 0.6652)	0.2227	0.2227

- x_3를 올리면 y_3가 커지고, 다른 입력을 올리면 y_3가 작아집니다. 확률의 합은 항상 1이므로, 다른 칸이 커지면 y_3의 몫이 줄어들기 때문입니다.
- 세 gradient를 더하면 0입니다. 모든 입력에 같은 수를 더해도 softmax 결과는 변하지 않기 때문입니다. 최댓값을 빼는 요령이 통하는 것도 같은 이유입니다.

```
    fn back_softmax(
        &self,
        dy: &[f32],
        y: &[f32],
        m: usize,
        a: usize,
    ) -> Vec<(usize, Vec<f32>)> {
        let mut da = vec![0.0; dy.len()];
        for ((dx, g), y) in
            da.chunks_mut(m).zip(dy.chunks(m)).zip(y.chunks(m))
        {
            let dot: f32 = g.iter().zip(y).map(|(g, y)| g * y).sum();
            for j in 0..m {
                dx[j] = y[j] * (g[j] - dot);
            }
        }
        vec![(a, da)]
    }
```

backward에는 forward의 출력 y만 필요하고 입력 x는 필요하지 않습니다. 그래서 아래 "match에 추가"의 줄에서 이 노드의 `data`(출력)를 넘겨줍니다.

match에 추가: `input_grads`의 `match`에서 `_ => todo!(...)` 줄 바로 위에 아래 줄을 추가합니다.

```
            Op::CausalMask(a) => self.back_causal_mask(dy, node.cols, *a),
            Op::Softmax(a) => {
                self.back_softmax(dy, &node.data, node.cols, *a)
            }
```

테스트: 파일 끝의 `mod tests` 안에서 닫는 `}` 바로 앞에 아래 테스트를 추가합니다.

```
    #[test]
    fn mask_and_softmax() {
        grad_check(&[(4, 4)], |g, x| {
            let y = g.causal_mask(x[0]);
            g.softmax(y) // -inf entries become exact zeros here
        });
    }
```

mask만 따로 검사하지 않고 softmax를 이어서 검사합니다. mask의 출력에는 음의 무한대가 들어 있어서, 그대로 숫자 하나로 줄이면 결과가 무한대가 되기 때문입니다. softmax를 거치면 음의 무한대가 정확한 0으로 바뀝니다.

확인: `cargo test --release -q mask_and_softmax`를 실행하면 `test result: ok. 1 passed`가 출력됩니다.

2.11 layernorm

목표: 행마다 평균을 0, 분산을 1로 맞추는 연산을 만듭니다. 각 부품에 들어가는 값의 크기를 일정하게 맞춰서, 층을 깊게 쌓아도 학습을 안정시킵니다.

$\hat{x}_j = (x_j - \mu) \div \sqrt{(\sigma^2 + \varepsilon)}$

μ는 행의 평균, σ^2는 행의 분산입니다. ε(0.00001)은 분산이 0일 때 0으로 나누는 것을 막는 작은 수입니다. 평균과 "표준편차의 역수(rstd)"는 forward와 backward에서 모두 필요하므로 함수로 나눕니다. 이 함수도 2.10절의 도우미 함수와 같은 자리(impl 블록이 끝난 뒤, 테스트 모듈보다 앞)에 둡니다.

```
/// Mean and 1 / standard deviation of a row (with a small epsilon).
fn mean_rstd(row: &[f32]) -> (f32, f32) {
    let m = row.len() as f32;
    let mean = row.iter().sum::<f32>() / m;
    let var = row.iter().map(|x| (x - mean) * (x - mean)).sum::<f32>() / m;
    (mean, 1.0 / (var + 1e-5).sqrt())
}
```

```
    /// Each row shifted to mean 0 and scaled to variance 1.
    pub fn layer_norm(&mut self, a: T) -> T {
        let (n, m) = self.shape(a);
        let mut out = self.data(a).to_vec();
        for row in out.chunks_mut(m) {
            let (mean, rstd) = mean_rstd(row);
            for x in row.iter_mut() {
                *x = (*x - mean) * rstd;
            }
        }
        self.push(out, n, m, Op::LayerNorm(a.0))
    }
```

backward 유도: 여기서는 표준편차를 $\sigma = \sqrt{(\sigma^2 + \varepsilon)}$로 쓰겠습니다. 코드의 `rstd`는 $1/\sigma$입니다. 행의 길이는 m이고, 출력은 $\hat{x}_j = (x_j - \mu) \div \sigma$입니다. softmax와 마찬가지로 x_i 하나를 h만큼 늘리고, 그 변화가 어디로 퍼지는지 따라갑니다. 단계가 많지만 흐름은 두 덩어리입니다. 1~5단계에서는 x_i의 변화가 평균, 편차, 분산, 표준편차로 차례로 전해지는 양을 구하고, 6~8단계에서는 그 결과로 출력 $\hat{x}_j$가 얼마나 변하는지 구합니다. 중간에 길을 잃으면 지금 어느 덩어리에 있는지부터 확인하면 됩니다.

1. **평균 μ**: m개 중 하나가 h 늘었으므로 평균은 h/m 늘어납니다.
2. **편차 $x_j - \mu$**: 자기 칸(j = i)은 h − h/m만큼, 다른 칸은 평균만 올랐으므로 −h/m만큼 변합니다.
3. **편차의 합은 0**: 0.9절에서 본 것처럼 편차를 모두 더하면 항상 0입니다. x = [1, 2, 3, 6]의 편차는 [−2, −1, 0, 3]이고, 합은 0입니다. 바로 다음 단계에서 이 성질을 씁니다.
4. **분산 σ^2**: 분산은 편차 제곱의 평균입니다. 0.3절의 표에 따라 편차 하나를 u라고 하면, u^2는 u가 Δ 변할 때 $2u \times \Delta$ 변합니다. 2번의 변화량을 넣으면 두 종류의 항이 생깁니다.
   - 자기 칸에만 있는 항: $2(x_i - \mu) \times h$
   - 모든 칸에 있는 항: $2(x_j - \mu) \times (-h/m)$을 모든 j에 대해 더한 값입니다. 이 값은 $(-2h/m) \times$ (편차의 합)이므로 3번에 따라 0입니다.
   - 분산은 이 합을 m으로 나눈 평균이므로, 결국 $\mathbf{(2/m) \times (x_i - \mu) \times h}$만큼 변합니다.
5. **표준편차 σ**: $\sigma = \sqrt{(\sigma^2 + \varepsilon)}$입니다. 0.3절의 표에서 $\sqrt{x}$의 미분은 $1 \div (2\sqrt{x})$이므로, σ^2가 Δ 변하면 σ는 $\Delta \div (2\sigma)$ 변합니다. 4번의 값을 넣으면 $(x_i - \mu) \times h \div (m \times \sigma)$이고, $\hat{x}_i = (x_i - \mu)/\sigma$이므로 σ는 $\mathbf{\hat{x}_i \times h/m}$만큼 변합니다. 숫자로 확인하면, x = [1, 2, 3, 6]에서 x_1을 0.01 늘렸을 때 이 식은 $-1.069 \times 0.01 \div 4 = -0.00267$을 주고, 실제로 계산한 σ의 변화도 −0.00267입니다.
6. **$\hat{x}_j$의 분자만 변할 때**: $\hat{x}_j$ = (편차) × (1/σ)이므로, 편차가 Δ 변하면 $\hat{x}_j$는 Δ/σ 변합니다(0.3절 표의 c × x).
7. **$\hat{x}_j$의 분모만 변할 때**: 0.3절의 표에서 1/x의 미분은 $-1/x^2$이므로, σ가 Δ 변하면 1/σ는 $-\Delta/\sigma^2$ 변합니다. 따라서 $\hat{x}_j$는 (편차) × $(-\Delta/\sigma^2)$ = $\mathbf{-\hat{x}_j \times \Delta/\sigma}$ 변합니다.
8. **둘을 합치기**: 분자와 분모가 동시에 조금씩 변하면 두 변화량을 그냥 더하면 됩니다(0.8절의 첫 번째 근사). 6번에 2번의 값을, 7번에 5번의 값을 넣어 더하면 다음 표가 됩니다.

칸	$\hat{x}_j$의 변화량
자기 칸 (j = i)	$(h/\sigma) \times (1 - 1/m - \hat{x}_j \times \hat{x}_i / m)$
다른 칸 (j ≠ i)	$(h/\sigma) \times (0 - 1/m - \hat{x}_j \times \hat{x}_i / m)$

이제 softmax 때와 같이 각 칸의 변화량에 g_j를 곱해 모두 더하고 h로 나눕니다. 표의 괄호를 세 덩어리로 나눠 더하면 다음과 같습니다.

- "1" 부분은 자기 칸에만 있으므로 g_i가 됩니다.
- "−1/m" 부분은 모든 칸에 있으므로 −(g의 합)/m = $\mathbf{-mean(g)}$가 됩니다.
- "$-\hat{x}_j \times \hat{x}_i / m$" 부분은 $-\hat{x}_i \times$ ($g_j \times \hat{x}_j$의 합)/m = $\mathbf{-\hat{x}_i \times mean(g \times \hat{x})}$가 됩니다.

$dx_i = (1/\sigma) \times (g_i - mean(g) - \hat{x}_i \times mean(g \times \hat{x}))$

세 항의 의미는 다음과 같습니다.

1. $\mathbf{g_i}$: x_i가 자기 자리의 출력에 직접 주는 영향입니다.

2. **$-\text{mean}(g)$**: x_i를 바꾸면 평균이 바뀌어 행 전체가 함께 이동하는 효과입니다.
3. **$-\hat{x}_i \times \text{mean}(g \times \hat{x})$**: x_i를 바꾸면 표준편차가 바뀌어 행 전체의 크기가 함께 변하는 효과입니다.

숫자로 확인: x = [1, 2, 3, 6]이면 μ = 3, 분산 = (4 + 1 + 0 + 9)/4 = 3.5, σ = 1.8708이고 $\hat{x}$ = [−1.069, −0.5345, 0, 1.6036]입니다. loss를 $\hat{x}_1$ 하나로 정하면 g = [1, 0, 0, 0]이므로, mean(g) = 0.25이고 mean(g × $\hat{x}$) = −1.069/4 = −0.2673입니다.

입력	공식 ($g_i - 0.25 - \hat{x}_i \times (-0.2673)) \div 1.8708$	결과	직접 잰 변화율
x_1	(1 − 0.25 − 0.2857) ÷ 1.8708	0.2482	0.2482
x_2	(0 − 0.25 − 0.1429) ÷ 1.8708	−0.2100	−0.2100
x_3	(0 − 0.25 − 0) ÷ 1.8708	−0.1336	−0.1336
x_4	(0 − 0.25 + 0.4286) ÷ 1.8708	0.0954	0.0954

x_1 자체를 올리면 $\hat{x}_1$이 커지는 것은 당연합니다. 흥미로운 점은 x_4를 올려도 $\hat{x}_1$이 커진다는 것입니다. x_4를 올리면 두 가지 일이 동시에 일어납니다. 평균이 올라가서 $\hat{x}_1$을 더 음수로 끌어내리고(약 −0.134), 동시에 표준편차가 커져서 음수인 $\hat{x}_1$을 0 쪽으로 끌어올립니다(약 +0.229). x_4는 평균에서 멀리 떨어져 있어서 두 번째 효과가 더 크고, 합치면 표의 0.0954가 됩니다.

```
    fn back_layer_norm(
        &self,
        dy: &[f32],
        y: &[f32],
        m: usize,
        a: usize,
    ) -> Vec<(usize, Vec<f32>)> {
        let x = &self.nodes[a].data;
        let mut da = vec![0.0; dy.len()];
        for r in 0..dy.len() / m {
            let row = r * m..(r + 1) * m;
            let (_, rstd) = mean_rstd(&x[row.clone()]);
            let (g, xhat) = (&dy[row.clone()], &y[row.clone()]);
            let g_mean = g.iter().sum::<f32>() / m as f32;
            let gx_mean =
                g.iter().zip(xhat).map(|(g, h)| g * h).sum::<f32>()
                    / m as f32;
            for j in 0..m {
                da[r * m + j] = rstd * (g[j] - g_mean - xhat[j] * gx_mean);
            }
        }
        vec![(a, da)]
    }
```

- $\hat{x}$는 forward의 출력이므로 다시 계산하지 않고 노드의 `data`를 사용합니다.
- rstd는 출력에 저장해 두지 않았으므로 입력 x에서 다시 계산합니다. 저장 공간을 아끼는 대신 계산을 조금 더 하는 선택입니다.
- `row`는 `Range`이고 `Range`는 `Copy`가 아니기 때문에, 여러 번 사용하려면 `.clone()`이 필요합니다.

match에 추가: `input_grads`의 `match`에서 `_ => todo!(...)` 줄 바로 위에 아래 줄을 추가합니다.

```
            Op::LayerNorm(a) => {
                self.back_layer_norm(dy, &node.data, node.cols, *a)
            }
```

테스트: 파일 끝의 `mod tests` 안에서 닫는 `}` 바로 앞에 아래 테스트를 추가합니다.

```
    #[test]
    fn layer_norm() {
        grad_check(&[(3, 5)], |g, x| g.layer_norm(x[0]));
    }
```

확인: `cargo test --release -q layer_norm`을 실행하면 `test result: ok. 1 passed`가 출력됩니다.

2.12 embedding

목표: 토큰 번호 목록을 받아, 표(table)에서 그 번호의 행을 꺼내 행렬로 모으는 연산을 만듭니다.

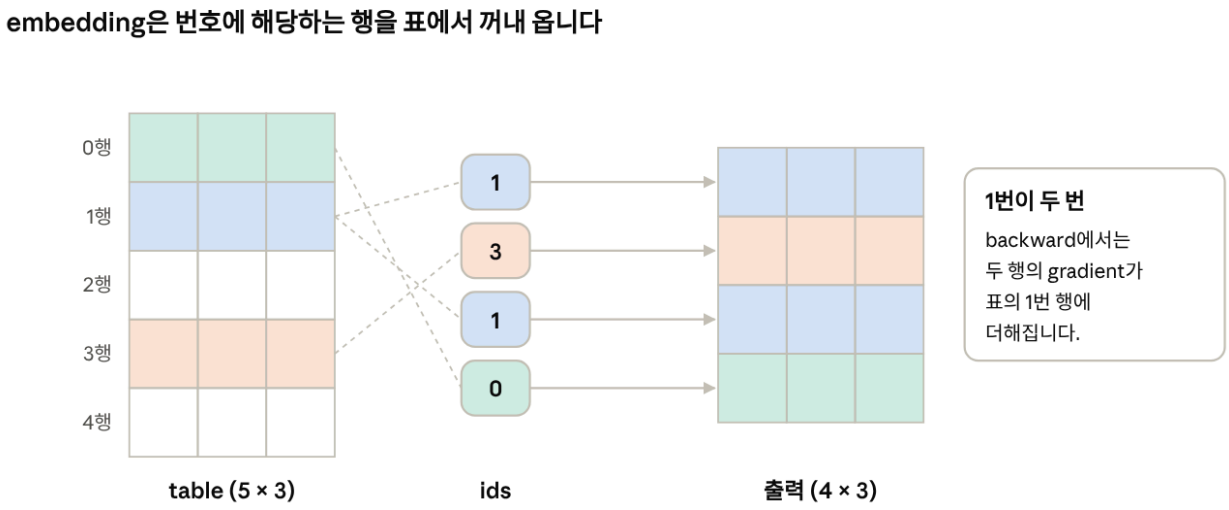


그림 2-5 · 5 × 3 표에서 ids = [1, 3, 1, 0]의 행을 꺼내기

이 그림은 이 절 끝의 embedding 테스트와 같은 예입니다. 같은 색은 같은 행을 뜻합니다.

```
    /// Output row r is row ids[r] of the table.
    pub fn embedding(&mut self, table: T, ids: &[usize]) -> T {
        let (_, m) = self.shape(table);
        let td = self.data(table);
        let mut out = Vec::with_capacity(ids.len() * m);
        for &id in ids {
            out.extend_from_slice(&td[id * m..(id + 1) * m]);
        }
        self.push(out, ids.len(), m, Op::Embedding(table.0, ids.to_vec()))
    }
```

```
    fn back_embedding(
        &self,
        dy: &[f32],
        table: usize,
        ids: &[usize],
```

```
    ) -> Vec<(usize, Vec<f32>)> {
        let m = self.nodes[table].cols;
        let mut dt = vec![0.0; self.nodes[table].data.len()];
        for (r, &id) in ids.iter().enumerate() {
            for j in 0..m {
                // The same id may appear many times.
                dt[id * m + j] += dy[r * m + j];
            }
        }
        vec![(table, dt)]
    }
```

같은 토큰이 문장에 여러 번 나오면 표의 같은 행이 여러 번 복사됩니다. 그래서 backward에서는 2.7절의 broadcast와 마찬가지로, 복사된 횟수만큼 gradient를 `+=`로 더합니다. 문장에 나오지 않은 토큰의 행은 gradient가 0입니다.

match에 추가: `input_grads`의 `match`에서 `_ => todo!(...)` 줄 바로 위에 아래 줄을 추가합니다.

```
            Op::Embedding(table, ids) => {
                self.back_embedding(dy, *table, ids)
            }
```

테스트: 파일 끝의 `mod tests` 안에서 닫는 `}` 바로 앞에 아래 테스트를 추가합니다.

```
    #[test]
    fn embedding() {
        grad_check(&[(5, 3)], |g, x| g.embedding(x[0], &[1, 3, 1, 0]));
    }
```

`embedding` 테스트는 1번 토큰을 두 번 넣습니다. 같은 행이 두 번 쓰일 때 gradient가 더해지는지 확인하려는 것입니다.

확인: `cargo test --release -q embedding`을 실행하면 `test result: ok. 1 passed`가 출력됩니다.

2.13 cross entropy loss

목표: 모델이 내놓은 점수(logits)와 정답 토큰을 비교해, "얼마나 틀렸는가"를 숫자 하나로 만듭니다.

모델은 각 위치에서 어휘 크기만큼의 점수를 냅니다. softmax를 적용하면 "다음 토큰이 각 문자일 확률"이 됩니다. cross entropy는 **정답 문자에 모델이 준 확률의 음의 로그**를 모든 위치에 대해 평균 낸 값입니다.

loss = $-(1/N) \times \Sigma_r \ln p_r[t_r]$ $(r = 1, 2, \cdots, N)$

- 정답에 확률 1을 주면 $-\ln 1 = 0$이므로 loss가 0입니다.
- 정답에 확률을 거의 주지 않으면 loss가 매우 커집니다.
- 모델이 65개 문자 모두에 같은 확률 1/65를 주면 loss는 $-\ln(1/65) = \ln 65 \approx 4.17$입니다. 5부에서 학습을 시작할 때 출력되는 loss가 이 값 근처인지 확인하면, 초기화가 제대로 되었는지 알 수 있습니다.

```
    /// Mean over rows of -log(softmax(row)[target]).
    /// Returns a 1 x 1 tensor.
    pub fn cross_entropy(&mut self, logits: T, targets: &[usize]) -> T {
        let (n, m) = self.shape(logits);
        assert_eq!(n, targets.len(), "cross_entropy: one target per row");
        let mut probs = self.data(logits).to_vec();
```

```
        let mut loss = 0.0;
        for (row, &t) in probs.chunks_mut(m).zip(targets) {
            softmax_in_place(row);
            loss -= row[t].max(1e-30).ln();
        }
        self.push(
            vec![loss / n as f32],
            1,
            1,
            Op::CrossEntropy(logits.0, targets.to_vec()),
        )
    }
```

`max(1e-30)`은 확률이 0이 되어 ln 0 = −∞가 나오는 것을 막습니다.

backward 유도: 행 하나만 보고, 정답 번호를 t, softmax 결과를 p라고 하겠습니다. 이 행의 loss는 $-\ln p_t$입니다. 계산은 "logits → softmax → −ln"의 두 단계이므로, 0.5절의 chain rule대로 뒤에서부터 차례로 구합니다.

1. **−ln 단계**: 0.3절의 표에서 ln y의 미분은 1/y입니다. 따라서 p_t의 gradient는 $-1/p_t$입니다. loss는 p_t에만 의존하므로 나머지 p_j의 gradient는 0입니다.
2. **softmax 단계**: 2.10절에서 구한 공식 $dx_i = p_i \times (g_i - \text{dot})$에 $g = [0, \ldots, -1/p_t, \ldots, 0]$을 넣습니다. $\text{dot} = (-1/p_t) \times p_t = -1$입니다.
    1. 정답 칸 ($i = t$): $p_t \times (-1/p_t + 1) = -1 + p_t = \mathbf{p_t - 1}$
    2. 다른 칸 ($i \neq t$): $p_i \times (0 + 1) = \mathbf{p_i}$

두 경우를 한 줄로 쓰면 "모델이 준 확률에서, 정답 칸에서만 1을 뺀 값"입니다. loss는 N개 행의 평균이므로 마지막에 1/N을 곱합니다.

정답 칸 ($i = t_r$): $d\,\text{logit}_r[i] = (1/N) \times (p_r[i] - 1)$

다른 칸 ($i \neq t_r$): $d\,\text{logit}_r[i] = (1/N) \times p_r[i]$

신기하게도 softmax의 복잡한 식과 −ln의 1/p가 서로 지워져서 매우 단순한 결과가 남았습니다. 정답 칸은 확률이 1보다 작으므로 gradient가 음수가 되고, 경사하강을 하면 점수가 올라갑니다. 오답 칸은 gradient가 양수이므로 점수가 내려갑니다. 두 연산을 하나로 합치면 backward에서 1/p처럼 아주 큰 수가 나오지 않아서 수치도 안정적입니다. PyTorch의 cross_entropy는 forward에서도 softmax를 거치지 않고 $\ln p_t = x_t - \max - \ln \Sigma\, e^{(x_j - \max)}$를 직접 계산해서, 확률이 아주 작은 경우에도 정확한 값을 냅니다. 이 교재는 단순함을 위해 softmax 결과에 ln을 취하고, 확률이 0이 되는 경우만 max(1e-30)으로 막았습니다. 그래서 PyTorch도 `cross_entropy`를 하나의 연산으로 제공합니다.

숫자로 확인: logits = [1, 2, 3]이고 정답이 세 번째 칸이면, p = [0.0900, 0.2447, 0.6652]이고 loss = −ln 0.6652 = 0.4076입니다. 행이 하나뿐이므로 N = 1이고, gradient는 [0.0900, 0.2447, 0.6652 − 1] = [0.0900, 0.2447, −0.3348]입니다. 정답인 세 번째 점수는 올리고, 나머지는 내리라는 뜻입니다.

```
    fn back_cross_entropy(
        &self,
        g: f32,
        logits: usize,
        targets: &[usize],
    ) -> Vec<(usize, Vec<f32>)> {
        let (n, m) = (self.nodes[logits].rows, self.nodes[logits].cols);
        let mut d = self.nodes[logits].data.clone();
        for (row, &t) in d.chunks_mut(m).zip(targets) {
```

```
            softmax_in_place(row);
            row[t] -= 1.0; // softmax - one_hot(target)
            for x in row.iter_mut() {
                *x *= g / n as f32;
            }
        }
        vec![(logits, d)]
    }
```

- forward에서 계산한 확률을 저장해 두지 않았으므로, logits에서 softmax를 다시 계산합니다.
- `g`는 loss 노드에 도착한 gradient입니다. loss가 마지막 노드라면 항상 1이지만, chain rule을 지키기 위해 곱해 둡니다.

match에 추가: `input_grads`의 `match`에서 `_ => todo!(...)` 줄 바로 위에 아래 줄을 추가합니다.

```
            Op::CrossEntropy(logits, targets) => {
                self.back_cross_entropy(dy[0], *logits, targets)
            }
```

테스트: 파일 끝의 `mod tests` 안에서 닫는 `}` 바로 앞에 아래 테스트를 추가합니다.

```
    #[test]
    fn cross_entropy() {
        grad_check(&[(3, 5)], |g, x| g.cross_entropy(x[0], &[0, 2, 4]));
    }
```

cross entropy의 출력은 이미 1 × 1이지만, 다른 테스트와 똑같이 `reduce`를 거칩니다. 1 × 1 행렬에 무작위 가중치 하나를 곱해도 검사에는 문제가 없습니다.

확인: `cargo test --release -q cross_entropy`를 실행하면 `test result: ok. 1 passed`가 출력됩니다.

2.14 완성된 input_grads와 전체 테스트

목표: 2부를 마무리합니다. 지금까지 연산마다 한 줄씩 추가한 `match`가 완성되었는지 확인하고, 테스트 전체를 실행합니다.

모든 연산의 줄을 추가하면 `input_grads`는 아래와 같아집니다. 모든 `Op`를 다루게 되었으므로 `_ => todo!(...)` 줄은 이제 지웁니다. 지우지 않으면 "unreachable pattern" 경고가 나옵니다.

```
    /// For node i, return pairs of
    /// (input index, gradient to add to that input).
    fn input_grads(&self, i: usize) -> Vec<(usize, Vec<f32>)> {
        let node = &self.nodes[i];
        let dy = &node.grad;
        match &node.op {
            Op::Leaf => vec![],
            Op::MatMul(a, b) => self.back_matmul(dy, *a, *b),
            Op::Add(a, b) => vec![(*a, dy.clone()), (*b, dy.clone())],
            Op::AddRow(a, b) => self.back_add_row(dy, *a, *b),
            Op::MulRow(a, b) => self.back_mul_row(dy, *a, *b),
            Op::Scale(a, s) => {
                vec![(*a, dy.iter().map(|g| g * s).collect())]
            }
```

```
            Op::Relu(a) => self.back_relu(dy, *a),
            Op::Transpose(a) => self.back_transpose(dy, *a),
            Op::SliceRows(a, start) => self.back_slice_rows(dy, *a, *start),
            Op::SliceCols(a, start) => {
                self.back_slice_cols(dy, node.cols, *a, *start)
            }
            Op::ConcatRows(parts) => self.back_concat_rows(dy, parts),
            Op::ConcatCols(parts) => {
                self.back_concat_cols(dy, node.rows, parts)
            }
            Op::CausalMask(a) => self.back_causal_mask(dy, node.cols, *a),
            Op::Softmax(a) => {
                self.back_softmax(dy, &node.data, node.cols, *a)
            }
            Op::LayerNorm(a) => {
                self.back_layer_norm(dy, &node.data, node.cols, *a)
            }
            Op::Embedding(table, ids) => {
                self.back_embedding(dy, *table, ids)
            }
            Op::CrossEntropy(logits, targets) => {
                self.back_cross_entropy(dy[0], *logits, targets)
            }
        }
    }
```

확인: `cargo test --release -q`를 실행하면 8개 테스트가 모두 통과합니다. `-q`를 빼면 테스트 이름이 한 줄에 하나씩 `... ok`와 함께 출력됩니다.

```
running 8 tests
........
test result: ok. 8 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

테스트가 정말 버그를 잡는지 궁금하다면 코드를 일부러 틀리게 바꿔 볼 수 있습니다. 예를 들어 `back_softmax`의 `dx[j] = y[j] * (g[j] - dot);`를 `dx[j] = y[j] * g[j];`로 바꾸면 `mask_and_softmax` 테스트가 실패합니다. 반면 `back_matmul`을 틀리게 바꾸면 8개가 모두 실패합니다. 2.6절에서 설명한 대로 검사 도구 자체가 matmul을 쓰기 때문입니다. 이 교재의 테스트도 이 방법으로 버그를 잡아내는지 확인했습니다. 테스트가 실패하는데 원인을 모르겠다면 부록 D를 참고합니다.

3부 데이터

3부가 끝나면 셰익스피어 텍스트 111만 글자를 숫자 목록으로 바꾸고, 그중 임의의 구간을 골라 "입력"과 "정답" 쌍을 만들 수 있게 됩니다. 3부는 짧지만, GPT가 무엇을 학습하는지가 여기서 정해집니다.

3.1 GPT가 풀려는 문제

GPT가 하는 일은 단 하나입니다. **지금까지의 글자를 보고 다음 글자를 맞히는 것**입니다. 예를 들어 `the groun`을 보여주면 다음 글자로 `d`에 높은 확률을 주어야 합니다.

이 문제에는 편리한 점이 있습니다. 정답을 사람이 따로 달아 줄 필요가 없습니다. 텍스트 자체에 이미 "다음 글자"가 적혀 있기 때문입니다. 그래서 텍스트만 있으면 얼마든지 학습 데이터를 만들 수 있습니다.

문장을 생성하는 방법도 이 문제에서 바로 나옵니다. 다음 글자를 하나 뽑고, 그 글자를 입력 끝에 붙여서 다시 다음 글자를 뽑는 일을 반복하면 됩니다.

3.2 문자 단위 tokenizer (`src/data.rs`)

목표: 글자를 숫자(토큰 번호)로, 숫자를 다시 글자로 바꿉니다. 신경망은 숫자만 다룰 수 있기 때문입니다.

```
use crate::rng::Rng;

/// Character-level tokenizer: every distinct character is one token.
pub struct Tokenizer {
    chars: Vec<char>, // token id -> character
}

impl Tokenizer {
    /// Build the vocabulary from every character that appears in `text`.
    pub fn new(text: &str) -> Self {
        let mut chars: Vec<char> = text.chars().collect();
        chars.sort();
        chars.dedup();
        Tokenizer { chars }
    }

    pub fn vocab_size(&self) -> usize {
        self.chars.len()
    }

    /// Text -> token ids.
    /// Panics on a character that is not in the vocabulary.
    pub fn encode(&self, text: &str) -> Vec<usize> {
        text.chars()
            .map(|c| {
                self.chars
                    .binary_search(&c)
                    .expect("character not in vocabulary")
            })
            .collect()
    }

    /// Token ids -> text.
```

```
    pub fn decode(&self, ids: &[usize]) -> String {
        ids.iter().map(|&i| self.chars[i]).collect()
    }
}
```

- `new`는 텍스트에 나오는 모든 글자를 모아 정렬하고(`sort`), 연달아 같은 값이 나오면 하나만 남깁니다(`dedup`). 정렬한 뒤에는 같은 글자가 모두 붙어 있으므로, 결과적으로 서로 다른 글자만 남습니다.
- 글자의 번호는 이 `Vec` 안에서의 위치입니다. 셰익스피어 텍스트에는 서로 다른 글자가 65개 있으므로 번호는 0부터 64까지입니다. 이 65를 **어휘 크기(vocab size)**라고 부릅니다.
- `encode`는 `chars`가 정렬되어 있다는 점을 이용해 `binary_search`로 글자의 위치를 찾습니다. 글자가 65개뿐이라 `HashMap`을 따로 만들 필요가 없습니다.
- `decode`는 번호를 인덱스로 삼아 글자를 꺼내고, `collect()`로 `String`을 만듭니다.

ChatGPT 같은 실제 LLM은 글자 대신 BPE(Byte Pair Encoding)로 만든 수만~수십만 개의 조각(단어 일부)을 토큰으로 씁니다. 토큰 하나가 더 많은 글자를 담으므로 같은 문맥 길이로 더 긴 문장을 볼 수 있기 때문입니다. 하지만 모델 구조는 토큰이 무엇이든 똑같으므로, 공부용으로는 가장 단순한 문자 단위를 씁니다.

3.3 학습 배치 만들기

목표: 긴 토큰 목록에서 학습용 "입력과 정답" 쌍을 무작위로 뽑습니다.

먼저 무작위 위치를 고르는 함수가 필요합니다. `src/rng.rs`의 `impl Rng` 블록 안에 아래 함수를 추가합니다.

```
    /// Uniform integer in [0, n).
    pub fn below(&mut self, n: usize) -> usize {
        (self.uniform() * n as f64) as usize
    }
```

`below(n)`은 0부터 n − 1까지의 정수 중 하나를 고르게 뽑습니다. 0 이상 1 미만의 실수에 n을 곱하면 0 이상 n 미만의 실수가 되고, `as usize`가 소수점 아래를 버리기 때문입니다.

이제 `src/data.rs`에 배치를 만드는 함수를 추가합니다.

```
/// Pick `batch` random windows of `seq + 1` tokens from `data`.
/// Inputs are the first `seq` tokens of each window, targets the last `seq`
/// (the same window shifted by one).
/// Both come back flattened: batch * seq ids.
pub fn get_batch(
    data: &[usize],
    batch: usize,
    seq: usize,
    rng: &mut Rng,
) -> (Vec<usize>, Vec<usize>) {
    let mut inputs = Vec::with_capacity(batch * seq);
    let mut targets = Vec::with_capacity(batch * seq);
    for _ in 0..batch {
        let start = rng.below(data.len() - seq);
        inputs.extend_from_slice(&data[start..start + seq]);
        targets.extend_from_slice(&data[start + 1..start + seq + 1]);
    }
    (inputs, targets)
}
```

길이가 seq + 1인 구간을 하나 골라, 앞의 seq개는 입력으로, 한 칸 밀린 seq개는 정답으로 씁니다. 예를 들어 구간이 `hello!`이고 seq가 5라면 다음과 같습니다.

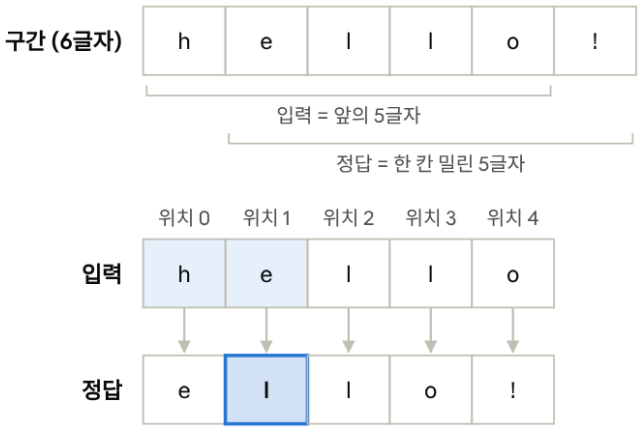
위치	0	1	2	3	4
입력	h	e	l	l	o
정답	e	l	l	o	!

이 표의 각 열은 하나의 독립된 문제입니다.

- 위치 0: `h`를 보고 `e`를 맞힙니다.
- 위치 1: `he`를 보고 `l`을 맞힙니다.
- 위치 4: `hello`를 보고 `!`를 맞힙니다.

따라서 구간 하나에서 seq개의 문제를 한 번에 학습합니다. 단, 위치 1의 문제를 풀 때 위치 2 이후의 입력(즉 정답의 일부)을 보면 안 됩니다. 2부에서 만든 causal mask가 바로 이 "미래를 보지 못하게 하는" 역할을 합니다.

정답은 입력을 한 칸 밀어서 만듭니다



예: 위치 1의 문제는 h, e만 보고 l을 맞히는 것입니다. 그 뒤의 입력 l, l, o는 볼 수 없습니다.

그림 3-1 · 구간 hello!에서 입력과 정답 만들기

각 위치는 입력에서 자기 위치까지만 보고, 바로 아래 칸의 정답을 맞힙니다.

- `batch`는 이런 구간을 한 번에 몇 개 뽑을지를 정합니다. 여러 구간의 gradient를 평균 내면 한 구간만 볼 때보다 방향이 덜 흔들립니다.
- 결과는 batch × seq 길이의 평탄한 `Vec` 두 개입니다. 첫 구간의 seq개, 두 번째 구간의 seq개가 차례로 이어집니다. 4부의 모델도 이 순서를 그대로 따릅니다.
- `start`의 범위가 `data.len() - seq` 미만이므로, 정답 구간의 끝 `start + seq + 1`은 최대 `data.len()`입니다. 따라서 배열 범위를 벗어나지 않습니다.

3.4 확인해 보기

목표: tokenizer와 배치가 제대로 동작하는지 눈으로 확인합니다.

`src/main.rs`를 아래처럼 바꿉니다. `mod data;`와 `"data"` 명령, `data_demo` 함수가 새로 들어갔습니다.

```
mod data;
mod rng;
mod scalar;
mod tensor;

use data::{get_batch, Tokenizer};
use rng::Rng;

fn main() {
    let cmd = std::env::args().nth(1).unwrap_or_default();
    match cmd.as_str() {
        "scalar" => scalar::demo(),
        "data" => data_demo(),
        _ => println!("usage: cargo run --release -- [scalar|data]"),
    }
}

fn data_demo() {
    let text =
        std::fs::read_to_string("input.txt").expect("input.txt not found");
    let tok = Tokenizer::new(&text);
    println!(
        "characters: {}  vocab: {}",
        text.chars().count(),
        tok.vocab_size()
    );
    let ids = tok.encode("Hello!");
    println!(
        "encode(\"Hello!\") = {ids:?}  decode -> {:?}",
        tok.decode(&ids)
    );

    let data = tok.encode(&text);
    let mut rng = Rng::new(1);
    let (x, y) = get_batch(&data, 1, 16, &mut rng);
    println!("input : {:?}", tok.decode(&x));
    println!("target: {:?}", tok.decode(&y));
}
```

`input.txt`는 머리말에서 받아 둔 파일입니다. `cargo run`은 프로젝트 최상위 폴더(`Cargo.toml`이 있는 곳)에서 실행해야 이 파일을 찾을 수 있습니다.

확인: `cargo run --release -- data`를 실행하면 아래와 같이 출력됩니다.

```
characters: 1115394  vocab: 65
encode("Hello!") = [20, 43, 50, 50, 53, 2]  decode -> "Hello!"
input : " the ground.\nNot"
target: "the ground.\nNot "
```

- 대문자 `H`(20)가 소문자 `e`(43)보다 작은 번호를 받았습니다. 유니코드 순서로 정렬했기 때문입니다.
- `target`은 `input`을 정확히 한 글자 왼쪽으로 밀어 놓은 문자열입니다.
- `\n`은 줄바꿈 문자입니다. 모델 입장에서는 줄바꿈도 맞혀야 할 문자 중 하나입니다. 그래서 학습이 끝난 모델은 대본처럼 줄을 바꾸며 문장을 생성합니다.

4부 Transformer 부품

4부가 끝나면 transformer block 하나를 구성하는 부품 다섯 개(Linear, LayerNorm, Attention, Mlp, Block)가 완성됩니다. 모든 부품은 2부의 연산을 조합해서만 만들기 때문에, 여기서는 backward를 한 줄도 작성하지 않습니다. 이 부분의 코드는 모두 `src/model.rs`에 들어갑니다.

4.1 GPT의 전체 구조

부품을 만들기 전에, 부품들이 어떻게 연결되는지 먼저 살펴보겠습니다. 토큰 번호 목록이 들어가서 "각 위치의 다음 토큰에 대한 점수"가 나오기까지 다음 단계를 거칩니다.

1. **embedding**: 토큰 번호를 길이 n_embd(이 교재에서는 64)의 벡터로 바꿉니다. 위치 번호도 같은 길이의 벡터로 바꾸어 더합니다.
2. **transformer block × n_layer**: 벡터를 조금씩 고쳐 씁니다. block 하나는 두 단계로 이루어져 있습니다.
    1. **attention**: 각 위치가 앞쪽 위치들에서 필요한 정보를 가져옵니다. 위치끼리 정보를 주고받는 유일한 단계입니다.
    2. **MLP**: 각 위치가 가져온 정보를 다른 위치와 상관없이 따로 가공합니다.
3. **마지막 layernorm과 head**: 각 위치의 벡터를 어휘 크기(65)만큼의 점수로 바꿉니다.

이 과정에서 벡터를 담는 행렬의 크기는 처음부터 끝까지 **(batch × seq) × n_embd**로 유지됩니다. 행 하나가 문장 안의 위치 하나입니다. 마지막 head만 열 수를 n_embd에서 vocab으로 바꿉니다. 여기서 head는 출력층을 뜻하며, 4.7절의 attention head와는 다른 것입니다.

각 부품은 residual 통로에 더해질 뿐입니다

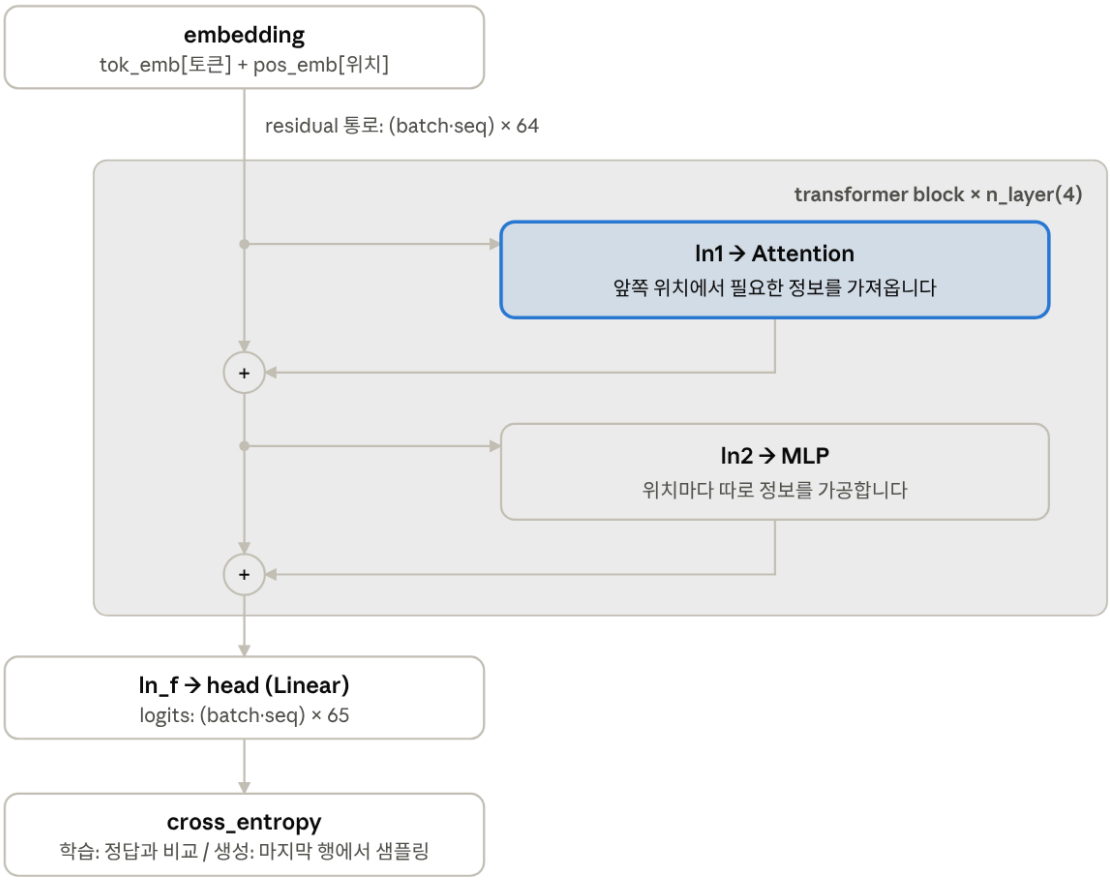


그림 4-1 · GPT forward 흐름, block 하나를 펼쳐 그린 그림

왼쪽의 세로선이 residual 통로입니다. attention과 MLP는 통로의 값을 읽어서 계산한 뒤, 그 결과를 통로에 더합니다 (4.10절). 강조한 attention만 위치끼리 정보를 섞습니다.

4.2 설정값 (Config)

목표: 모델의 크기를 정하는 숫자 다섯 개를 한곳에 모읍니다.

```
use crate::rng::Rng;
use crate::tensor::{Graph, T};

/// Model size settings.
#[derive(Clone, Copy)]
pub struct Config {
    pub vocab: usize,      // number of distinct tokens
    pub block_size: usize, // longest context the model can see
    pub n_embd: usize,     // width of every token vector
    pub n_head: usize,     // attention heads per block
    pub n_layer: usize,    // number of transformer blocks
}
```

이름	이 교재의 값	GPT-2 small의 값	의미
`vocab`	65	50257	토큰 종류의 수
`block_size`	32	1024	모델이 한 번에 볼 수 있는 최대 토큰 수(transformer block과는 관계없습니다)
`n_embd`	64	768	토큰 하나를 표현하는 벡터의 길이
`n_head`	4	12	attention head의 수
`n_layer`	4	12	transformer block의 수

구조의 골격(pre-norm block, 학습되는 위치 embedding, 4배 MLP)은 GPT-2와 같고 숫자만 작습니다. 활성화 함수(ReLU와 GELU)와 출력층(가중치 공유 여부, bias 유무) 같은 세부 사항은 다르며, 부록 C에서 정리합니다. 이 교재의 모델은 파라미터가 약 21만 개이고, GPT-2 small은 약 1억 2천4백만 개입니다.

main.rs에 추가: `src/main.rs`의 맨 위에 `mod model;`을 추가합니다. 이제부터 `src/model.rs`도 컴파일 대상이 되므로, 4부의 각 절을 마칠 때마다 `cargo build --release`로 오타가 없는지 확인할 수 있습니다. 아직 호출하지 않는 코드가 많아서 "never used"나 "unused import" 경고가 나오는 것은 정상입니다.

4.3 파라미터 만들기 (ParamBuilder)

목표: 파라미터를 그래프에 만들면서, 만든 파라미터를 빠짐없이 목록에 기록합니다. optimizer는 이 목록에 있는 tensor만 수정합니다.

```
/// Creates parameters on the graph and remembers them for the optimizer.
pub struct ParamBuilder<'a> {
    pub g: &'a mut Graph,
    pub rng: &'a mut Rng,
    pub params: Vec<T>,
}

impl ParamBuilder<'_> {
    /// rows x cols matrix with values drawn from N(0, std^2).
    pub fn normal(&mut self, rows: usize, cols: usize, std: f32) -> T {
        let data = (0..rows * cols)
            .map(|_| self.rng.normal() as f32 * std)
            .collect();
        let t = self.g.leaf(data, rows, cols);
        self.params.push(t);
        t
    }

    /// rows x cols matrix filled with one value.
    pub fn constant(&mut self, rows: usize, cols: usize, value: f32) -> T {
        let t = self.g.leaf(vec![value; rows * cols], rows, cols);
        self.params.push(t);
        t
    }
}
```

- 1부에서는 `Neuron`, `Layer`, `Mlp`가 각각 `params()` 함수를 가지고 있었습니다. 부품이 다섯 종류로 늘어나면 같은 코드를 다섯 번 써야 하므로, 여기서는 만드는 순간에 목록에 넣도록 바꾸었습니다.
- `<'a>`는 lifetime 표기입니다. `ParamBuilder`가 `Graph`와 `Rng`를 빌려서 들고 있으므로, 빌린 대상보다 오래 살아남을 수 없다는 사실을 컴파일러에 알려 줍니다. `impl ParamBuilder<'_>`의 `'_`는 "어떤 lifetime이든 상관없다"는 뜻입니다.
- 가중치 행렬은 평균 0, 표준편차 0.02인 정규분포로 초기화합니다. GPT-2의 기본값과 같습니다. 다만 GPT-2는 residual 통로에 더해지는 마지막 Linear(attention과 MLP의 proj)의 표준편차를 1/√(2 × n_layer)만큼 더 줄이는데, 이 교재는 층이 4개뿐이라 이 단계를 생략했습니다. 초기값이 너무 크면 출력 점수가 한쪽으로 크게 치우쳐서 초기 loss가 4.17(무작위 추측)보다 훨씬 커지고, 학습 초반이 불안정해집니다.
- bias는 0으로, layernorm의 gamma는 1로 시작합니다. `constant`가 이 경우를 처리합니다.

4.4 Linear

목표: 신경망의 가장 기본적인 부품인 "행렬을 곱하고 bias를 더하는 층"을 만듭니다.

```
/// y = x @ w + b
pub struct Linear {
    w: T, // n_in x n_out
    b: T, // 1 x n_out
}

impl Linear {
    pub fn new(p: &mut ParamBuilder, n_in: usize, n_out: usize) -> Self {
        Linear {
            w: p.normal(n_in, n_out, 0.02),
            b: p.constant(1, n_out, 0.0),
        }
    }

    pub fn forward(&self, g: &mut Graph, x: T) -> T {
        let y = g.matmul(x, self.w);
        g.add_row(y, self.b)
    }
}
```

1부의 `Layer`와 하는 일이 같습니다. 다른 점은 뉴런별 반복문 대신 행렬 곱셈 한 번으로 처리한다는 것과, 비선형 함수(tanh)를 포함하지 않는다는 것입니다. x의 행 하나하나가 각각 독립된 입력이며, 모든 행에 같은 w와 b가 적용됩니다.

4.5 LayerNorm

목표: 2부의 `layer_norm` 연산에 학습되는 배율(gamma)과 이동량(beta)을 붙입니다.

```
/// Normalize each row, then scale by gamma and shift by beta
/// (both learned).
pub struct LayerNorm {
    gamma: T, // 1 x n
    beta: T,  // 1 x n
}

impl LayerNorm {
    pub fn new(p: &mut ParamBuilder, n: usize) -> Self {
```

```
        LayerNorm {
            gamma: p.constant(1, n, 1.0),
            beta: p.constant(1, n, 0.0),
        }
    }

    pub fn forward(&self, g: &mut Graph, x: T) -> T {
        let y = g.layer_norm(x);
        let y = g.mul_row(y, self.gamma);
        g.add_row(y, self.beta)
    }
}
```

평균 0, 분산 1로 강제로 맞추기만 하면 모델이 표현할 수 있는 값의 범위가 좁아집니다. 그래서 열마다 배율과 이동량을 학습하게 해서, 필요하면 모델이 정규화를 일부 되돌릴 수 있게 합니다. gamma가 1이고 beta가 0인 초기 상태는 "순수한 정규화"에서 출발한다는 뜻입니다.

4.6 attention의 아이디어

attention은 transformer의 핵심 부품입니다. 코드를 보기 전에 무엇을 하려는지 먼저 정리하겠습니다.

`the groun` 다음에 `d`를 맞히려면, 마지막 위치(`n`)는 앞쪽에 있는 `g`, `r`, `o`, `u`를 참고해야 합니다. 하지만 지금까지의 부품(Linear, LayerNorm)은 모두 행마다 따로 계산하므로, 어떤 위치도 다른 위치를 볼 수 없습니다. attention은 각 위치가 앞쪽 위치들의 정보를 가져올 수 있게 해 줍니다.

이때 앞쪽 위치를 모두 같은 비중으로 보는 것이 아니라, **지금 필요한 위치를 골라서** 봅니다. 이를 위해 각 위치는 세 가지 벡터를 만듭니다.

이름	의미	비유하자면
query (q)	내가 찾는 정보는 무엇인가	검색어
key (k)	나는 어떤 정보를 가지고 있는가	문서의 제목
value (v)	나를 참고하는 위치에 실제로 건네줄 내용	문서의 본문

계산은 다음 순서로 진행됩니다.

1. **점수 계산**: 위치 i의 query와 위치 j의 key를 내적합니다(0.10절). 두 벡터가 비슷한 방향일수록, 그리고 벡터가 클수록 점수가 커집니다. 모든 (i, j) 쌍에 대해 한 번에 계산하면 $q \cdot k^T$라는 seq × seq 행렬이 됩니다.
2. **크기 조절**: 점수를 √(head_size)로 나눕니다. q와 k의 각 칸이 평균 0, 분산 1인 무작위 값이라면, 두 벡터의 내적은 head_size개의 곱을 더한 값이므로 분산이 head_size가 되고 전형적인 크기는 √(head_size)가 됩니다. √(head_size)로 나누면 head_size와 상관없이 점수의 크기가 1 정도로 유지됩니다. 점수가 너무 크면 softmax가 값 하나에만 거의 1을 주고 나머지에는 거의 0을 주게 되어, gradient가 거의 흐르지 않고 학습이 매우 느려집니다.
3. **미래 가리기**: j > i인 점수를 음의 무한대로 바꿉니다(causal mask). 3.3절에서 설명한 대로, 미래 위치는 정답을 담고 있기 때문입니다.
4. **비중으로 바꾸기**: 행마다 softmax를 적용합니다. 각 행은 합이 1인 비중이 되고, 가려진 위치의 비중은 정확히 0입니다.
5. **가져오기**: 비중으로 value를 가중 평균합니다. 점수가 높았던 위치의 value가 많이 섞여 들어옵니다.

$$\text{Attention}(Q, K, V) = \text{softmax}(\text{mask}(QK^T \div \sqrt{d})) \times V$$

식의 d는 head_size입니다. q, k, v는 모두 같은 입력 x에 서로 다른 Linear를 적용해서 만듭니다. 그 Linear의 가중치도 학습되므로, "무엇을 찾고 무엇을 내어줄지"도 모델이 스스로 배웁니다.

숫자로 따라가기: 위치가 3개이고 head_size가 2인 작은 예를 직접 계산해 보겠습니다. q, k, v는 모두 3 × 2 행렬이고, 행 하나가 위치 하나입니다.

위치	q	k	v
0	[1, 0]	[1, 0]	[1, 0]
1	[0, 1]	[0, 1]	[0, 1]
2	[1, 1]	[1, 1]	[2, 2]

1단계, **점수 계산** (3 × 3): (i, j) 칸은 위치 i의 q와 위치 j의 k의 내적입니다. 예를 들어 (2, 0) 칸은 [1, 1]과 [1, 0]의 내적이므로 1입니다.

	j = 0	j = 1	j = 2
i = 0	1	0	1
i = 1	0	1	1
i = 2	1	1	2

2단계, **크기 조절**: 모든 칸을 $\sqrt{2} \approx 1.414$로 나눕니다. 1은 0.7071, 2는 1.4142가 됩니다.

3단계, **미래 가리기**: j > i인 칸, 즉 (0, 1), (0, 2), (1, 2)를 $-\infty$로 바꿉니다. 위치 0은 자기 자신만, 위치 1은 0과 1만 볼 수 있습니다.

4단계, **비중으로 바꾸기** (3 × 3): 행마다 softmax를 적용합니다. $-\infty$였던 칸은 0이 됩니다.

	j = 0	j = 1	j = 2
i = 0	1	0	0
i = 1	0.3302	0.6698	0
i = 2	0.2483	0.2483	0.5035

위치 2의 query [1, 1]은 위치 2의 key [1, 1]과 가장 같은 방향이므로, 자기 자신에게 가장 큰 비중(0.5035)을 줍니다.

5단계, **가져오기** (3 × 2): 각 행의 비중으로 v를 가중 평균합니다. 위치 2의 출력은 0.2483 × [1, 0] + 0.2483 × [0, 1] + 0.5035 × [2, 2] = [1.2552, 1.2552]입니다. 위치 0의 출력은 자기 v인 [1, 0] 그대로이고, 위치 1의 출력은 [0.3302, 0.6698]입니다.

다음 절의 코드는 이 다섯 단계를 한 줄씩 그대로 옮긴 것입니다.

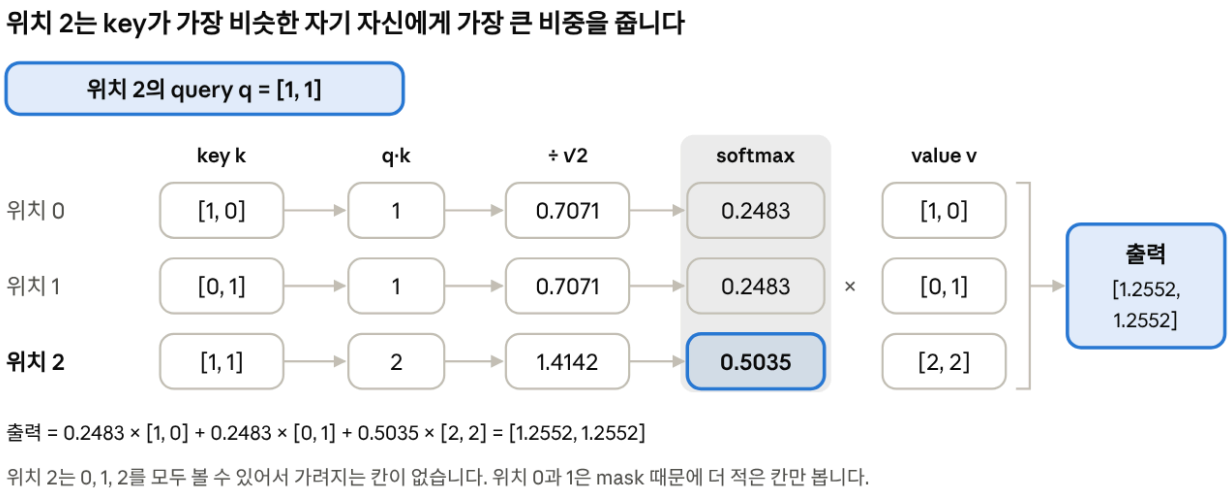


그림 4-2 · 위치 2의 attention 다섯 단계

위 예에서 위치 2의 계산만 꺼내 그린 그림입니다. 한 줄이 위치 2가 참고하는 위치 하나이고, 왼쪽에서 오른쪽으로 1단계부터 5단계까지 진행됩니다.

4.7 attention head 하나

목표: 문장 하나에 대해 4.6절의 다섯 단계를 그대로 코드로 옮깁니다.

```
/// Causal multi-head self-attention.
pub struct Attention {
    qkv: Linear,  // n_embd -> 3 * n_embd (query, key, value side by side)
    proj: Linear, // n_embd -> n_embd
    n_head: usize,
}

impl Attention {
    pub fn new(p: &mut ParamBuilder, cfg: &Config) -> Self {
        Attention {
            qkv: Linear::new(p, cfg.n_embd, 3 * cfg.n_embd),
            proj: Linear::new(p, cfg.n_embd, cfg.n_embd),
            n_head: cfg.n_head,
        }
    }

    /// One head for one sequence: q, k, v are seq x head_size.
    fn head(g: &mut Graph, q: T, k: T, v: T) -> T {
        let head_size = g.shape(q).1;
        let kt = g.transpose(k); // head_size x seq
        // seq x seq: how much each query matches each key
        let scores = g.matmul(q, kt);
        let scores = g.scale(scores, 1.0 / (head_size as f32).sqrt());
        let scores = g.causal_mask(scores); // no peeking at future tokens
        let weights = g.softmax(scores); // each row sums to 1
        g.matmul(weights, v) // seq x head_size: weighted average of values
    }
```

- q, k, v를 만드는 Linear 세 개를 따로 두지 않고, 출력이 3배 넓은 Linear 하나(`qkv`)로 한 번에 만듭니다. 결과 행렬의 열을 세 등분하면 앞에서부터 q, k, v가 됩니다. 계산 결과는 세 개를 따로 두는 것과 같고, 행렬 곱셈 횟수만

줄어듭니다.

- `proj`는 여러 head의 결과를 섞어 주는 마지막 Linear입니다. 4.8절에서 사용합니다.
- `head`의 각 줄은 4.6절 목록의 한 단계에 대응합니다. 크기가 바뀌는 과정을 주석으로 적어 두었습니다. 아래에서 hs는 head_size를 줄여 쓴 것입니다. (seq × hs)와 (hs × seq)를 곱해 (seq × seq) 점수표가 되고, 이 점수표와 (seq × hs)를 곱해 다시 (seq × hs)가 됩니다.
- `head`는 `self`를 쓰지 않으므로 연관 함수(associated function)로 만들었습니다. 호출할 때는 `Self::head(...)`로 부릅니다.

4.8 multi-head와 batch

목표: head 여러 개를 동시에 쓰고, batch 안의 문장 여러 개를 한 번에 처리합니다.

head를 여러 개 두는 이유는, 한 위치가 동시에 여러 종류의 정보를 찾아야 하기 때문입니다. 예를 들어 한 head는 "바로 앞 글자"를, 다른 head는 "이 단어가 시작된 위치"를 보도록 학습될 수 있습니다. n_embd가 64이고 head가 4개라면, 각 head는 길이 16짜리 q, k, v를 씁니다. 네 head의 결과를 옆으로 이어 붙이면 다시 길이 64가 됩니다.

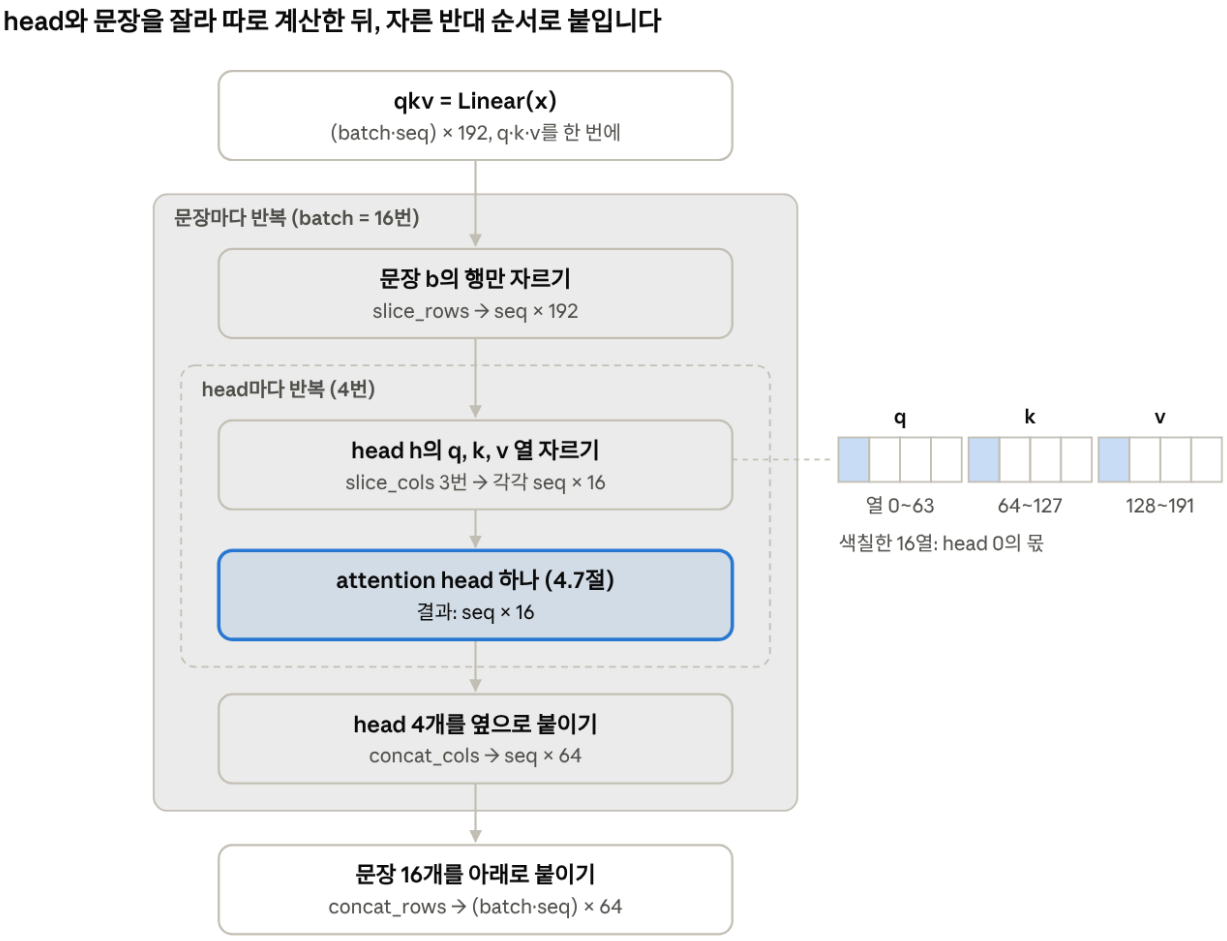


그림 4-3 · multi-head attention의 자르기와 붙이기

오른쪽 띠는 qkv 행렬의 열 192개입니다. head h는 q, k, v 구역에서 각각 16열씩 자기 몫을 잘라 씁니다. 아래 코드의 두 반복문이 그림의 두 테두리에 해당합니다.

```
    /// x: (batch * seq) x n_embd -> same shape.
    pub fn forward(
        &self,
        g: &mut Graph,
        x: T,
        batch: usize,
        seq: usize,
    ) -> T {
        let c = g.shape(x).1;
        let hs = c / self.n_head;
        let qkv = self.qkv.forward(g, x); // (batch * seq) x 3c
        let mut seqs = Vec::with_capacity(batch);
        for b in 0..batch {
            // seq x 3c: the rows of one sequence
            let rows = g.slice_rows(qkv, b * seq, seq);
            let mut heads = Vec::with_capacity(self.n_head);
            for h in 0..self.n_head {
                let q = g.slice_cols(rows, h * hs, hs);
                let k = g.slice_cols(rows, c + h * hs, hs);
                let v = g.slice_cols(rows, 2 * c + h * hs, hs);
                heads.push(Self::head(g, q, k, v));
            }
            seqs.push(g.concat_cols(&heads)); // seq x c
        }
        let y = g.concat_rows(&seqs); // (batch * seq) x c
        self.proj.forward(g, y)
    }
}
```

열을 자르는 위치를 표로 보면 다음과 같습니다(c = 64, hs = 16).

열 범위	0~15	16~31	32~47	48~63	64~79	…	128~143	…
내용	head 0의 q	head 1의 q	head 2의 q	head 3의 q	head 0의 k	…	head 0의 v	…

- 바깥 반복문은 batch 안의 문장을 하나씩 꺼냅니다. 3.3절에서 만든 배치는 문장들을 차례로 이어 붙였으므로, b번째 문장은 `b * seq` 행부터 seq개 행입니다. 문장을 이렇게 나누지 않으면 서로 다른 문장의 글자끼리 attention을 하게 됩니다.
- 안쪽 반복문은 head를 하나씩 처리합니다. 각 head는 자기 몫의 q, k, v 열만 잘라서 씁니다.
- 결과는 잘랐던 순서의 반대로 합칩니다. head는 옆으로(`concat_cols`), 문장은 아래로(`concat_rows`) 이어 붙입니다.
- batch가 16이고 head가 4개이면 `head`가 block 하나에서 64번 호출됩니다. PyTorch는 이 64번의 계산을 4차원 tensor 하나로 묶어 한 번에 처리하지만, 계산 내용은 똑같습니다.

4.9 MLP

목표: attention으로 모은 정보를 위치마다 가공하는 작은 신경망을 만듭니다.

```
/// Position-wise feed-forward network: expand 4x, ReLU, project back.
pub struct Mlp {
    fc: Linear,
    proj: Linear,
}
```

```
impl Mlp {
    pub fn new(p: &mut ParamBuilder, n_embd: usize) -> Self {
        Mlp {
            fc: Linear::new(p, n_embd, 4 * n_embd),
            proj: Linear::new(p, 4 * n_embd, n_embd),
        }
    }

    pub fn forward(&self, g: &mut Graph, x: T) -> T {
        let h = self.fc.forward(g, x);
        let h = g.relu(h);
        self.proj.forward(g, h)
    }
}
```

- 폭을 4배(64 → 256)로 넓혔다가 ReLU를 거쳐 다시 원래 폭(256 → 64)으로 줄입니다. 넓은 공간에서 비선형 변환을 하면 더 복잡한 패턴을 표현할 수 있습니다. 4배라는 비율은 원래 transformer 논문과 GPT-2가 쓴 값입니다.
- 이 연산은 행마다 독립적입니다. 즉 attention과 달리 위치끼리 정보를 주고받지 않습니다.
- GPT-2는 ReLU 대신 이를 부드럽게 다듬은 GELU를 씁니다. 이 교재는 backward가 더 단순한 ReLU를 사용했고, GELU로 바꾸는 방법은 부록 A에 적었습니다.

4.10 transformer block과 residual 연결

목표: 네 부품을 조립해 block 하나를 만듭니다.

```
/// One transformer block: x + attn(ln(x)), then x + mlp(ln(x)).
pub struct Block {
    ln1: LayerNorm,
    attn: Attention,
    ln2: LayerNorm,
    mlp: Mlp,
}

impl Block {
    pub fn new(p: &mut ParamBuilder, cfg: &Config) -> Self {
        Block {
            ln1: LayerNorm::new(p, cfg.n_embd),
            attn: Attention::new(p, cfg),
            ln2: LayerNorm::new(p, cfg.n_embd),
            mlp: Mlp::new(p, cfg.n_embd),
        }
    }

    pub fn forward(
        &self,
        g: &mut Graph,
        x: T,
        batch: usize,
        seq: usize,
    ) -> T {
        let h = self.ln1.forward(g, x);
        let h = self.attn.forward(g, h, batch, seq);
        let x = g.add(x, h); // residual connection
        let h = self.ln2.forward(g, x);
        let h = self.mlp.forward(g, h);
```

```
        g.add(x, h) // residual connection
    }
}
```

여기서 가장 중요한 줄은 `g.add(x, h)` 입니다. attention의 결과로 x를 **바꾸지 않고**, x에 attention의 결과를 **더합니다**. 이것을 residual 연결이라고 부릅니다.

- **의미**: 각 부품은 벡터를 새로 만드는 것이 아니라 "고칠 부분"만 더합니다. 벡터 x는 모든 block을 관통하는 하나의 통로이고, 각 부품이 그 통로에 정보를 조금씩 보탭니다.
- **학습에 주는 효과**: 2.7절에서 본 대로 덧셈의 backward는 gradient를 그대로 양쪽에 전달합니다. 그래서 block 사이의 residual 통로를 따라 gradient가 곱해지거나 줄어들지 않는 직통 경로가 생기고, 각 block의 계산을 거친 gradient는 이 직통 경로의 값에 더해질 뿐입니다. residual이 없다면 gradient는 층마다 여러 배율을 차례로 곱하며 (0.5절의 chain rule) 전달되고, 작은 배율이 여러 번 곱해지면 앞쪽 층에 도착할 때 거의 0이 될 수 있습니다. residual 연결이 없으면 층을 깊게 쌓았을 때 학습이 잘 되지 않습니다.
- **layernorm의 위치**: layernorm을 부품의 입력 쪽에 둡니다. 이 방식을 pre-norm이라고 부르며, GPT-2 이후 대부분의 LLM이 이 방식을 씁니다. residual 통로 자체는 정규화하지 않으므로 gradient가 정규화를 거치지 않고 그대로 전달됩니다.

확인: `cargo build --release`가 오류 없이 끝나면 4부의 코드는 모두 문법적으로 올바릅니다. 4부에서 만든 부품들은 아직 어디에서도 호출하지 않으며, 5부에서 이 부품들로 GPT를 조립하고 학습시킵니다.

테스트: 4부의 부품이 실제로 동작하는지 작은 입력으로 확인합니다. `src/model.rs`의 맨 끝에 아래 테스트 모듈을 추가합니다.

```
#[cfg(test)]
mod tests {
    use super::*;

    /// A model small enough to build in a few milliseconds.
    fn tiny_config() -> Config {
        Config {
            vocab: 65,
            block_size: 8,
            n_embd: 16,
            n_head: 4,
            n_layer: 2,
        }
    }

    #[test]
    fn block_keeps_shape() {
        let cfg = tiny_config();
        let mut rng = Rng::new(1);
        let mut g = Graph::new();
        let mut p =
            ParamBuilder { g: &mut g, rng: &mut rng, params: Vec::new() };
        let block = Block::new(&mut p, &cfg);

        let (batch, seq) = (2, 8);
        let data = (0..batch * seq * cfg.n_embd)
            .map(|_| rng.normal() as f32)
            .collect();
        let x = g.leaf(data, batch * seq, cfg.n_embd);
```

```
        let y = block.forward(&mut g, x, batch, seq);
        assert_eq!(g.shape(y), (batch * seq, cfg.n_embd));

        // Backward must run through every op without hitting todo!().
        let targets = vec![0; batch * seq];
        let loss = g.cross_entropy(y, &targets);
        g.backward(loss);
    }
}
```

- 테스트가 빨리 끝나도록 작은 설정(열 16개, head 4개)을 씁니다.
- 무작위 입력을 block에 통과시킨 뒤, 출력의 크기가 입력과 같은지 확인합니다. residual 연결이 있으므로 block의 입력과 출력은 크기가 같아야 합니다.
- 마지막 세 줄은 cross entropy로 loss를 만들고 backward를 실행합니다. block 안의 연산 중 하나라도 backward가 빠져 있으면 2.4절의 `todo!`에 걸려 테스트가 실패합니다. 여기서 정답 번호는 의미가 없고, backward를 실행하기 위한 형식일 뿐입니다.
- `ParamBuilder`는 `g`와 `rng`를 빌려 갑니다. block을 만든 뒤에는 `p`를 다시 쓰지 않으므로, 그 뒤로는 `g`와 `rng`를 직접 쓸 수 있습니다.

확인: `cargo test --release -q block_keeps_shape`를 실행하면 `test result: ok. 1 passed`가 출력됩니다.

5부 GPT 조립과 학습

5부가 끝나면 파라미터 210,497개짜리 GPT가 Mac CPU에서 약 5분 동안 학습되고, validation loss가 4.15에서 1.99로 내려갑니다. 학습이 끝나면 모델이 대본 형식의 셰익스피어풍 문장을 500글자 생성합니다.

5.1 GPT 조립 (`src/model.rs`)

목표: 4부의 부품을 4.1절의 순서대로 연결합니다. 이 코드는 `src/model.rs`에서 4.10절의 테스트 모듈 바로 앞에 추가합니다.

```
/// The whole model.
pub struct Gpt {
    pub cfg: Config,
    tok_emb: T, // vocab x n_embd
    pos_emb: T, // block_size x n_embd
    blocks: Vec<Block>,
    ln_f: LayerNorm,
    head: Linear, // n_embd -> vocab
    pub params: Vec<T>,
}

impl Gpt {
    /// Create every parameter. Call this on an empty graph so the
    /// parameters occupy the first `params.len()` slots.
    pub fn new(g: &mut Graph, cfg: Config, rng: &mut Rng) -> Self {
        let mut p = ParamBuilder { g, rng, params: Vec::new() };
        let tok_emb = p.normal(cfg.vocab, cfg.n_embd, 0.02);
        let pos_emb = p.normal(cfg.block_size, cfg.n_embd, 0.02);
        let blocks =
            (0..cfg.n_layer).map(|_| Block::new(&mut p, &cfg)).collect();
        let ln_f = LayerNorm::new(&mut p, cfg.n_embd);
        let head = Linear::new(&mut p, cfg.n_embd, cfg.vocab);
        let params = p.params;
        Gpt { cfg, tok_emb, pos_emb, blocks, ln_f, head, params }
    }

    /// Total number of learned numbers.
    pub fn num_params(&self, g: &Graph) -> usize {
        self.params.iter().map(|&t| g.data(t).len()).sum()
    }
```

- `tok_emb`은 토큰별 벡터를 담은 표입니다. 65개 문자 각각에 길이 64의 벡터가 하나씩 있습니다. 이 숫자들은 처음에는 무작위이지만 가중치처럼 학습됩니다. 학습이 진행되면 비슷하게 쓰이는 문자는 비슷한 벡터를 갖는 경향이 생기고, 4.6절의 attention은 바로 이런 벡터에서 만든 q와 k의 내적으로 위치끼리의 관련성을 잽니다.
- `pos_emb`은 위치별 벡터를 담은 표입니다. attention의 점수 계산 자체에는 위치 정보가 들어 있지 않으므로, 이 벡터가 없으면 모델이 "바로 앞 글자"와 "열 글자 앞"을 구분하기가 매우 어려워집니다. causal mask 때문에 위치마다 볼 수 있는 글자 수가 달라서 간접적인 단서는 남지만, 위치 벡터를 더해 주면 이 정보를 직접 전달할 수 있습니다.
- `Gpt::new`는 모든 파라미터를 그래프의 맨 앞에 만듭니다. 그래서 이 함수가 끝난 직후의 `g.len()`이 파라미터 노드의 개수이며, 1부와 같은 방식으로 `truncate`의 기준이 됩니다.

- `let params = p.params;`는 `ParamBuilder`에서 목록을 꺼냅니다. 이 줄 뒤로는 `p`를 쓰지 않으므로, `p`가 빌렸던 `g`와 `rng`를 호출한 쪽에서 다시 쓸 수 있습니다.
- `num_params`는 모든 파라미터 행렬의 원소 수를 더합니다. `params.len()`은 행렬의 개수이고, 이 함수의 값은 학습되는 숫자의 개수입니다.

```
    /// ids: batch * seq token ids -> logits: (batch * seq) x vocab.
    pub fn forward(
        &self,
        g: &mut Graph,
        ids: &[usize],
        batch: usize,
        seq: usize,
    ) -> T {
        assert!(
            seq <= self.cfg.block_size,
            "sequence longer than block_size"
        );
        let positions: Vec<usize> =
            (0..batch).flat_map(|_| 0..seq).collect();
        let tok = g.embedding(self.tok_emb, ids);
        let pos = g.embedding(self.pos_emb, &positions);
        let mut x = g.add(tok, pos);
        for block in &self.blocks {
            x = block.forward(g, x, batch, seq);
        }
        let x = self.ln_f.forward(g, x);
        self.head.forward(g, x)
    }
}
```

- `positions`는 `[0, 1, ..., seq-1]`을 batch번 반복한 목록입니다. 모든 문장이 0번 위치부터 시작하기 때문입니다. 이 목록을 토큰 번호처럼 `embedding`에 넣으면 위치 벡터 행렬이 나옵니다.
- 토큰 벡터와 위치 벡터는 이어 붙이지 않고 더합니다. 3 + 5에서 3과 5를 되찾을 수 없는 것처럼 보이지만, 여기서 더하는 것은 64칸짜리 벡터이고 두 벡터 모두 학습됩니다. 그래서 모델은 두 정보를 서로 다른 방향(칸들의 조합)에 나눠 담는 방법을 배울 수 있습니다.
- `assert!`는 block_size보다 긴 문장을 막습니다. `pos_emb`에는 block_size개의 행밖에 없으므로, 그보다 뒤의 위치는 표현할 수 없습니다.
- 출력은 각 위치의 "다음 토큰 점수"입니다. softmax를 적용하기 전의 점수를 **logits**라고 부릅니다. 이 값을 2.13절의 `cross_entropy`에 바로 넣습니다.

테스트: 4.10절의 `mod tests` 안에서 닫는 `}` 바로 앞에 아래 테스트를 추가합니다.

```
    #[test]
    fn initial_loss_is_near_ln_vocab() {
        let cfg = tiny_config();
        let mut rng = Rng::new(1);
        let mut g = Graph::new();
        let model = Gpt::new(&mut g, cfg, &mut rng);

        let (batch, seq) = (2, 8);
        let ids: Vec<usize> =
            (0..batch * seq).map(|_| rng.below(cfg.vocab)).collect();
        let targets: Vec<usize> =
```

```
            (0..batch * seq).map(|_| rng.below(cfg.vocab)).collect();
        let logits = model.forward(&mut g, &ids, batch, seq);
        assert_eq!(g.shape(logits), (batch * seq, cfg.vocab));

        let loss = g.cross_entropy(logits, &targets);
        let value = g.data(loss)[0];
        println!(
            "initial loss {value:.4}, ln(vocab) {:.4}",
            (cfg.vocab as f32).ln()
        );
        assert!((value - (cfg.vocab as f32).ln()).abs() < 0.1);
        g.backward(loss);
    }
```

- 학습하기 전의 모델은 가중치가 아주 작아서 모든 문자에 거의 같은 점수를 줍니다. 그래서 무작위 정답에 대한 loss가 2.13절에서 계산한 ln 65 ≈ 4.17에 가까워야 합니다.
- logits의 크기가 (batch × seq) × vocab인지도 함께 확인합니다.

확인: `cargo test --release -q initial_loss -- --nocapture`를 실행하면 아래와 같이 출력됩니다. `--nocapture`는 테스트 안의 `println!` 출력을 보여 달라는 옵션입니다.

```
initial loss 4.1737, ln(vocab) 4.1744
.
test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 9 filtered out; finished in 0.00s
```

5.2 AdamW optimizer (`src/optim.rs`)

목표: 1부의 단순한 경사하강보다 훨씬 빨리 학습되는 방법으로 파라미터를 수정합니다.

단순한 경사하강에는 두 가지 약점이 있습니다.

1. **방향이 흔들립니다.** 배치는 전체 데이터 중 일부이므로, 배치마다 gradient의 방향이 조금씩 다릅니다.
2. **모든 파라미터가 같은 lr을 씁니다.** gradient가 항상 큰 파라미터는 너무 크게 움직이고, 항상 작은 파라미터는 거의 움직이지 않습니다.

Adam은 파라미터마다 두 가지 값을 기억해서 이 문제를 해결합니다.

이름	기억하는 것	해결하는 문제
m	gradient의 이동 평균	흔들림을 평균 내서 방향을 안정시킵니다
v	gradient 제곱의 이동 평균	√v로 나눠서 파라미터마다 움직이는 폭을 비슷하게 맞춥니다

이동 평균은 "예전 평균을 대부분 유지하고 새 값을 조금 섞는" 방식으로 구합니다. $\beta_1 = 0.9$라면 예전 평균을 90%, 새 gradient를 10% 섞습니다. 이 교재는 $\beta_1 = 0.9$, $\beta_2 = 0.99$를 씁니다.

$m \leftarrow \beta_1 m + (1 - \beta_1) g$

$v \leftarrow \beta_2 v + (1 - \beta_2) g^2$

$w \leftarrow w - lr \times (\hat{m} \div (\sqrt{\hat{v}} + \varepsilon) + \lambda w)$

m과 v는 0에서 시작하므로, 학습 초반에는 실제보다 0에 가까운 값을 갖습니다. $\beta_1 = 0.9$인 m은 수십 단계, $\beta_2 = 0.99$인 v는 수백 단계 동안 이 치우침이 남습니다. t번째 단계의 보정은 $\hat{m} = m \div (1 - \beta_1^t)$, $\hat{v} = v \div (1 - \beta_2^t)$입니다. $\hat{m}$과 $\hat{v}$는 이 치우침을 보정한 값입니다. 마지막 항 λw는 weight decay입니다. 매 단계 가중치를 0 쪽으로 조금씩 당겨서, 가중치가 불필요하게 커지는 것을 막습니다. λw를 gradient g에 미리 더하면 이 항도 m과 v를 거쳐 $\sqrt{\hat{v}}$로 나눠지므로, 파라미터마다 당기는 세기가 제각각 달라집니다. AdamW는 λw를 $\hat{m} \div \sqrt{\hat{v}}$ 계산과 분리해서 마지막에 따로 더하므로, 모든 가중치가 같은 비율(lr × λ)로 당겨집니다. 이렇게 분리한(decoupled) 형태를 AdamW라고 부릅니다.

숫자로 보기: gradient가 항상 0.001인 파라미터 A와 항상 10인 파라미터 B가 있고, lr = 0.001이라고 하겠습니다. 비교를 쉽게 하기 위해 weight decay와 ε은 빼고 첫 단계만 계산합니다.

계산	A (gradient 0.001)	B (gradient 10)
경사하강의 한 걸음: lr × g	0.000001	0.01
$m = 0.1 \times g$	0.0001	1
보정한 $\hat{m} = m \div (1 - 0.9)$	0.001	10
$v = 0.01 \times g^2$	0.00000001	1
보정한 $\hat{v} = v \div (1 - 0.99)$	0.000001	100
Adam의 한 걸음: lr × $\hat{m} \div \sqrt{\hat{v}}$	0.001	0.001

- 경사하강에서는 B가 A보다 10,000배 크게 움직입니다. Adam에서는 두 파라미터가 같은 폭(lr)으로 움직입니다. $\hat{m} \div \sqrt{\hat{v}}$가 "gradient ÷ gradient의 크기", 즉 방향만 남기기 때문입니다.
- 보정을 하지 않으면 첫 단계의 m은 실제 gradient의 10%밖에 되지 않습니다. 보정 계수 1 − 0.9 = 0.1로 나누면 정확히 원래 크기로 돌아옵니다. 단계가 쌓일수록 보정 계수는 1에 가까워져서 보정이 저절로 사라집니다.

```
use crate::tensor::{Graph, T};

/// Adam with decoupled weight decay.
pub struct AdamW {
    pub lr: f32,
    beta1: f32,
    beta2: f32,
    eps: f32,
    weight_decay: f32,
    t: i32,          // steps taken so far
    m: Vec<Vec<f32>>, // running mean of gradients, one Vec per parameter
    v: Vec<Vec<f32>>, // running mean of squared gradients
}

impl AdamW {
    pub fn new(g: &Graph, params: &[T], lr: f32) -> Self {
        let zeros =
            || params.iter().map(|&p| vec![0.0; g.data(p).len()]).collect();
        AdamW {
            lr,
            beta1: 0.9,
            beta2: 0.99,
            eps: 1e-8,
            weight_decay: 0.1,
            t: 0,
            m: zeros(),
```

```
            v: zeros(),
        }
    }

    pub fn step(&mut self, g: &mut Graph, params: &[T]) {
        self.t += 1;
        // Early on m and v are biased toward 0; these factors undo that.
        let bc1 = 1.0 - self.beta1.powi(self.t);
        let bc2 = 1.0 - self.beta2.powi(self.t);
        for (i, &p) in params.iter().enumerate() {
            // Decay matrices only; biases and LayerNorm gains are 1 x n.
            let decay =
                if g.shape(p).0 > 1 { self.weight_decay } else { 0.0 };
            let (data, grad) = g.data_and_grad(p);
            let (m, v) = (&mut self.m[i], &mut self.v[i]);
            for j in 0..data.len() {
                m[j] = self.beta1 * m[j] + (1.0 - self.beta1) * grad[j];
                v[j] = self.beta2 * v[j]
                    + (1.0 - self.beta2) * grad[j] * grad[j];
                let m_hat = m[j] / bc1;
                let v_hat = v[j] / bc2;
                data[j] -= self.lr
                    * (m_hat / (v_hat.sqrt() + self.eps) + decay * data[j]);
            }
        }
    }
}
```

- m과 v는 파라미터 행렬마다 같은 크기의 `Vec`을 하나씩 둡니다. 그래서 optimizer가 차지하는 메모리는 파라미터의 2배입니다.
- `bc1`과 `bc2`는 초반 보정 계수입니다. 첫 단계에서 `bc1`은 0.1이므로, m을 10배로 키워서 "새 gradient를 10%만 섞은" 효과를 되돌립니다. 단계가 쌓일수록 1에 가까워져 보정이 사라집니다.
- weight decay는 행이 2개 이상인 행렬(가중치와 embedding)에만 적용합니다. bias와 layernorm의 gamma, beta는 모두 1 × n이고, 이런 값을 0으로 당기면 학습에 방해가 되기 때문입니다. nanoGPT는 "2차원 이상인 tensor만 decay한다"는 기준을 쓰며, 이 교재에서는 모든 tensor가 2차원이므로 "행이 2개 이상"으로 바꿔 같은 파라미터를 고릅니다.
- `let (m, v) = (&mut self.m[i], &mut self.v[i]);`는 서로 다른 필드의 가변 참조 두 개를 동시에 만듭니다. 반복문 안에서 `self.beta1` 같은 다른 필드를 읽는 것도 허용됩니다.

5.3 평가와 생성 (`src/main.rs`)

목표: 학습 중간에 모델을 평가하고, 학습이 끝나면 문장을 생성하는 함수를 만듭니다. 5.3절과 5.4절의 코드는 모두 `src/main.rs`에 들어갑니다. 완성된 파일 전체는 5.4절 끝에 있습니다.

먼저 `src/main.rs`의 윗부분을 아래처럼 바꿉니다. 모든 모듈을 선언하고, 필요한 이름을 가져오고, 학습 설정값을 상수로 둡니다. `train` 명령도 미리 추가했습니다. `train` 함수는 5.4절에서 만들므로, 5.4절을 마칠 때까지는 컴파일 오류가 나는 것이 정상입니다.

```
mod data;
mod model;
mod optim;
mod rng;
```

```
mod scalar;
mod tensor;

use data::{get_batch, Tokenizer};
use model::{Config, Gpt};
use optim::AdamW;
use rng::Rng;
use std::time::Instant;
use tensor::{softmax_in_place, Graph};

const BATCH: usize = 16;
const STEPS: usize = 2000;
const EVAL_EVERY: usize = 200;

fn main() {
    let cmd = std::env::args().nth(1).unwrap_or_default();
    match cmd.as_str() {
        "scalar" => scalar::demo(),
        "data" => data_demo(),
        "train" => train(),
        _ => println!("usage: cargo run --release -- [scalar|data|train]"),
    }
}
```

학습에 쓰는 설정값(하이퍼파라미터)은 다음과 같습니다. 일부는 5.4절의 `train` 함수 안에 있습니다.

이름	값	하는 일	바꾸면
`BATCH`	16	한 번에 학습하는 문장 수	키우면 gradient가 덜 흔들리지만 단계당 시간이 늘어납니다
`STEPS`	2000	학습 단계 수	늘리면 loss가 더 내려가지만 시간이 비례해서 늘어납니다
`EVAL_EVERY`	200	몇 단계마다 loss를 출력할지	학습 결과에는 영향이 없습니다
`block_size`	32	모델이 볼 수 있는 문맥 길이	키우면 긴 문맥을 보지만 attention 계산이 길이의 제곱으로 늘어납니다
lr	0.001	AdamW의 한 걸음 크기	3e-3으로 키워 본 한 번의 실행에서는 val loss가 1.99에서 2.03으로 오히려 나빠졌습니다

```
/// Average loss over a few random batches, without training.
fn estimate_loss(
    model: &Gpt,
    g: &mut Graph,
    data: &[usize],
    rng: &mut Rng,
) -> f32 {
    let n_params = g.len();
    let seq = model.cfg.block_size;
    let mut total = 0.0;
    for _ in 0..10 {
        g.truncate(n_params);
        let (x, y) = get_batch(data, BATCH, seq, rng);
        let logits = model.forward(g, &x, BATCH, seq);
        let loss = g.cross_entropy(logits, &y);
        total += g.data(loss)[0];
```

```
    }
    total / 10.0
}
```

- 배치 하나의 loss는 우연에 따라 크게 흔들리므로, 배치 10개의 평균을 냅니다.
- backward와 optimizer를 호출하지 않으므로 파라미터는 바뀌지 않습니다.
- 이 함수는 항상 파라미터만 남은 그래프에서 호출된다고 가정합니다. 그래서 첫 줄의 `g.len()`을 파라미터 개수로 씁니다. 5.4절의 학습 루프가 이 조건을 지킵니다.

```
/// Generate `n` new tokens after `prompt`, one at a time.
fn generate(
    model: &Gpt,
    g: &mut Graph,
    prompt: &[usize],
    n: usize,
    temperature: f32,
    rng: &mut Rng,
) -> Vec<usize> {
    let n_params = g.len();
    let mut ids = prompt.to_vec();
    for _ in 0..n {
        g.truncate(n_params);
        // Only the last block_size tokens fit in the context.
        let start = ids.len().saturating_sub(model.cfg.block_size);
        let context = &ids[start..];
        let logits = model.forward(g, context, 1, context.len());
        // The last row predicts the token that comes next.
        let vocab = model.cfg.vocab;
        let last = &g.data(logits)[(context.len() - 1) * vocab..];
        let mut probs: Vec<f32> =
            last.iter().map(|x| x / temperature).collect();
        softmax_in_place(&mut probs);
        ids.push(sample(&probs, rng));
    }
    ids
}

/// Draw an index with probability probs[i].
fn sample(probs: &[f32], rng: &mut Rng) -> usize {
    let mut r = rng.uniform() as f32;
    for (i, &p) in probs.iter().enumerate() {
        if r < p {
            return i;
        }
        r -= p;
    }
    probs.len() - 1 // rounding left a tiny remainder
}
```

`generate`는 3.1절에서 설명한 방식 그대로 한 글자씩 만듭니다.

1. 지금까지의 토큰 중 마지막 block_size(32)개만 잘라서 문맥으로 씁니다. `saturating_sub`는 빼기 결과가 음수가 되면 0을 돌려주므로, 토큰이 32개보다 적을 때는 전부를 씁니다.
2. batch 1로 forward를 실행합니다. 출력은 위치마다 한 행씩이지만, 필요한 것은 마지막 행(마지막 글자 다음에 올 글자의 점수)뿐입니다.

3. 점수를 temperature로 나눈 뒤 softmax로 확률을 만듭니다.
4. 그 확률대로 글자를 하나 뽑아 목록 끝에 붙이고, 1번으로 돌아갑니다.

temperature는 생성 결과가 얼마나 무작위적인지를 조절합니다. 1보다 작으면 점수 차이가 커져서 높은 점수의 글자가 더 자주 뽑히고, 문장이 안전하지만 단조로워집니다. 1보다 크면 점수 차이가 줄어들어서 낮은 점수의 글자도 자주 나오고, 문장이 다양해지지만 오타가 늘어납니다. 이 교재는 0.8을 씁니다.

`sample`은 0과 1 사이의 난수 r을 하나 뽑고, 확률을 앞에서부터 차례로 빼 나갑니다. 길이 1짜리 막대를 확률만큼의 구간으로 나누고, r이 어느 구간에 떨어졌는지 찾는 것과 같습니다. 확률의 합이 반올림 오차 때문에 1보다 아주 조금 작을 수 있어서, 반복문을 끝까지 돌면 마지막 번호를 돌려줍니다.

5.4 학습 루프와 최종 `main.rs`

목표: 지금까지 만든 모든 것을 연결해 GPT를 학습시킵니다.

```
fn train() {
    let text =
        std::fs::read_to_string("input.txt").expect("input.txt not found");
    let tok = Tokenizer::new(&text);
    let data = tok.encode(&text);
    let split = data.len() * 9 / 10;
    let (train_data, val_data) = (&data[..split], &data[split..]);

    let cfg = Config {
        vocab: tok.vocab_size(),
        block_size: 32,
        n_embd: 64,
        n_head: 4,
        n_layer: 4,
    };
    let mut rng = Rng::new(1337);
    let mut g = Graph::new();
    let model = Gpt::new(&mut g, cfg, &mut rng);
    let n_params = g.len();
    println!("parameters: {}", model.num_params(&g));

    let mut opt = AdamW::new(&g, &model.params, 1e-3);
    let start = Instant::now();
    for step in 0..=STEPS {
        if step % EVAL_EVERY == 0 {
            let train_loss =
                estimate_loss(&model, &mut g, train_data, &mut rng);
            let val_loss =
                estimate_loss(&model, &mut g, val_data, &mut rng);
            let secs = start.elapsed().as_secs_f32();
            print!("step {step:5}  train {train_loss:.3}  ");
            println!("val {val_loss:.3}  ({secs:.0}s)");
        }
        if step == STEPS {
            break;
        }
        g.truncate(n_params);
        let (x, y) = get_batch(train_data, BATCH, cfg.block_size, &mut rng);
        let logits = model.forward(&mut g, &x, BATCH, cfg.block_size);
        let loss = g.cross_entropy(logits, &y);
        g.backward(loss);
```

```
        opt.step(&mut g, &model.params);
    }

    let prompt = tok.encode("\n");
    let out = generate(&model, &mut g, &prompt, 500, 0.8, &mut rng);
    println!("{}", tok.decode(&out));
}
```

- **데이터 나누기**: 앞의 90%는 학습에, 뒤의 10%는 평가에 씁니다. 모델이 학습 데이터를 외워버리면 train loss는 계속 내려가지만 val loss는 오히려 올라갑니다. 이 현상을 과적합(overfitting)이라고 부르며, val loss를 보면 그 시점을 알 수 있습니다.
- **학습 한 단계**: 1부의 학습 루프와 순서가 같습니다. 파라미터 뒤를 지우고(`truncate`), forward로 loss를 계산하고, backward로 gradient를 구하고, optimizer로 파라미터를 수정합니다.
- **평가 시점**: `0..=STEPS`는 STEPS를 포함하는 범위입니다. 마지막 반복에서는 평가만 하고 학습하지 않은 채 반복문을 빠져나오므로, 학습이 끝난 상태의 loss까지 출력됩니다.
- **생성**: 줄바꿈 문자 하나를 시작점으로 주고 500글자를 생성합니다.

완성된 `src/main.rs` 전체는 아래와 같습니다. 위에서부터 5.3절의 윗부분, 3.4절의 `data_demo`, 5.3절의 `estimate_loss`, `generate`, `sample`, 그리고 이 절의 `train` 순서입니다.

```
mod data;
mod model;
mod optim;
mod rng;
mod scalar;
mod tensor;

use data::{get_batch, Tokenizer};
use model::{Config, Gpt};
use optim::AdamW;
use rng::Rng;
use std::time::Instant;
use tensor::{softmax_in_place, Graph};

const BATCH: usize = 16;
const STEPS: usize = 2000;
const EVAL_EVERY: usize = 200;

fn main() {
    let cmd = std::env::args().nth(1).unwrap_or_default();
    match cmd.as_str() {
        "scalar" => scalar::demo(),
        "data" => data_demo(),
        "train" => train(),
        _ => println!("usage: cargo run --release -- [scalar|data|train]"),
    }
}

fn data_demo() {
    let text =
        std::fs::read_to_string("input.txt").expect("input.txt not found");
    let tok = Tokenizer::new(&text);
    println!(
        "characters: {}  vocab: {}",
        text.chars().count(),
```

```
        tok.vocab_size()
    );
    let ids = tok.encode("Hello!");
    println!(
        "encode(\"Hello!\") = {ids:?}  decode -> {:?}",
        tok.decode(&ids)
    );

    let data = tok.encode(&text);
    let mut rng = Rng::new(1);
    let (x, y) = get_batch(&data, 1, 16, &mut rng);
    println!("input : {:?}", tok.decode(&x));
    println!("target: {:?}", tok.decode(&y));
}

/// Average loss over a few random batches, without training.
fn estimate_loss(
    model: &Gpt,
    g: &mut Graph,
    data: &[usize],
    rng: &mut Rng,
) -> f32 {
    let n_params = g.len();
    let seq = model.cfg.block_size;
    let mut total = 0.0;
    for _ in 0..10 {
        g.truncate(n_params);
        let (x, y) = get_batch(data, BATCH, seq, rng);
        let logits = model.forward(g, &x, BATCH, seq);
        let loss = g.cross_entropy(logits, &y);
        total += g.data(loss)[0];
    }
    total / 10.0
}

/// Generate `n` new tokens after `prompt`, one at a time.
fn generate(
    model: &Gpt,
    g: &mut Graph,
    prompt: &[usize],
    n: usize,
    temperature: f32,
    rng: &mut Rng,
) -> Vec<usize> {
    let n_params = g.len();
    let mut ids = prompt.to_vec();
    for _ in 0..n {
        g.truncate(n_params);
        // Only the last block_size tokens fit in the context.
        let start = ids.len().saturating_sub(model.cfg.block_size);
        let context = &ids[start..];
        let logits = model.forward(g, context, 1, context.len());
        // The last row predicts the token that comes next.
        let vocab = model.cfg.vocab;
        let last = &g.data(logits)[(context.len() - 1) * vocab..];
        let mut probs: Vec<f32> =
            last.iter().map(|x| x / temperature).collect();
        softmax_in_place(&mut probs);
        ids.push(sample(&probs, rng));
    }
    ids
```

```
}

/// Draw an index with probability probs[i].
fn sample(probs: &[f32], rng: &mut Rng) -> usize {
    let mut r = rng.uniform() as f32;
    for (i, &p) in probs.iter().enumerate() {
        if r < p {
            return i;
        }
        r -= p;
    }
    probs.len() - 1 // rounding left a tiny remainder
}

fn train() {
    let text =
        std::fs::read_to_string("input.txt").expect("input.txt not found");
    let tok = Tokenizer::new(&text);
    let data = tok.encode(&text);
    let split = data.len() * 9 / 10;
    let (train_data, val_data) = (&data[..split], &data[split..]);

    let cfg = Config {
        vocab: tok.vocab_size(),
        block_size: 32,
        n_embd: 64,
        n_head: 4,
        n_layer: 4,
    };
    let mut rng = Rng::new(1337);
    let mut g = Graph::new();
    let model = Gpt::new(&mut g, cfg, &mut rng);
    let n_params = g.len();
    println!("parameters: {}", model.num_params(&g));

    let mut opt = AdamW::new(&g, &model.params, 1e-3);
    let start = Instant::now();
    for step in 0..=STEPS {
        if step % EVAL_EVERY == 0 {
            let train_loss =
                estimate_loss(&model, &mut g, train_data, &mut rng);
            let val_loss =
                estimate_loss(&model, &mut g, val_data, &mut rng);
            let secs = start.elapsed().as_secs_f32();
            print!("step {step:5}  train {train_loss:.3}  ");
            println!("val {val_loss:.3}  ({secs:.0}s)");
        }
        if step == STEPS {
            break;
        }
        g.truncate(n_params);
        let (x, y) = get_batch(train_data, BATCH, cfg.block_size, &mut rng);
        let logits = model.forward(&mut g, &x, BATCH, cfg.block_size);
        let loss = g.cross_entropy(logits, &y);
        g.backward(loss);
        opt.step(&mut g, &model.params);
    }

    let prompt = tok.encode("\n");
    let out = generate(&model, &mut g, &prompt, 500, 0.8, &mut rng);
```

```
    println!("{}", tok.decode(&out));
}
```

확인: `cargo run --release -- train`을 실행하면 아래와 같이 출력됩니다. Apple M1 Mac에서 약 5분이 걸렸고, 시간은 컴퓨터마다 다릅니다. seed를 고정했으므로 같은 코드라면 loss와 생성된 문장이 같게 나옵니다. 다만 exp, ln 같은 수학 함수는 운영체제의 라이브러리마다 마지막 자리가 아주 조금 다를 수 있습니다. 이 작은 차이가 2000단계 동안 쌓이면 loss의 뒷자리가 달라지고, 생성 단계에서 글자 하나만 다르게 뽑혀도 그 뒤의 문장 전체가 달라질 수 있습니다. loss가 내려가지 않거나 NaN이 나오면 부록 D를 참고합니다.

```
parameters: 210497
step     0  train 4.156  val 4.151  (0s)
step   200  train 2.470  val 2.466  (29s)
step   400  train 2.344  val 2.354  (58s)
step   600  train 2.229  val 2.255  (87s)
step   800  train 2.153  val 2.182  (116s)
step  1000  train 2.042  val 2.086  (146s)
step  1200  train 2.019  val 2.107  (175s)
step  1400  train 1.962  val 2.027  (204s)
step  1600  train 1.943  val 2.026  (233s)
step  1800  train 1.871  val 1.999  (262s)
step  2000  train 1.846  val 1.991  (292s)

Came, twan come the still to their.
ISAn I prom the calint but to but the deather of so, tar,
Tooward hold proy aced men which fator and
The whower not hime, prochious'd the here.
That I king nowice Godd a drows with your for how'll.

BENCELO:
What nayir to somle, hith in now all him
Where fare os noble o' world cher
Of the coused should the tout with of Strows.

RICHARDIA:
whome, theak do crown Rominagys come.
...
```

출력을 읽는 방법은 다음과 같습니다.

- **step 0의 loss 4.15**는 2.13절에서 계산한 무작위 추측값 ln 65 ≈ 4.17과 거의 같습니다. 초기화가 제대로 되었다는 뜻입니다.
- **train과 val이 함께 내려갑니다.** 두 값의 차이는 학습이 진행될수록 조금씩 벌어지지만(0.004 → 0.145), val loss가 여전히 내려가고 있으므로 아직 과적합 단계는 아닙니다. step 1200에서 val이 잠깐 올라간 것은 배치 10개만으로 loss를 추정해서 생기는 흔들림입니다. 과적합은 한 번의 상승이 아니라 val loss가 여러 번 연속으로 올라가는 추세로 판단합니다. 최종 val loss 1.99는 정답 글자에 준 확률을 기하평균으로 나타내면 e^(−1.99) ≈ 0.14, 즉 약 14%라는 뜻입니다. 학습 전(1/65 ≈ 1.5%)보다 약 9배 높아졌습니다.
- **생성된 문장**을 보면 모델이 대본의 형식(대문자 인물 이름과 콜론, 줄바꿈, 단락 구분)과 `the`, `noble`, `crown` 같은 짧은 단어를 배웠음을 알 수 있습니다. 반면 문법과 긴 단어는 아직 틀리는 경우가 많습니다. 모델이 볼 수 있는 문맥이 32글자뿐이고 학습도 2000단계만 했기 때문입니다.

여기까지 오면 GPT의 모든 구성 요소를 직접 만든 것입니다. 실제 LLM도 구조의 핵심은 같습니다. 차이는 모델 크기, 데이터 양, 그것을 감당하기 위한 속도 최적화, 그리고 RoPE(회전 위치 표현), RMSNorm, SwiGLU 같은 몇 가지 세부

선택입니다. 또 ChatGPT 같은 대화형 모델은 이렇게 다음 토큰 맞히기로 학습한 모델을 질문과 답변 예시로 다시 학습시키고(instruction tuning), 사람의 선호를 반영하는 추가 학습(RLHF 등)을 거친 것입니다. 이 교재의 모델이 대본 흉내만 내는 이유도 이 추가 학습이 없기 때문입니다.

부록

본문을 마친 뒤 이어서 해 볼 만한 작업과, 실습 중에 자주 막히는 지점을 정리했습니다. A의 코드는 직접 컴파일하고 테스트해서 확인했고, B와 C는 방향만 제시합니다.

A. ReLU를 GELU로 바꾸기

GPT-2는 MLP에서 GELU를 씁니다. GELU는 ReLU와 비슷하게 음수를 억누르지만, 0 근처에서 꺾이지 않고 부드럽게 휘어집니다. 아래 네 곳을 고치면 됩니다.

1. `Op`에 `Gelu(usize),`를 추가합니다.
2. `impl Graph` 바깥에 값과 도함수를 함께 계산하는 함수를 추가하고, `impl Graph` 안에 forward 연산을 추가합니다.

```
/// GELU value and its derivative at x.
fn gelu_parts(x: f32) -> (f32, f32) {
    const C: f32 = 0.797_884_6; // sqrt(2 / pi)
    let u = C * (x + 0.044715 * x * x * x);
    let t = u.tanh();
    let du = C * (1.0 + 3.0 * 0.044715 * x * x);
    let y = 0.5 * x * (1.0 + t);
    let dy = 0.5 * (1.0 + t) + 0.5 * x * (1.0 - t * t) * du;
    (y, dy)
}
```

```
    /// GELU, tanh approximation (as in GPT-2).
    pub fn gelu(&mut self, a: T) -> T {
        let out = self.data(a).iter().map(|&x| gelu_parts(x).0).collect();
        let (n, m) = self.shape(a);
        self.push(out, n, m, Op::Gelu(a.0))
    }
```

3. `input_grads`의 `match`에 backward를 추가합니다.

```
            Op::Gelu(a) => {
                let ad = &self.nodes[*a].data;
                let da: Vec<f32> = dy
                    .iter()
                    .zip(ad)
                    .map(|(g, &x)| g * gelu_parts(x).1)
                    .collect();
                vec![(*a, da)]
            }
```

4. `Mlp::forward`의 `g.relu(h)`를 `g.gelu(h)`로 바꾸고, 2.8절의 `relu` 테스트를 복사해 `gelu` 테스트를 만듭니다. 이렇게 바꾼 코드에서 11개 테스트(4.10절과 5.1절의 테스트 2개 포함)가 모두 통과하는 것을 확인했습니다. 더 이상 쓰지 않는 `relu`와 Op::Relu에는 "never used"와 "never constructed" 경고가 나옵니다. 둘 다 지워도 되고 남겨 두어도 됩니다.

새 연산을 추가할 때는 항상 이 순서를 따르면 됩니다. **forward 작성 → backward 작성 → gradient 검사 테스트 추가**입니다.

B. 속도를 높이는 방법

이 교재의 코드는 읽기 쉬움을 위해 속도를 포기한 부분이 많습니다. 효과가 클 것으로 예상되는 순서대로 적었지만, 바꾸기 전에 항상 측정부터 해야 합니다. macOS에서는 `cargo instruments`나 `samply` 같은 profiler로 어느 함수가 시간을 쓰는지 볼 수 있습니다.

1. **행렬 곱셈 병렬화**: 계산량 대부분은 `matmul`과 `back_matmul`에 있습니다. forward 결과의 각 행은 서로 독립적이므로 `rayon` crate의 `par_chunks_mut`로 행을 여러 코어에 나눠 줄 수 있습니다. backward에서는 dB가 모든 행의 기여를 더한 값이므로, dA와 dB를 각각 행렬 곱셈(dY · B^T와 A^T · dY)으로 나눠 따로 병렬화해야 합니다.
2. **메모리 할당 줄이기**: 학습 한 단계마다 수천 개의 `Vec`을 만들고 버립니다. 노드의 메모리를 재사용하거나, backward에서 새 `Vec`을 만들지 않고 바로 더하도록 `split_at_mut`를 쓰는 방법이 있습니다.
3. **attention 묶기**: 지금은 batch × head마다 작은 연산 아홉 개가 따로 만들어집니다. attention 전체를 하나의 `Op`로 만들고 backward를 직접 작성하면 노드 수와 복사가 크게 줄어듭니다.
4. **생성용 KV cache**: `generate`는 글자 하나를 만들 때마다 문맥 전체를 다시 계산합니다. 문맥이 block_size보다 짧은 동안에는 이전 위치의 k와 v가 바뀌지 않으므로, 저장해 두고 새 위치의 k와 v만 계산할 수 있습니다. 다만 이 교재의 generate처럼 문맥을 마지막 32글자로 밀어 가며 자르고 위치 embedding을 더하는 구조에서는, 문맥이 밀리는 순간 모든 글자의 위치 번호가 바뀌어 저장해 둔 k와 v를 그대로 쓸 수 없습니다. 실제 LLM은 위치 표현 방식을 바꿔 이 문제를 피합니다. 실제 LLM 추론 서버가 모두 쓰는 기법입니다.

C. 더 나아가기

- **GPT-2 가중치 불러오기**: 이 교재의 모델은 골격이 GPT-2와 같으므로, 실제 GPT-2 small의 가중치를 불러와 문장을 생성하는 것이 좋은 다음 목표입니다. Hugging Face에서 받을 수 있는 `model.safetensors`는 "8바이트 길이 + JSON 헤더 + 원본 숫자" 형식이라 외부 crate 없이도 읽을 수 있습니다. GPT-2 가중치는 OpenAI가 공개한 것이므로, 받아서 쓸 때는 해당 모델의 라이선스를 확인합니다. 이때 필요한 변경은 다음과 같습니다.
  - 부록 A의 GELU를 씁니다.
  - 문자 단위 tokenizer 대신 GPT-2의 BPE tokenizer를 구현합니다. 이 작업이 가장 큽니다.
  - 출력 head는 bias가 없고, 토큰 embedding 행렬을 전치해서 그대로 씁니다(weight tying).
  - 가중치를 불러오기만 하면 backward가 필요 없으므로, 2부의 forward 연산만 있으면 됩니다. 얻은 logits를 PyTorch의 결과와 비교하면 구현이 맞는지 확인할 수 있습니다.
- **프레임워크로 옮기기**: 직접 만든 autograd를 이해했다면, candle이나 burn으로 같은 모델을 다시 작성해 보기 바랍니다. 4부와 5부의 구조는 그대로 옮길 수 있고, batch와 head를 나누던 반복문은 3차원·4차원 tensor 연산 한 번으로 바뀝니다. GPU(Mac에서는 Metal)를 쓸 수 있어 훨씬 큰 모델을 학습할 수 있습니다.
- **참고 자료와 감사의 말**: 이 교재의 1부는 Andrej Karpathy의 micrograd(MIT License)를, 4부와 5부는 같은 저자의 nanoGPT(MIT License)를 설계의 참고로 삼았습니다. 코드는 Rust로 새로 작성했으며 원본 코드를 복사하지 않았습니다. nanoGPT는 현재 저자가 더 이상 관리하지 않는 저장소로 표시되어 있으며, 후속 프로젝트로 nanochat을 안내하고 있습니다. 같은 일을 다른 언어로 어떻게 작성했는지 비교하며 읽으면 이해에 도움이 됩니다.

- **기법의 원 출처**: Transformer(Vaswani 외, 2017), GPT-2(Radford 외, 2019), GELU(Hendrycks와 Gimpel, 2016), AdamW(Loshchilov와 Hutter, 2019), xorshift(Marsaglia, 2003), Box-Muller 변환(Box와 Muller, 1958).

D. 자주 막히는 지점

증상	가장 흔한 원인	확인할 것
학습이 수십 배 느리다	debug 모드로 실행했다	`cargo run`에 `--release`를 붙였는지 확인합니다.
`input.txt not found`	다른 폴더에서 실행했다	`Cargo.toml`이 있는 폴더에서 실행합니다.
loss가 NaN이 된다	softmax에서 최댓값을 빼지 않아 점수가 커질 때 overflow가 났거나, 초기화나 forward 코드에 버그가 있다	2.10절의 `softmax_in_place`와 비교하고, 초기값이 표준편차 0.02인지와 layernorm의 ε이 빠지지 않았는지 확인합니다.
loss가 줄다가 갑자기 NaN이 된다	learning rate가 너무 크다	lr을 절반으로 줄여 봅니다.
loss가 4.17 근처에서 내려가지 않는다	파라미터가 수정되지 않는다	`opt.step`을 호출하는지, `n_params`를 `Gpt::new` 직후에 구했는지 확인합니다.
학습 도중 메모리가 계속 늘어난다	그래프를 지우지 않는다	학습 단계마다 `g.truncate(n_params)`를 호출하는지 확인합니다.
gradient 검사가 원소 하나에서만 실패한다	ReLU처럼 꺾인 점 근처의 값이다	입력값이 0과 ε(0.01) 이내로 가까운지 확인합니다. 그렇다면 backward 버그가 아닙니다.
gradient 검사가 여러 원소에서 크게 틀린다	backward의 인덱스나 수식이 틀렸다	행 우선 인덱스(`r * cols + c`)에서 rows와 cols가 바뀌지 않았는지부터 봅니다.
"cannot borrow as mutable" 오류	같은 `Vec`에서 읽기와 쓰기를 동시에 하려 했다	2.4절처럼 새 `Vec`에 계산한 뒤 따로 더합니다.
val loss만 올라간다	과적합이다	모델을 줄이거나 학습 단계를 줄입니다.